

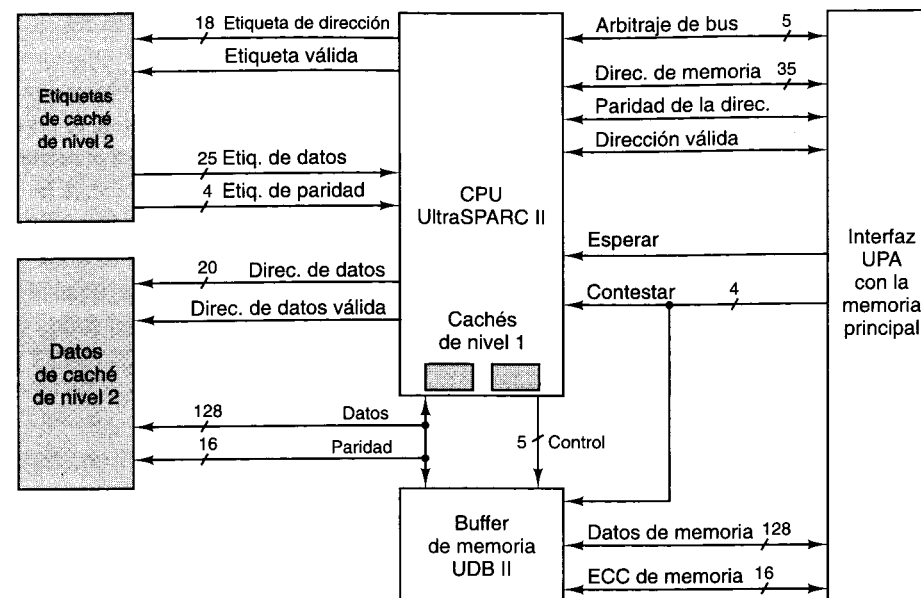


Figura 3-47. Principales características del corazón de un sistema UltraSPARC II.

64 bytes. Las 256 líneas de instrucciones que más se usan y las 256 líneas de datos que más se usan están en la caché nivel 1. Las líneas de caché que se usan mucho pero que no caben en la caché de nivel 1 se guardan en la caché de nivel 2. Éste contiene líneas de datos y de instrucciones mezcladas aleatoriamente, y se almacenan en el rectángulo rotulado "Datos de caché de nivel 2". El sistema tiene que saber cuáles líneas están en la caché de nivel 2. Esta información se guarda en una segunda SRAM, rotulada "Etiquetas de caché de nivel 2".

Cuando no se encuentra una línea en la caché de nivel 1, la CPU envía su identificador (dirección de etiqueta) a la caché de nivel 2. La respuesta (datos de etiqueta) proporciona la información que permite a la CPU saber si la línea está o no en la caché de nivel 2 y, si está, en qué estado está. Si la línea está guardada ahí, la CPU la trae. Las transferencias de datos tienen una anchura de 16 bytes, por lo que se necesitan cuatro ciclos para traer toda una línea a la caché de nivel 1.

Si la línea de caché no está en la caché de nivel 2, deberá traerse de la memoria principal vía la interfaz UPA. La UPA del UltraSPARC II se implementa con un controlador centralizado. Las señales de dirección y control de la CPU (de todas las CPU si hay más de una) llegan ahí. Para acceder la memoria, la CPU debe usar primero las líneas de arbitraje de bus para obtener permiso de continuar. Una vez concedido el permiso, la CPU envía la dirección de memoria, especifica el tipo de solicitud, y habilita la línea de dirección válida. (Estas líneas son bidireccionales porque otras CPU de un multiprocesador UltraSPARC II necesitan tener acceso a las cachés remotas para mantener la coherencia de todas las cachés.) La dirección y el tipo de ciclo de bus se colocan en las líneas de Dirección en dos ciclos: el renglón en el primer ciclo y la columna en el segundo, como vimos en la figura 3-31.

Mientras espera los resultados, la CPU bien podría continuar con otros trabajos. Por ejemplo, una caché no inhibe la ejecución de una o más instrucciones que ya se obtuvieron mientras se preobtiene una instrucción, las cuales podrían hacer referencia a datos que no están en ninguna caché. Así, podría haber varias transacciones con la UPA pendientes en un momento dado. La UPA puede manejar dos flujos de transacción independientes (por lo regular lecturas y escrituras), cada uno con múltiples transacciones pendientes. Corresponde al controlador centralizado seguir la pista a todo esto y efectuar las solicitudes de memoria propiamente dichas en el orden más eficiente.

Cuando por fin llegan los datos de la memoria, podrían venir en grupos de 8 bytes, y con un código de corrección de errores de 16 bits para mayor confiabilidad. Una transacción podría pedir todo un bloque de caché, una palabra cuádruple (8 bytes) o incluso menos bytes. Todos los datos que llegan se colocan en buffers en el UDB. El propósito del UDB es desacoplar aun más la CPU del sistema de memoria, de modo que puedan operar asincrónicamente. Por ejemplo, si la CPU tiene que escribir una palabra o línea de caché en la memoria, en lugar de esperar para accederla UPA puede escribir los datos en el UDB de inmediato y dejar que el UDB se encargue de hacerlos llegar a la memoria posteriormente. El UDB también genera y verifica el código de corrección de errores. Cabe señalar que la descripción del UltraSPARC II que acabamos de presentar, al igual que la del Pentium II que la precedió, se ha simplificado considerablemente, aunque se describió la esencia del funcionamiento.

3.5.3 El picoJava II

Tanto el Pentium II como el UltraSPARC II son ejemplos de CPU de alto desempeño diseñadas para construir PC y estaciones de trabajo extremadamente rápidas. Cuando la gente piensa en las computadoras, éste es el tipo de sistemas en el que tienden a concentrarse. Sin embargo, existe todo un mundo de computadoras distinto que en realidad es mucho más grande: los sistemas incorporados. En esta sección daremos un vistazo a ese mundo.

Es probable que no exageremos mucho si decimos que cualquier dispositivo eléctrico que cueste más de 100 dólares probablemente incluye una computadora. Sin duda los televisores, teléfonos celulares, organizadores personales electrónicos, hornos de microondas, cámaras de video, videograbadoras, impresoras láser, alarmas contra robo, aditamentos para la sordera, juegos electrónicos y otros dispositivos están controlados por computadoras. Las computadoras contenidas en estos artículos suelen estar optimadas para tener un precio bajo más que para tener un buen desempeño, lo que da lugar a concesiones diferentes de las que se hacen en las CPU de alta tecnología que hemos estado estudiando hasta ahora.

Tradicionalmente, los procesadores incorporados se han programado en lenguaje ensamblador, pero a medida que los aparatos se vuelven más complicados y las consecuencias de los errores de software se hacen más severos, otros enfoques se han vuelto competitivos. En particular, el uso de Java como lenguaje de programación para sistemas incorporados es atractivo en virtud de su relativa facilidad de programación, el tamaño reducido del código y su independencia de la plataforma. La principal desventaja de usar Java en aplicaciones incorporadas es la necesidad de tener un intérprete grande en software que ejecute el código JVM producido por el compilador de Java, y la lentitud del proceso de interpretación.

Sun y otras compañías han atacado este problema diseñando y construyendo chips de CPU que tienen JVM como su conjunto de instrucciones nativo. Este enfoque combina las ventajas de usar Java como lenguaje de programación, la transportabilidad y el tamaño compacto del código JVM binario producido por el compilador de Java, y la rapidez de ejecución por hardware especializado. En esta sección examinaremos una arquitectura de CPU moderna basada en Java diseñada específicamente para el mercado de los sistemas incorporados.

La CPU es la picoJava II de Sun en que se basa el chip microJava 701 de Sun, aunque Sun también ha otorgado licencias para el diseño a otras compañías. Es una CPU de un solo chip con dos interfaces de bus, una para el bus de memoria que tiene una anchura de 64 bits y una para el bus PCI que tiene una anchura de 32 bits, como se muestra en la figura 3-48. Al igual que el Pentium II y el UltraSPARC II, el picoJava II tiene una caché nivel 1 (opcional) dividida en el chip, con hasta 16 KB para instrucciones y hasta 16 KB para datos. Sin embargo, a diferencia de esas dos CPU, el picoJava II no tiene una caché nivel 2 porque uno de los parámetros de diseño fundamentales de los sistemas incorporados es el bajo costo. A continuación describiremos la implementación del picoJava II hecha por Sun: el microJava 701. El chip es pequeño según los criterios actuales: sólo dos millones de transistores para el núcleo más otro millón y medio para las dos cachés de 16 KB juntas.

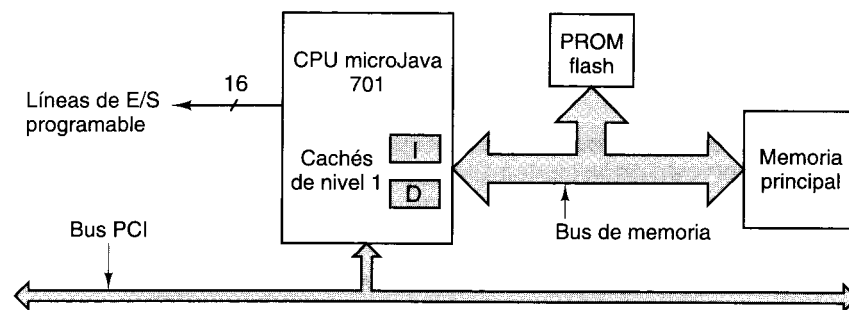


Figura 3-48. Un sistema microJava 701.

Tres características de la figura 3-48 saltan a la vista. Primera, el microJava 701 utiliza el bus PCI (a 33 MHz o bien 66 MHz). Este bus fue creado por Intel para usarse en los sistemas Pentium del extremo superior, pero es independiente del procesador. La ventaja de usar el bus PCI es que es un estándar, y así se evita tener que diseñar un bus nuevo. Además, ya existen muchas tarjetas hechas para insertarse en él. Aunque la existencia de tarjetas para PCI no es gran ventaja cuando se están construyendo teléfonos celulares, en el caso de televisores de Web y otros aparatos más grandes sí es un punto a favor.

Segunda, un sistema microJava 701 normalmente incluye una PROM flash. La cuestión aquí es que en el caso de un aparato una buena parte del programa, si no es que todo, tiene que estar incluido ya en el dispositivo. Una PROM flash es un buen lugar para almacenar el programa, así que es útil tener una interfaz para una de ellas. Otro chip que se puede añadir al sistema (no se muestra) contiene las interfaces de E/S serial y paralela que se encuentran en una PC.

Tercera, el microJava 701 tiene 16 líneas de E/S programable que pueden conectarse a botones, interruptores y lámparas del aparato. Por ejemplo, un horno de microondas suele tener un teclado numérico y algunos otros botones que podrían conectarse directamente al chip de CPU. Tener líneas de E/S programable directamente en la CPU hace innecesario incluir chips controladores de E/S programables, así que el diseño es más simple y menos costoso. El chip también tiene tres temporizadores programables incluidos, lo cual también es útil porque los aparatos a menudo operan en tiempo real.

El microJava 701 se vende en un encapsulado **BGA (Ball Grid Array)** de 316 terminales que es un estándar de la industria. De estas terminales, 59 están conectadas al bus PCI. Estudiaremos este bus en una sección posterior del capítulo. Otras 123 terminales son para el bus de memoria, incluidas 64 terminales de datos bidireccionales así como terminales de dirección aparte. Otras terminales se usan para control (7), temporizadores (3), interrupciones (11), pruebas (10) y E/S programable (16). Algunas de las terminales restantes se usan (con redundancia) para la alimentación y la tierra, pero otras no se usan. Otros fabricantes del picoJava II están en libertad de escoger un bus distinto, otro paquete, etcétera.

El chip también tiene varias características de hardware más, como un modo inactivo para ahorrar energía de baterías, un controlador de interrupciones en el chip y apoyo pleno para el estándar de prueba IEEE 1149.1 JTAG.

3.6 EJEMPLOS DE BUSES

Los buses son el tejido conectivo de los sistemas de computación. En esta sección examinaremos de cerca algunos buses muy utilizados: el bus ISA, el bus PCI y el Bus Serial Universal. El bus ISA es una expansión del bus original de la IBM PC. Por razones de compatibilidad hacia atrás, se sigue incluyendo en todas las PC basadas en Intel, aunque todas estas máquinas tienen también un segundo bus, más rápido: el PCI. El bus PCI es más ancho que el ISA y opera con una tasa de reloj más alta. El Bus Serial Universal es un bus de E/S cada vez más popular para periféricos de baja velocidad, como los ratones y los teclados. En las secciones que siguen examinaremos cada uno de estos buses.

3.6.1 El bus ISA

El bus de la IBM PC era de hecho el estándar en los sistemas basados en el 8088 porque casi todos los fabricantes de clones de la PC lo copiaron para poder usar en sus sistemas las muchas tarjetas de E/S existentes hechas por otros fabricantes. Ese bus tenía 62 líneas de señal, incluidas 20 para una dirección de memoria, 8 para datos y una en cada caso para habilitar lectura de memoria, escritura de memoria, lectura de E/S y escritura de E/S. También había señales para solicitar y conceder interrupciones y usar DMA, y nada más. Era un bus muy sencillo.

Físicamente, el bus estaba grabado en la tarjeta madre de la PC, con una media docena de conectores situados a intervalos de 2 cm para insertar en ellos tarjetas (*plug-ins*). Cada tarjeta

tenía una pestaña que embonaba en el conector. La pestaña tenía 31 tiras chapeadas en oro en cada lado que hacían contacto eléctrico con el conector.

Cuando IBM introdujo la PC/AT basada en el 80286, enfrentó un problema grave. Si hubiera comenzado desde cero y diseñado un bus de 16 bits totalmente nuevo, muchos clientes en potencia habrían dudado en comprar la máquina porque ninguna de las numerosas tarjetas para PC insertables hechas por terceros fabricantes habría funcionado con la nueva máquina. Por otra parte, seguir con el bus de PC y sus 20 líneas de dirección y 8 líneas de datos habría impedido aprovechar la capacidad del 80286 para direccionar 16M de memoria y transferir palabras de 16 bits.

La solución que se escogió fue extender el bus de PC. Las tarjetas insertables para PC tienen un conector de arista con 62 contactos, pero que no corre a todo lo largo de la tarjeta. La solución para la PC/AT fue colocar un segundo conector de arista en la parte inferior de la tarjeta, junto al principal, y diseñar los circuitos de la AT de modo que funcionaran con ambos tipos de tarjetas. la idea general se ilustra en la figura 3-49.

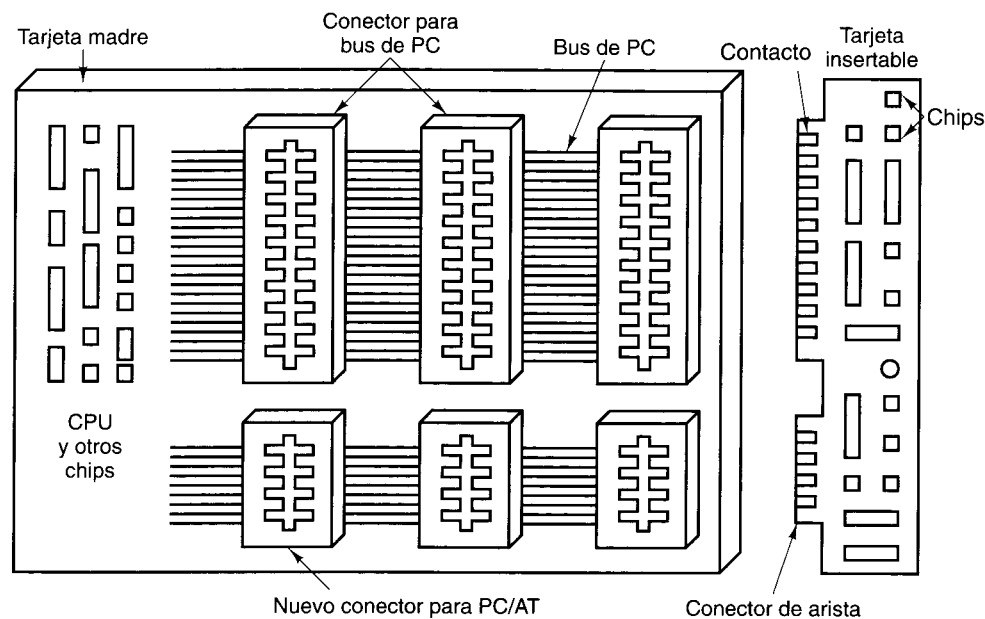


Figura 3-49. El bus PC/AT tiene dos componentes, la parte de la PC original y la parte nueva.

El segundo conector del bus PC/AT contiene 36 líneas. De éstas, 31 se dedican a más líneas de dirección, más líneas de datos, más líneas de interrupciones y más canales de DMA, además de alimentación y tierra. El resto se ocupa de diferencias entre las transferencias de 8 bits y las de 16 bits.

Cuando IBM introdujo la serie PS/2 como sucesora de la PC y la PC/AT, decidió que era hora de comenzar de nuevo. En parte, esta decisión pudo haber sido técnica (el bus de PC a

esas alturas ya era realmente obsoleto), pero en parte se debió sin duda a un deseo de poner un obstáculo en el camino de las compañías que fabricaban clones de la PC, las cuales se habían apoderado de una porción preocupante del mercado. Así, las máquinas PS/2 de precio medio y alto se equiparon con un bus, el Microchannel, que era totalmente nuevo y que estaba protegido por una pared de patentes respaldada por un ejército de abogados.

El resto de la industria de las computadoras personales reaccionó adoptando su propio estándar, el bus **ISA (arquitectura estándar de la industria, Industry Standard Architecture)**, que es básicamente el bus de la PC/AT a 8.33 MHz. La ventaja de este enfoque es que mantiene la compatibilidad con las máquinas y las tarjetas existentes; además, se basa en un bus para el que IBM había otorgado generosamente licencias a muchas compañías con el fin de asegurar que el mayor número de fabricantes posible produjera tarjetas para el PC original, algo que resultó contraproducente para IBM. Todas las PC basadas en Intel tienen aún este bus, aunque generalmente acompañado por uno o más buses distintos. Se puede encontrar una descripción exhaustiva del bus ISA en (Shanley y Anderson, 1995a).

Más adelante, el bus ISA se extendió a 32 bits con unas cuantas funciones adicionales (por ejemplo, para multiprocesamiento). Este nuevo bus se llamó **EISA (ISA extendido)**. Sin embargo, casi no se han producido tarjetas para él.

3.6.2 El bus PCI

En la IBM PC original, casi todas las aplicaciones se basaban en texto. Gradualmente, con la introducción de Windows, comenzaron a usarse las interfaces gráficas con el usuario. Ninguna de estas aplicaciones forzaba mucho al bus ISA, pero al pasar el tiempo cada vez más aplicaciones, sobre todo los juegos de multimedia, comenzaron a usar computadoras para exhibir video de pantalla completa y pleno movimiento, y la situación cambió radicalmente.

Hagamos un cálculo sencillo. Consideremos una pantalla de 1024×768 que se usa para imágenes en movimiento de color natural (3 bytes/pixel). Una pantalla contiene 2.25 MB de datos. Para que el movimiento sea continuo, se requieren al menos 30 pantallas por segundo, lo que corresponde a una tasa de datos de 67.5 MB/s. De hecho, la situación es peor, porque para exhibir un video desde un disco duro, CD-ROM o DVD los datos deben pasar por el bus de la unidad de disco a la memoria. Luego, para exhibirlos, los datos deben pasar otra vez por el bus hacia el adaptador de gráficos. Así, pues, necesitamos un ancho de banda de bus de 135 MB/s sólo para el video, sin contar el ancho de banda que necesitan la CPU y otros dispositivos.

El bus ISA opera con una rapidez máxima de 8.33 MHz y puede transferir dos bytes por ciclo, lo que da un ancho de banda máximo de 16.7 MB/s. El bus EISA puede transferir cuatro bytes por ciclo, así que alcanza los 33.3 MB/s. Es evidente que ninguno de ellos se acerca siquiera a lo que se necesita para video de pantalla completa.

En 1990 Intel previó esto y diseñó un nuevo bus con un ancho de banda mucho mayor que incluso el del bus EISA, y lo llamó **bus PCI (interconexión de componentes periféricos, Peripheral Component Interconnect)**. Para fomentar su uso, Intel patentó el bus PCI y luego puso todas las patentes en el dominio público para que cualquier compañía pudiera construir periféricos para él sin tener que pagar regalías. Además, Intel formó un consorcio de la industria, el PCI Special Interest Group, para controlar el futuro del bus PCI. Como resultado de

estas acciones, el bus PCI se ha popularizado enormemente. Casi todas las computadoras basadas en Intel a partir de la Pentium tienen un bus PCI, y muchas otras computadoras también lo tienen. Incluso Sun tiene una versión del UltraSPARC que usa el bus PCI, el UltraSPARC IIi. El bus PCI se cubre con lujo de detalle en (Shanley y Anderson 1995b; y Solari y Willse, 1998).

El bus de PCI original transfería 32 bits por ciclo y operaba a 33 MHz (tiempo de ciclo de 30 ns), para un ancho de banda total de 133 MB/s. En 1993 se introdujo PCI 2.0, y en 1995 salió PCI 2.1. El PCI 2.2 cuenta con funciones para computadoras móviles (principalmente para ahorrar la energía de las baterías). El bus PCI opera hasta 66 MHz y puede manejar transferencias de 64 bits, para un ancho de banda total de 528 MB/s. Con este tipo de capacidad es posible tener video de pantalla completa y pleno movimiento (suponiendo que el disco y el resto del sistema están a la altura del reto). En todo caso, el bus PCI no será el cuello de botella.

Aunque 528 MB/s parece muy rápido, aún tiene dos problemas. Primero, no es lo bastante bueno como para ser bus de memoria. Segundo, no es compatible con todas las viejas tarjetas para ISA que todavía existen. La solución que ideó Intel fue diseñar computadoras con tres o más buses, como se muestra en la figura 3-50. Aquí vemos que la CPU puede comunicarse con la memoria principal por un bus de memoria especial, y que se puede conectar un bus ISA al bus PCI. Esta organización satisface todos los requisitos, así que prácticamente todas las computadoras Pentium II emplean esta arquitectura.

Dos componentes clave de esta arquitectura son los dos chips de puente (que Intel fabrica: de ahí su interés en todo este proyecto). El puente PCI conecta la CPU, la memoria y el bus PCI. El puente ISA conecta el bus PCI al bus ISA y también apoya uno o dos discos IDE. Casi todos los sistemas Pentium II vienen con una o más ranuras PCI libres para añadir nuevos periféricos de alta velocidad, y una o más ranuras ISA para agregar periféricos de baja velocidad.

La gran ventaja del diseño de la figura 3-50 es que la CPU tiene un ancho de banda a la memoria gigantesco. Al usar un bus de memoria patentado, el bus PCI tiene un ancho de banda muy grande para periféricos rápidos como discos SCSI, adaptadores de gráficos, etc., y es posible seguir usando tarjetas ISA viejas. El cuadro USB de la figura se refiere al Bus Serial Universal, que veremos más adelante en este capítulo.

Aunque ilustramos un sistema con un bus PCI y un bus ISA, es posible tener varios ejemplares de cada uno. Existen chips de puente PCI a PCI que conectan dos buses PCI, de modo que los sistemas más grandes pueden tener dos o más buses PCI independientes. También es posible tener dos o más chips de puente PCI a ISA en un sistema, y así incluir varios buses ISA.

Habría sido útil que sólo existiera un tipo de tarjeta para PCI. Lamentablemente, no es así. Se ofrecen opciones de voltaje, capacidad y temporización. Muchas computadoras viejas usan 5 volts, mientras que las más nuevas tienden a usar 3.3 volts, por lo que el bus PCI apoya ambos voltajes. Los conectores son iguales excepto por dos trozos de plástico que impiden que la gente inserte una tarjeta de 5 volts en un bus PCI de 3.3 volts o viceversa. Por fortuna existen tarjetas universales que manejan ambos voltajes y pueden insertarse en cualquier tipo de ranura. Además de la opción de voltaje, hay tarjetas en versiones de 32 bits y de 64 bits.

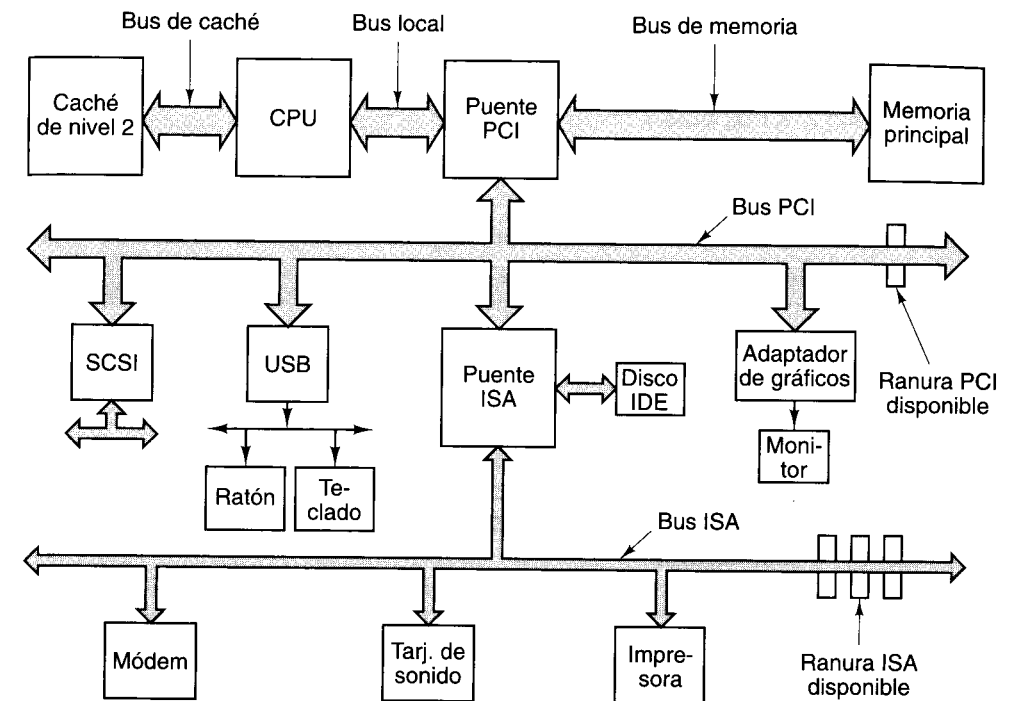


Figura 3-50. Arquitectura de un sistema Pentium II representativo. Los buses más gruesos tienen más ancho de banda que los más delgados.

Las tarjetas de 32 bits tienen 120 terminales; las de 64 bits poseen las mismas 120 terminales más 64 adicionales, semejante a la manera en que se extendió a 16 bits el bus de la IBM PC (vea la figura 3-49). Un sistema de bus PCI que acepta tarjetas de 64 bits también puede aceptar tarjetas de 32 bits, pero no al revés. Por último, los buses y las tarjetas PCI pueden operar a 33 MHz o 66 MHz. La decisión se toma conectando permanentemente una terminal ya sea a la fuente de alimentación o a tierra. Los conectores son idénticos para ambas velocidades.

El bus PCI es sincrónico, como lo fueron todos los buses de PC hasta el de la IBM PC original. Todas las transacciones se efectúan entre un amo, llamado oficialmente **iniciador**, y un esclavo, llamado oficialmente **objetivo**. Para reducir el número de contactos del bus PCI, las líneas de direcciones y de datos se multiplexan. Así, sólo se necesitan 64 terminales en las tarjetas PCI para las señales de dirección y de datos, aunque PCI maneja direcciones de 64 bits y datos de 64 bits.

Las líneas de dirección y datos multiplexadas funcionan como sigue. En una operación de lectura, durante el ciclo 1, el iniciador coloca la dirección en el bus. En el ciclo 2 el iniciador quita la dirección y el bus se invierte para que el esclavo pueda usarlo. En el ciclo 3, el esclavo envía los datos solicitados. En las operaciones de escritura, el bus no tiene que invertirse porque el iniciador coloca en él tanto la dirección como los datos. No obstante, la

transacción mínima sigue siendo de tres ciclos. Si el esclavo no puede responder en tres ciclos, puede insertar estados de espera. También se permiten transferencias de bloques de tamaño ilimitado, así como varios otros tipos de ciclos de bus.

Arbitraje del bus PCI

Para usar el bus PCI un dispositivo primero debe adquirirlo. El arbitraje del bus PCI emplea un sistema centralizado que se muestra en la figura 3-51. En casi todos los diseños, el árbitro de bus viene incorporado en uno de los chips de puente. Todo dispositivo PCI tiene dos líneas dedicadas que van de él al árbitro. Una línea, REQ#, sirve para solicitar el bus. La otra, GNT#, es útil para recibir concesiones de bus.

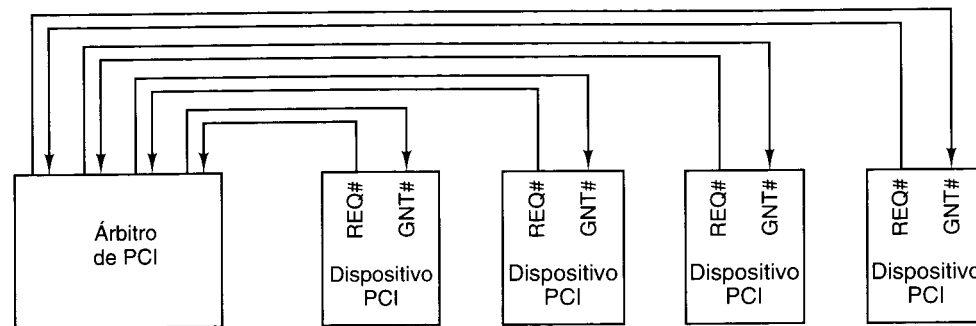


Figura 3-51. El bus PCI emplea un árbitro de bus centralizado.

Para solicitar el bus, un dispositivo PCI (incluida la CPU) aserta REQ# y espera hasta detectar que el árbitro habilitó su línea GNT#. Cuando eso sucede, el dispositivo puede usar el bus en el siguiente ciclo. El algoritmo empleado por el árbitro no está definido en la especificación de PCI. Se permiten arbitrajes por turno circular, por prioridad y otros esquemas. Es evidente que un buen árbitro es justo y no deja que algunos dispositivos esperen en forma indefinida.

Una concesión de bus es válida para una transacción, aunque la longitud de dicha transacción en teoría es ilimitada. Si un dispositivo quiere ejecutar una segunda transacción y ningún otro dispositivo está solicitando el bus, podrá continuar, aunque normalmente es preciso insertar un ciclo ocioso entre las transacciones. Sin embargo, en circunstancias especiales y si no hay competencia por el bus, un dispositivo puede efectuar transacciones una tras otra sin tener que insertar un ciclo ocioso. Si un iniciador de bus está efectuando una transferencia muy larga y algún otro dispositivo solicitó el bus, el árbitro puede deshabilitar la línea GNT#. Se espera que el iniciador de bus en turno vigile la línea GNT#, así que cuando percibe la deshabilitación debe liberar el bus en el siguiente ciclo. Este esquema permite transferencias muy largas (que son eficientes) cuando sólo hay un candidato a iniciador de bus, pero también ofrece una respuesta rápida a los dispositivos competidores.

Señales del bus PCI

El bus PCI tiene varias señales obligatorias que se muestran en la figura 3-52(a), y varias señales opcionales, que se muestran en la figura 3-52(b). El resto de las 120 o 184 terminales se usa para alimentación, tierra y otras funciones diversas relacionadas; no se mostrarán aquí. Las columnas *Amo* (iniciador) y *Esclavo* (objetivo) indican quién habilita la señal en una transacción normal. Si un dispositivo distinto (digamos CLK) habilita la señal, las dos columnas se dejan en blanco.

Examinemos brevemente cada una de las señales del bus PCI. Comenzaremos con las señales obligatorias (de 32 bits); luego pasaremos a las señales opcionales (de 64 bits). La señal CLK controla el bus; casi todas las demás señales son sincrónicas con ella. En contraste con el bus ISA, una transacción de bus PCI se inicia en el flanco descendente de CLK, que está a la mitad del ciclo, no al principio.

Las 32 señales AD son para la dirección y los datos (en transacciones de 32 bits). En general, durante el ciclo 1 se habilita la dirección y durante el ciclo 3 se habilitan los datos. La señal PAR es un bit de paridad para AD. La señal C/BE# se usa para dos cosas distintas: en el ciclo 1, contiene el comando de bus (leer una palabra, leer un bloque, etc.); en el ciclo 2 contiene un mapa de bits de 4 bits que indica cuáles bytes de la palabra de 32 bits son válidos. Con la ayuda de C/BE# es posible leer o escribir cualesquier 1, 2 o 3 bytes, así como una palabra entera.

El iniciador de bus habilita la señal FRAME# para iniciar una transacción de bus. Esta señal le dice al esclavo que la dirección y los comandos de bus ya son válidos. En una lectura casi siempre se habilita IRDY# al mismo tiempo que FRAME#. Esa señal indica que el iniciador está listo para aceptar los datos. En una escritura, IRDY# se habilita después, cuando los datos ya están en el bus.

La señal IDSEL tiene que ver con el hecho de que todo dispositivo PCI debe tener un espacio de configuración de 256 bytes que otros dispositivos pueden leer (habilitando IDSEL). Este espacio de configuración contiene propiedades del dispositivo. La característica de "conectar y operar" (*Plug 'n Play*) de algunos sistemas operativos emplea el espacio de configuración para averiguar cuáles dispositivos están en el bus.

Ahora llegamos a las señales que el esclavo habilita. La primera, DEVSEL#, anuncia que el esclavo detectó su dirección en las líneas AD y está listo para intervenir en la transacción. Si DEVSEL# no se habilita dentro de cierto límite de tiempo, el iniciador supone que el dispositivo direccionado está ausente o bien descompuesto.

La segunda señal de esclavo es TRDY#, la cual habilita en las lecturas para anunciar que los datos están en las líneas AD, y en las escrituras para anunciar que está preparado para aceptar datos.

Las siguientes tres señales son para informar sobre los errores. La primera es STOP#, que el esclavo habilita si ocurre algún desastre y quiere abortar la transacción en curso. La siguiente, PERR#, sirve para informar de un error de paridad de datos en el ciclo anterior. En el caso de una lectura el iniciador la habilita; en una escritura el esclavo lo hace. Corresponde al receptor tomar las medidas apropiadas. Por último, SERR# sirve para informar sobre errores de dirección y errores de sistema.

Señal	Líneas	Amo	Esclavo	Descripción
CLK	1			Reloj (33 MHz o 66 MHz)
AD	32	×	×	Líneas de dirección y datos multiplexados
PAR	1	×		Bit de paridad de dirección o datos
C/BE	4	×		Comando de bus/mapa de bits para bytes habilitados
FRAME#	1	×		Indica que AD y C/BE están asertadas
IRDY#	1	×		Lectura: el amo aceptará; escritura: datos presentes
IDSEL	1	×		Seleccionar espacio de configuración en lugar de memoria
DEVSEL#	1		×	El esclavo decodificó su dirección y está escuchando
TRDY#	1		×	Lectura: datos presentes; escritura, el esclavo aceptará
STOP#	1		×	El esclavo quiere parar la transacción de inmediato
PERR#	1			El receptor detectó error de paridad de datos
SERR#	1			Se detectó error de paridad de dirección o error de sistema
REQ#	1			Arbitraje de bus: solicitud para tener el bus
GNT#	1			Arbitraje de bus: conceder el bus
RST#	1			Restablecer el sistema y todos los dispositivos

(a)

Señal	Líneas	Amo	Esclavo	Descripción
REQ64#	1	×		Solicitud para ejecutar una transacción de 64 bits
ACK64#	1		×	Se da permiso para una transacción de 64 bits
AD	32	×		32 bits de dirección o datos adicionales
PAR64	1	×		Paridad para los 32 bits de dirección/datos extra
C/BE#	4	×		4 bits adicionales para habilitar bytes
LOCK	1	×		Bloquear el bus para efectuar múltiples transacciones
SBO#	1			Acierto en caché remoto (en multiprocesador)
SDONE	1			Espionaje efectuado (en multiprocesador)
INTx	4			Solicitar interrupción
JTAG	5			Señales de prueba IEEE 1149.1 JTAG
M66EN	1			Conectado a potencia o tierra (66 MHz o 33 MHz)

(b)

Figura 3-52. (a) Señales obligatorias del bus PCI. (b) Señales opcionales del bus PCI.

Las señales REQ# y GNT# sirven para efectuar arbitraje de bus. Éstas no las habilita el iniciador de bus vigente, sino un dispositivo que quiere convertirse en iniciador de bus. La última señal obligatoria es RST#, que sirve para restablecer el sistema, sea porque el usuario oprimió el botón RESET o porque algún dispositivo del sistema detectó un error fatal. La habilitación de esta señal restablece todos los dispositivos y activa nuevamente la computadora.

Ahora llegamos a las señales opcionales, que en su mayoría se relacionan con la expansión de 32 a 64 bits. Las señales REQ64# y ACK64# permiten al iniciador pedir permiso para realizar una transacción de 64 bits, y al esclavo, aceptar. Las señales AD, PAR64 y C/BE# no son más que extensiones de las señales de 32 bits correspondientes.

Las siguientes tres señales no tienen que ver con el número de bits, sino con los sistemas mutiprocesador, algo que las tarjetas PCI no están obligadas a apoyar. La señal LOCK permite bloquear el bus para efectuar múltiples transacciones. Las dos señales siguientes tienen que ver con el espionaje de bus para mantener la coherencia de las cachés.

Las señales INTx son para solicitar interrupciones. Una tarjeta PCI puede contener hasta cuatro dispositivos lógicos individuales, y cada uno puede tener su propia línea para solicitar interrupciones. Las señales JTAG son para el procedimiento de prueba IEEE 1149.1 JTAG. Por último, la señal M66EN se pone permanentemente alta o baja, para fijar la velocidad del reloj, y no debe cambiar durante la operación del sistema.

Transacciones en el bus PCI

El bus PCI es en realidad muy sencillo (en comparación con otros buses). Para entenderlo mejor, considere el diagrama de temporización de la figura 3-53. Aquí vemos una transacción de lectura, seguida de un ciclo ocioso y una transacción de escritura por el mismo iniciador de bus.

Cuando el flanco descendente del reloj ocurre durante T₁, el iniciador coloca la dirección de memoria en AD y el comando de bus en C/BE#; luego habilita FRAME# para iniciar la transacción de bus.

Durante T₂, el iniciador deja que flote el bus de dirección a fin de que pueda invertirse como preparación a ser alimentado por el esclavo durante T₃. El iniciador también modifica C/BE# para indicar cuáles bytes de la palabra direccionada quiere habilitar (es decir, leer).

En T₃, el esclavo aserta DEVSEL# para que el iniciador sepa que recibió la dirección y planee responder; además, coloca los datos en las líneas AD y habilita TRDY# para avisarle al iniciador que lo hizo. Si el esclavo no puede responder con tanta rapidez, de todas maneras habilita DEVSEL# para anunciar su presencia pero mantiene TRDY# deshabilitado hasta que puede colocar los datos en el bus. Este procedimiento introduciría uno o más estados de espera.

En este ejemplo (y con frecuencia en la realidad), el siguiente ciclo es ocioso. A partir de T₅ vemos al mismo amo iniciar una escritura. Lo primero que hace es colocar la dirección y el comando en el bus, como siempre, sólo que ahora habilita los datos durante el segundo ciclo. Puesto que el mismo dispositivo está alimentando las líneas AD, no hay necesidad de un ciclo de inversión. En T₇ la memoria acepta los datos.

3.6.3 El bus serial universal

El bus PCI está bien para conectar periféricos de alta velocidad a una computadora, pero es demasiado costoso tener una interfaz PCI para cada dispositivo de E/S de baja velocidad, como un teclado o un ratón. Históricamente, todos los dispositivos de E/S estándar se conectaban a la computadora de una forma especial, dejando algunas ranuras ISA y PCI libres para

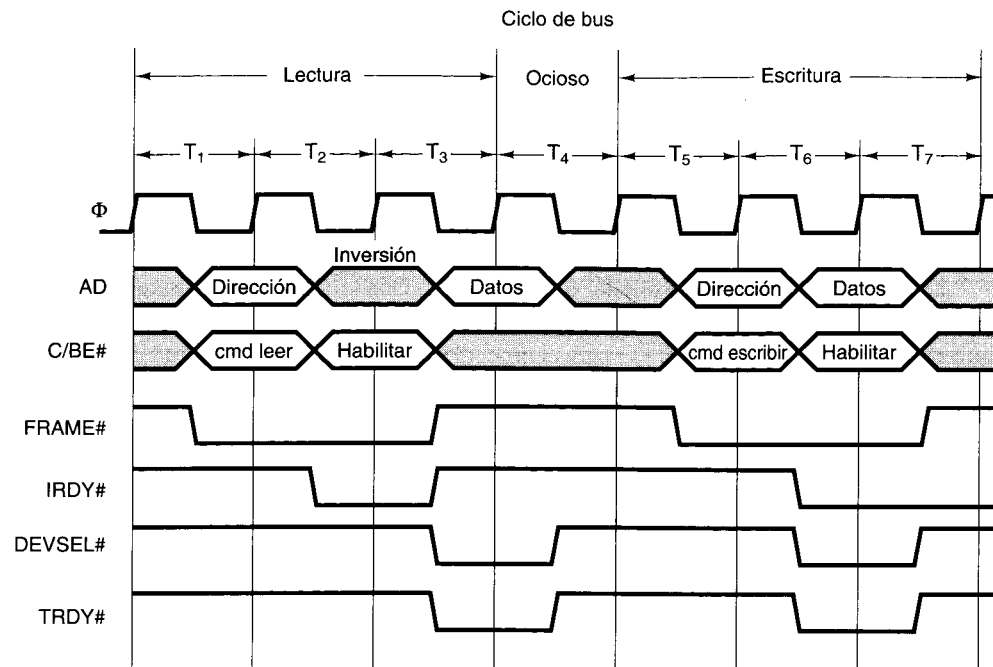


Figura 3-53. Ejemplos de transacciones de bus PCI de 32 bits. Los primeros tres ciclos se emplean para una operación de lectura, luego un ciclo ocioso y luego tres ciclos para una operación de escritura.

añadir nuevos dispositivos. Lamentablemente, este esquema tuvo muchos problemas desde el principio. Por ejemplo, cada dispositivo de E/S nuevo por lo regular viene con su propia tarjeta ISA o PCI. El usuario a menudo tiene la responsabilidad de ajustar interruptores y puentes en la tarjeta y asegurarse de que los ajustes no estén en conflicto con otras tarjetas. Luego, el usuario debe abrir el gabinete de la computadora, insertar la tarjeta con mucho cuidado, cerrar el gabinete y reanunciar la computadora. Para muchos usuarios este proceso es difícil y hay una posibilidad alta de cometer errores. Además, el número de ranuras ISA y PCI es muy limitado (dos o tres, por lo regular). Las tarjetas Plug 'n Play eliminan los ajustes de puentes, pero de todos modos el usuario tiene que abrir la computadora para insertar la tarjeta, y el número de ranuras sigue siendo limitado.

Para resolver este problema, a mediados de la década de los noventa se reunieron representantes de siete compañías (Compaq, DEC, IBM, Intel, Microsoft, NEC y Northern Telecom) para diseñar una mejor manera de conectar dispositivos de E/S de baja velocidad a una computadora. Desde entonces, cientos de compañías más se les han unido. El estándar resultante se llama **USB (Bus Serial Universal)** y se le está implementando ampliamente en las computadoras personales. Este bus se describe con detalle en (Anderson, 1997; y Tan, 1997).

Algunos de los objetivos de las compañías que originalmente concibieron el USB e iniciaron el proyecto son los siguientes:

1. Los usuarios no deben tener que ajustar interruptores ni puenteadores en las tarjetas o dispositivos.
2. Los usuarios no deben tener que abrir la caja para instalar dispositivos de E/S nuevos.
3. Sólo debe haber un tipo de cable, que sirva para conectar todos los dispositivos.
4. Los dispositivos de E/S deberán obtener su energía del cable.
5. Se deberán poder conectar hasta 127 dispositivos a una sola computadora.
6. El sistema deberá apoyar dispositivos de tiempo real (por ejemplo, sonido, teléfono).
7. Los dispositivos deberán poder instalarse mientras la computadora está funcionando.
8. No se deberá requerir un rearranque después de instalar un nuevo dispositivo.
9. El nuevo bus y sus dispositivos de E/S deberán ser de bajo costo.

El USB cumple con todos estos objetivos; se le diseñó para dispositivos de baja velocidad como teclados, ratones, cámaras de foto fija, digitalizadores de imágenes, teléfonos digitales, etc. El ancho de banda total del USB es de 1.5 MB/s, que es suficiente para una cantidad sustancial de tales dispositivos. Se escogió este límite bajo para mantener bajos los costos.

Un sistema USB consiste en un **eje raíz** que se conecta al bus principal (vea la figura 3-50). Este eje tiene zócalos para cables que se pueden conectar a dispositivos de E/S o a ejes de expansión, a fin de tener más zócalos, de modo que la topología de un sistema USB es un árbol con su raíz en el eje raíz, dentro de la computadora. Los cables tienen diferentes conectores en el extremo del eje y en el extremo del dispositivo, para evitar que las personas conecten accidentalmente un zócalo de eje a otro.

El cable consta de cuatro hilos: dos para datos, uno para alimentación (+ 5 volts) y uno para tierra. El sistema de señalización transmite un 0 como una transición de voltaje y un 1 como la ausencia de una transición de voltaje, de modo que las series largas de ceros generan una serie de pulsaciones regular.

Cuando se conecta un nuevo dispositivo de E/S, el eje raíz detecta este suceso e interrumpe el sistema operativo. Luego éste consulta al dispositivo para averiguar qué es y qué ancho de banda de USB necesita. Si el sistema operativo decide que hay suficiente ancho de banda para el dispositivo, asigna a éste una dirección única (1-127) y coloca esta dirección y otra información en los registros de configuración dentro del dispositivo. De este modo es posible añadir dispositivos nuevos sobre la marcha, sin que el usuario tenga que configurarlos y sin tener que instalar nuevas tarjetas ISA o PCI. Las tarjetas no inicializadas tienen la dirección 0, para poderlas direccionar. A fin de simplificar el cableado, muchos dispositivos USB contienen ejes integrados que aceptan dispositivos USB adicionales. Por ejemplo, un monitor podría tener dos zócalos de eje para dar cabida a los altavoces derecho e izquierdo.

Lógicamente, el sistema USB puede verse como un conjunto de conductos de bit que van del eje raíz a los dispositivos de E/S. Cada dispositivo puede dividir su conducto de bits

en, cuando más, 16 subconductos para diferentes tipos de datos (por ejemplo, audio y video). Dentro de cada conducto o subconducto, fluyen datos del eje raíz al dispositivo o en la otra dirección. No hay tráfico entre dos dispositivos de E/S.

Precisamente cada 1.00 ± 0.05 ms, el eje raíz transmite una nueva trama para mantener a todos los dispositivos sincronizados en el tiempo. Una trama se asocia a un conducto de bits y consiste en paquetes, el primero de los cuales va del eje raíz al dispositivo. Los paquetes subsecuentes de la trama también podrían enviarse en esta dirección, pero también podrían regresar del dispositivo al eje raíz. En la figura 3-54 se muestra una sucesión de cuatro tramas.

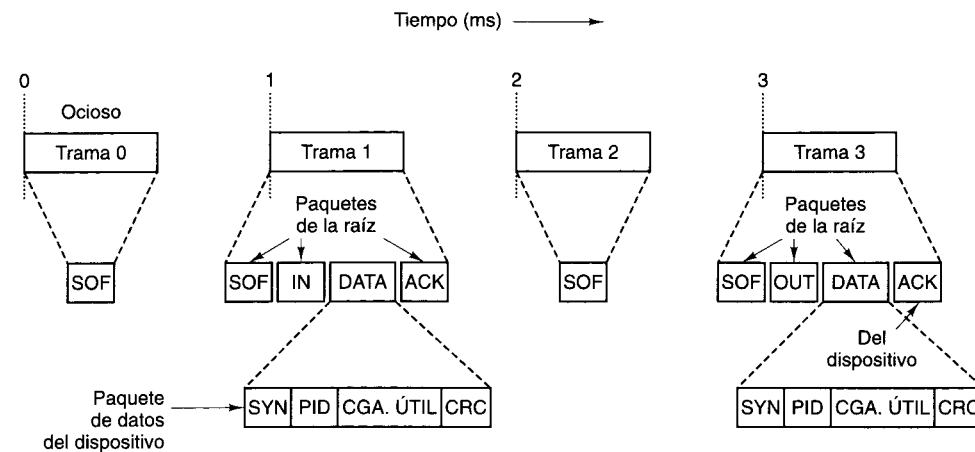


Figura 3-54. El eje raíz USB envía tramas cada 1.00 ms.

En la figura 3-54 no hay trabajo que realizar en las tramas 0 y 2, así que todo lo que se necesita es un paquete SOF (inicio de trama, *Start of Frame*). Este paquete siempre se difunde a todos los dispositivos. La trama 1 es un escrutinio, por ejemplo, una solicitud a un escáner para que devuelva los bits que ha encontrado en la imagen que está digitalizando. La trama 3 consiste en datos que se envían a algún dispositivo, digamos una impresora.

USB reconoce cuatro clases de tramas: de control, isocrónicas, de volumen y de interrupción. Las tramas de control sirven para configurar dispositivos, dar órdenes y preguntar por su estado. Las tramas isocrónicas son para dispositivos de tiempo real como micrófonos, altavoces y teléfonos que necesitan enviar o aceptar datos a intervalos de tiempo precisos. Estas tramas tienen un retraso altamente predecible pero no ofrecen retransmisión en caso de ocurrir errores. Las tramas de volumen son para transferencias grandes hacia o desde dispositivos que no requieren tiempo real, como las impresoras. Por último, las tramas de interrupción son necesarias porque USB no reconoce interrupciones. Por ejemplo, en lugar de hacer que un teclado cause una interrupción cada vez que se pulsa una tecla, el sistema operativo puede escrutarlo cada 50 ms para recolectar cualesquier digitaciones que estén pendientes.

Una trama consiste en uno o más paquetes, posiblemente algunos para cada dirección. Existen cuatro tipos de paquetes: de testigo, de datos, de saludo y especiales. Los paquetes de testigo van de la raíz a un dispositivo y sirven para control del sistema. Los paquetes SOF, IN y

OUT de la figura 3-54 son paquetes de testigo. El paquete SOF (inicio de trama) es el primero de cada trama y marca su principio. Si no hay trabajo que efectuar, el paquete SOF es el único de la trama. El paquete de testigo IN es un escrutinio, que pide al dispositivo devolver ciertos datos. Los campos del paquete IN indican cuál conducto de bits se está escrutando para que el dispositivo sepa qué datos debe devolver (si tiene múltiples flujos). El paquete de testigo OUT anuncia que a continuación vienen datos para el dispositivo. Un cuarto tipo de paquete de testigo, SETUP (que no se muestra en la figura), sirve para configuración.

Además del paquete de testigo, hay otros tres tipos. Éstos son los paquetes DATA (que sirven para transmitir hasta 64 bytes de información en cualquier dirección), los de saludo y los especiales. En la figura 3-54 se muestra el formato de un paquete de datos: consiste en un campo de sincronización de 8 bits, un tipo de paquete (PID) de 8 bits, la carga útil, y un CRC (Código de Redundancia Cíclica) para detectar errores. Se han definido tres clases de paquetes de saludo: ACK (el paquete de datos anterior se recibió correctamente), NAK (se detectó un error de CRC) y STALL (espera por favor; estoy ocupado).

Examinemos otra vez la figura 3-54. Cada 1.00 ms debe enviarse una trama del eje raíz, incluso si no hay trabajo. Las tramas 0 y 2 consisten en un solo paquete SOF, lo que indica que no hubo trabajo. La trama 1 es un escrutinio, de modo que principia con paquetes SOF e IN de la computadora al dispositivo de E/S, seguidos de un paquete DATA del dispositivo a la computadora. El paquete ACK le indica al dispositivo que los datos se recibieron correctamente. En caso de haber un error, se devolvería un NAK al dispositivo y el paquete se retransmitiría en el caso de datos de volumen (pero no en el de datos isocrónicos). La trama 3 tiene una estructura similar a la 1, excepto que ahora los datos fluyen de la computadora al dispositivo.

3.7 INTERFACES

Un sistema de computación de tamaño pequeño a mediano consiste en un chip de CPU, chips de memoria y algunos controladores de E/S, todos conectados por un bus. Ya estudiamos las memorias, las CPU y los buses con cierto detalle. Ha llegado el momento de examinar la última pieza del rompecabezas, los chips de E/S. Es a través de estos chips que la computadora se comunica con el mundo exterior.

3.7.1 Chips de E/S

Ya existen numerosos chips de E/S y constantemente se introducen nuevos. Entre los chips más comunes están los UART, los USART, los controladores de CRT, los controladores de disco y los PIO. Un **UART (receptor transmisor universal asincrónico, Universal Asynchronous Receiver Transmitter)** es un chip que puede leer un byte del bus de datos y enviarlo bit por bit a través de una línea serial hacia una terminal, o recibir datos de una terminal. Los UART por lo regular manejan varias velocidades; anchuras de caracteres de 5 a 8 bits; 1, 1.5 o 2 bits de paro; y paridad par, impar o ninguna, todo bajo control de un programa. Un **USART (receptor transmisor universal sincrónico asincrónico, Universal Synchronous Asynchronous Receiver Transmitter)** puede manejar transmisiones sincrónicas utilizando diversos protocolos, además de realizar todas las funciones del UART. Puesto que

ya examinamos los UART en el capítulo 2, estudiemos ahora la interfaz paralela como ejemplo de chip de E/S

Chips PIO

Un chip de **entrada/salida paralela (PIO, Parallel Input/Output)** representativo es el Intel 8255A, que se muestra en la figura 3-55. Este chip tiene 24 líneas de E/S que pueden conectarse con cualquier dispositivo compatible con TTL, por ejemplo, teclados, interruptores, lámparas o impresoras. En pocas palabras, el programa de la CPU puede escribir un 0 o un 1 en cualquier línea, o leer el estado de entrada de cualquier línea, lo que ofrece gran flexibilidad. Un sistema pequeño basado en CPU que usa un PIO a menudo puede sustituir a toda una tarjeta llena de chips SSI o MSI, sobre todo en los sistemas incorporados.

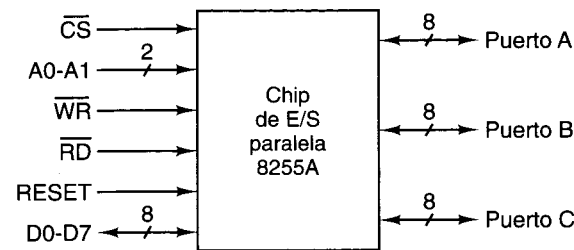


Figura 3-55. Chip PIO 8255A.

Aunque la CPU puede configurar el 8255A de muchas maneras cargando registros de situación dentro del chip, nos concentraremos en algunos de los modos de operación más sencillos. La forma más simple de usar el 8255A es como tres puertos de 8 bits independientes, A, B y C. Cada puerto tiene asociado un registro latch de 8 bits. Para establecer las líneas de un puerto, la CPU sólo tiene que escribir un número de ocho bits en el registro correspondiente; el número de ocho bits aparecerá en las líneas de salida y permanecerá ahí hasta que se coloque un valor distinto en el registro. Si la CPU quiere usar un puerto para recibir entradas, sólo tiene que leer el registro correspondiente.

Otros modos de operación permiten sincronizarse con dispositivos externos. Por ejemplo, para enviar salidas a un dispositivo que no siempre está listo para aceptar datos, el 8255A puede presentar datos en un puerto de salida y esperar a que el dispositivo envíe de vuelta una pulsación para indicar que aceptó los datos y quiere más. La lógica necesaria para captar tales pulsaciones y proporcionarlas a la CPU viene incluida en el hardware del 8255A.

En el diagrama funcional del 8255A podemos ver que, además de 24 terminales para los tres puertos, el chip tiene ocho líneas que se conectan directamente con el bus de datos, una línea de selección de chip, líneas de lectura y escritura, dos líneas de dirección y una línea para restablecer el chip. Las dos líneas de dirección (A0 y A1) seleccionan uno de los cuatro registros internos, que corresponden a los puertos A, B, C y el registro de situación, que tiene bits que determinan cuáles puertos son para entrada y cuáles para salida, además de otras

funciones. Normalmente, las dos líneas de dirección están conectadas a los bits de orden bajo del bus de dirección.

3.7.2 Decodificación de direcciones

Hasta ahora hemos evitado deliberadamente explicar con claridad cómo se habilita la línea de selección de chip (cs) en los chips de memoria y de E/S que hemos examinado. Ha llegado el momento de ver con más detalle cómo se hace esto. Consideremos una sencilla computadora de 16 bits que consiste de una CPU, una EPROM de $2K \times 8$ bytes para el programa, una RAM de $2K \times 8$ bytes para los datos, y un PIO. Este pequeño sistema podría usarse como prototipo del cerebro de un juguete barato o un aparato doméstico sencillo. Ya en producción, la EPROM podría sustituirse por una ROM.

El PIO puede seleccionarse de una de dos maneras: como verdadero dispositivo de E/S o como parte de la memoria. Si optamos por usarla como dispositivo de E/S, deberemos seleccionarla usando una línea de bus explícita que indique que se está haciendo referencia a un dispositivo de E/S, no a la memoria. Si usamos la otra estrategia, **E/S con mapa en la memoria**, deberemos asignar 4 bytes del espacio de memoria para los tres puertos y el registro de control. La decisión es un tanto arbitraria. Escogeremos E/S con mapa en la memoria porque ilustra algunos aspectos interesantes de las interfaces de E/S.

La EPROM necesita 2K de espacio de direcciones, la RAM también necesita 2K de espacio de direcciones, y el PIO necesita 4 bytes. Puesto que en nuestro ejemplo el espacio de direcciones es de 64K, deberemos decidir dónde poner los tres dispositivos. Una posible opción se muestra en la figura 3-56. La EPROM ocupa las direcciones hasta 2K, la RAM ocupa las direcciones de 32K a 34K, y el PIO ocupa los 4 bytes más altos del espacio de direcciones, 65532 a 65535. Desde el punto de vista del programador no importa cuáles direcciones se usen, pero para las interfaces sí importa. Si hubiéramos optado por direccionar el PIO a través del espacio de E/S, no necesitaría direcciones de memoria (pero sí necesitaría cuatro direcciones del espacio de E/S).

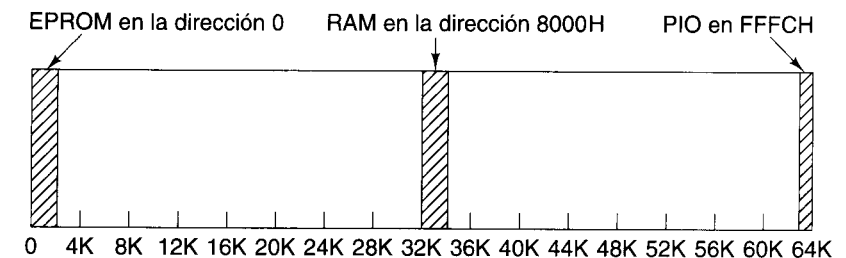


Figura 3-56. Ubicación de la EPROM, la RAM y el PIO en nuestro espacio de direcciones de 64K.

Con las asignaciones de direcciones de la figura 3-56, la EPROM deberá seleccionarse con cualquier dirección de memoria de 16 bits de la forma 00000xxxxxxxxxxx (binario). En otras palabras, cualquier dirección de memoria cuyos 5 bits de orden alto sean todos cero

quedará en los 2K más bajos de la memoria, y por ende en la EPROM. Así, la selección de chip de la EPROM podría conectarse a un comparador de 5 bits, una de cuyas entradas estaría conectada permanentemente a 00000.

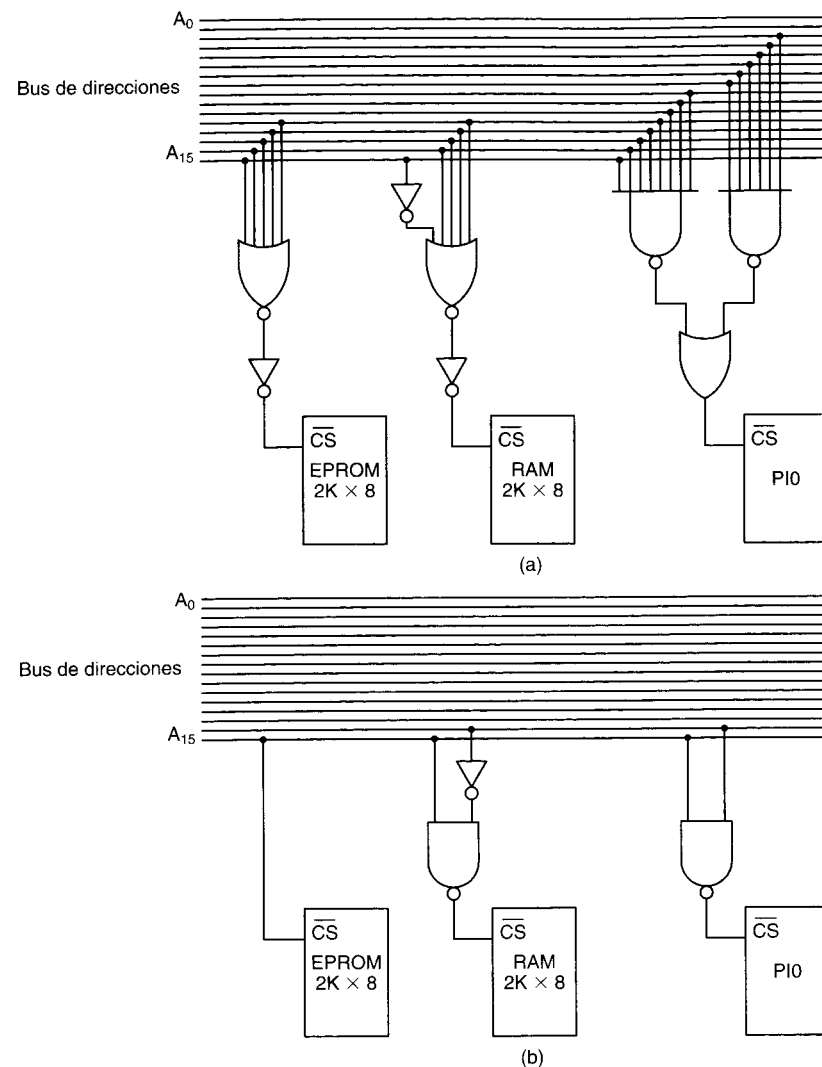


Figura 3-57. (a) Decodificación de dirección completa. (b) Decodificación de dirección parcial.

Una mejor forma de lograr el mismo efecto es usar una compuerta OR de cinco entradas, conectando éstas a las líneas de dirección A₁₁ a A₁₅. Sólo si las cinco líneas son 0 la salida será 0, lo que habilitará $\overline{CS}$. Lo malo es que no existen compuertas OR de cinco entradas en las

series SSI estándar. Lo más cercano que hay es una compuerta NOR de ocho entradas. Si conectamos a tierra tres entradas e invertimos la salida podremos producir la señal correcta, como se muestra en la figura 3-57(a). Los chips SSI son tan baratos que, excepto en circunstancias especiales, no importa si se usa uno de ellos de forma ineficiente. Por convención, las entradas que no se usan no se muestran en los diagramas de circuito.

Se puede usar el mismo principio para la RAM, sólo que en este caso la RAM deberá responder a direcciones binarias de la forma 10000xxxxxxxxxxx. Por ello, se necesita un inversor adicional como se muestra en la figura. La decodificación de direcciones del PIO es un poco más complicada, porque se selecciona con las cuatro direcciones de la forma 111111111111xx. En la figura se muestra un posible circuito que habilita $\overline{CS}$ sólo cuando la dirección correcta aparece en el bus de direcciones. Este circuito usa dos compuertas NAND de ocho entradas para alimentar una compuerta OR. La construcción de la lógica de decodificación de direcciones de la figura 3-57(a) utilizando SSI requiere seis chips: los cuatro chips de ocho entradas, una compuerta OR y un chip con tres inversores.

Sin embargo, si la computadora realmente consiste sólo en una CPU, dos chips de memoria y el PIO, podemos usar un truco para simplificar mucho la decodificación de direcciones. El truco se basa en el hecho de que todas las direcciones de la EPROM, y sólo esas direcciones, tienen un 0 en el bit de orden alto, A₁₅. Por tanto, basta con conectar $\overline{CS}$ a A₁₅ directamente, como se muestra en la figura 3-57(b).

A estas alturas la decisión de colocar la RAM en 8000H estará pareciendo menos arbitraria. La decodificación de RAM puede efectuarse observando que las únicas direcciones válidas de la forma 10xxxxxxxxxxxx están en la RAM, de modo que basta con dos bits de decodificación. Así mismo, cualquier dirección que comience con 11 deberá ser una dirección del PIO. La lógica de decodificación completa requiere ahora dos compuertas NAND y un inversor. Puesto que se puede crear un inversor a partir de una compuerta NAND con sólo unir las dos entradas, un solo chip NAND cuádruple es ahora más que suficiente.

La lógica de decodificación de direcciones de la figura 3-57(b) se llama **decodificación de direcciones parcial**, porque no se usan las direcciones completas. Esta lógica tiene la propiedad de que una lectura de las direcciones 0001000000000000, 0001100000000000 o 0010000000000000 dará el mismo resultado. De hecho, todas las direcciones de la mitad inferior del espacio de direcciones seleccionarán la EPROM. Dado que no se usan las direcciones adicionales, no hay problema, pero si se está diseñando una computadora que podría expandirse en el futuro (algo que no es muy probable en un juguete), debe evitarse la decodificación parcial porque ocupa mucho espacio de direcciones.

Otra técnica común para decodificar direcciones es emplear un decodificador, como el que se muestra en la figura 3-13. Si conectamos las tres entradas a las tres líneas de dirección de orden alto, obtendremos ocho salidas, que corresponden a direcciones en los primeros 8K, los segundos 8K, y así. En el caso de una computadora con ocho memorias RAM, cada una de 8K × 8, un chip de este tipo basta para toda la decodificación. En el caso de una computadora con ocho chips de memoria de 2K × 8 también es suficiente un solo decodificador, siempre que cada chip de memoria esté situado en un bloque de 8K distinto del espacio de direcciones. (Recuerde lo que dijimos antes, de que la posición de los chips de memoria y de E/S dentro del espacio de direcciones es importante.)

3.8 RESUMEN

Las computadoras se construyen con chips de circuitos integrados que contienen diminutos elementos de conmutación llamados compuertas. Las compuertas más comunes son AND, OR, NAND, NOR y NOT. Es posible construir directamente circuitos sencillos combinando compuertas individuales.

Otros circuitos más complejos son los multiplexores, demultiplexores, codificadores, decodificadores, desplazadores y los ALU. Pueden programarse funciones booleanas arbitrarias empleando un PLA. Si se requieren muchas funciones booleanas, los PLA a menudo son más eficientes. Podemos utilizar las leyes del álgebra booleana para transformar circuitos de una forma a otra. En muchos casos es posible producir circuitos más económicos.

La aritmética de computadora se efectúa con sumadores. Se puede construir un sumador completo de un bit con dos medios sumadores. Un sumador para palabras de varios bits se puede construir conectando varios sumadores completos de modo que el acarreo de salida de cada uno se alimente a su vecino de la izquierda.

Los componentes de las memorias (estáticas) son latches y flip-flops, que pueden almacenar un bit de información. Éstos pueden combinarse linealmente para formar latches y flip-flops octales, o logarítmicamente para crear memorias a gran escala orientadas hacia las palabras. Los tipos de memoria disponibles son RAM, ROM, PROM, EPROM, EEPROM y flash. Las RAM estáticas no necesitan refrescarse; conservan sus valores en tanto hay alimentación eléctrica. Las RAM dinámicas, en cambio deben refrescarse periódicamente para compensar las fugas de los diminutos condensadores del chip.

Los componentes de un sistema de computación se conectan con buses. Muchas de las terminales de un chip de CPU típico, pero no todas, alimentan directamente una línea de bus. Las líneas de bus pueden dividirse en líneas de dirección, de datos y de control. Los buses sincrónicos se controlan con un reloj maestro. Los buses asincrónicos utilizan protocolos completos para sincronizar el esclavo con el iniciador.

El Pentium II es un ejemplo de CPU moderna. Los sistemas modernos que lo usan tienen un bus de memoria, un bus PCI, un bus ISA y un bus USB. El bus PCI puede transferir 64 bits a la vez con una rapidez de 66 MHz, lo que lo hace lo bastante rápido para casi todos los periféricos, pero no para la memoria.

Los interruptores, las lámparas, las impresoras y muchos otros dispositivos de E/S pueden conectarse a las computadoras utilizando chips de E/S paralela como el 8255A. Estos chips pueden configurarse de modo que sean parte del espacio de E/S o del espacio de memoria, según sea necesario. Pueden decodificarse total o parcialmente, dependiendo de la aplicación.

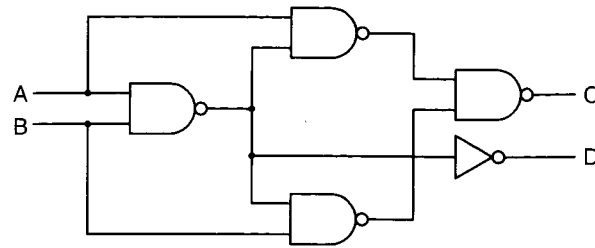
PROBLEMAS

1. Un lógico llega a un restaurante de comida rápida y dice: "Quiero una hamburguesa o una salchicha y papas fritas". Por desgracia, el cocinero no pasó del sexto grado y no sabe (ni le importa) si "y" tiene precedencia o no sobre "o". En lo que a él concierne, una interpretación es tan buena

como otra. ¿Cuáles de los siguientes casos son interpretaciones válidas de la orden? (Cabe señalar que en español "o" es una "o exclusiva".)

- a. Sólo una hamburguesa.
- b. Sólo una salchicha.
- c. Sólo papas fritas.
- d. Una salchicha y papas fritas.
- e. Una hamburguesa y papas fritas.
- f. Una salchicha y una hamburguesa.
- g. Las tres cosas.
- h. Nada; el lógico se queda con hambre por pasarse de listo.

2. Un misionero perdido en el sur de California se detiene en una bifurcación del camino. Él sabe que dos pandillas de motociclistas habitan en la región, una que siempre dice la verdad y una que siempre miente. Él quiere saber cuál camino lleva a Disneyland. ¿Qué pregunta deberá hacer?
3. Existen cuatro funciones booleanas de una sola variable y 16 funciones de dos variables. ¿Cuántas funciones de tres variables hay? ¿De n variables?
4. Utilice una tabla de verdad para demostrar que $P = (P \text{ AND } Q) \text{ OR } (P \text{ AND NOT } Q)$.
5. Muestre cómo puede construirse la función AND con dos compuertas NAND.
6. Use la ley de DeMorgan para encontrar el complemento de $\overline{AB}$.
7. Utilizando el chip multiplexor de tres variables de la figura 3-12, implemente una función cuya salida sea la paridad de las entradas, es decir, la salida es 1 si y sólo si un número impar de entradas son 1.
8. Póngase su gorro de pensar. El chip multiplexor de tres variables de la figura 3-12 en realidad puede calcular una función arbitraria de *cuatro* variables booleanas. Describa cómo y dé un ejemplo, dibuje el diagrama de lógica de la función que es 0 si la palabra en inglés para el número de fila de la tabla de verdad tiene un número impar de letras (por ejemplo, 0000 = cero = cuatro letras $\rightarrow 0$; 0111 = siete = cinco letras $\rightarrow 1$; 1101 = trece = ocho letras $\rightarrow 0$). *Sugerencia:* Si llamamos a la cuarta variable de entrada D , las ocho líneas de entrada podrían conectarse a V_{cc} , tierra, D o $\overline{D}$.
9. Dibuje el diagrama de lógica de un demultiplexor de 2 bits, un circuito cuya única línea de entrada se envía a una de las cuatro líneas de salida dependiendo del estado de las dos líneas de control.
10. Dibuje el diagrama de lógica de un codificador de dos bits, con cuatro líneas de entrada, de las cuales exactamente una está alta en cualquier instante, y dos líneas de salida cuyo valor binario de dos bits indica cuál entrada está alta.
11. Redibuje el PLA de la figura 3-15 con suficiente detalle para mostrar cómo puede implementarse la función lógica de mayoría de la figura 3-3. En particular, no olvide mostrar cuáles conexiones están presentes en ambas matrices.
12. ¿Qué hace este circuito?



13. Un chip MSI común es un sumador de 4 bits. Cuatro de estos chips pueden conectarse para formar un sumador de 16 bits. ¿Cuántas terminales esperaría usted que tuviera el chip sumador de 4 bits? ¿Por qué?
14. Se puede construir un sumador de n bits conectando en serie n sumadores completos, con el acarreo de entrada a la etapa i , C_i , tomado de la salida de la etapa $i - 1$. El acarreo de entrada de la etapa 0, C_0 , es 0. Si cada etapa tarda T ns en producir su suma y acarreo, el acarreo de entrada de la etapa i no será válido sino hasta iT ns después de iniciada la suma. Si n es grande, el tiempo necesario para que el acarreo se propague hasta la etapa de orden alto podría ser inaceptablemente largo. Diseñe un sumador que funcione con mayor rapidez. *Sugerencia:* Cada C_i se puede expresar en términos de los bits del operando A_{i-1} y B_{i-1} así como el acarreo C_{i-1} . Con esta relación es posible expresar C_i en función de las entradas de las etapas 0 a $i - 1$, y todos los acarreos pueden generarse simultáneamente.
15. Si todas las compuertas de la figura 3-19 tienen un retraso de propagación de 10 ns, y todos los demás retrasos son insignificantes, ¿en qué tiempo mínimo podemos estar seguros de que un circuito que usa este diseño tiene un bit de salida válido?
16. La ALU de la figura 3-20 puede realizar sumas de 8 bits en complemento a 2. ¿También puede efectuar restas en complemento a 2? Si esto es posible, explique cómo. Si no, modifíquela para que pueda efectuar restas.
17. A veces es útil que una ALU de 8 bits como la de la figura 3-20 genere la constante -1 como salida. Sugiera dos formas distintas de hacerlo. En cada caso, especifique los valores de las seis señales de control.
18. Una ALU de 16 bits se construye con 16 ALU de un bit, cada una de las cuales tiene un tiempo de suma de 10 ns. Si hay un retraso adicional de 1 ns por la propagación de una ALU a la siguiente, ¿cuánto tardará en aparecer el resultado de una suma de 16 bits?
19. ¿Cuál es el estado de reposo de las entradas S y R de un latch SR que se construye con dos compuertas NAND?
20. El circuito de la figura 3-26 es un flip-flop que se dispara en el flanco ascendente del reloj. Modifique este circuito para producir un flip-flop que se dispare en el flanco descendente del reloj.
21. Para ayudarse a hacer los pagos de su nueva computadora personal, usted está realizando trabajos de consultoría para compañías que se inician en la fabricación de chips SSI. Uno de sus clientes está pensando en sacar un chip que contenga cuatro flip-flops D, cada uno de los cuales produzca tanto Q como $\bar{Q}$, por solicitud de un cliente que podría ser importante. El diseño propuesto tiene las cuatro señales de reloj conectadas juntas, también por solicitud. No hay ni señal de preestablecer ni señal de borrar. Su tarea es dar una evaluación profesional del diseño.

22. La memoria 4×3 de la figura 3-29 usa 22 compuertas AND y tres compuertas OR. Si el circuito se fuera a expandir a 256×8 , ¿cuántas compuertas de cada tipo se necesitarían?
23. A medida que aumenta más y más la capacidad de memoria en un solo chip, también se incrementa el número de líneas necesarias para direccionarla. En muchos casos no es conveniente tener un gran número de líneas de dirección en un chip. Proponga una forma de direccionar 2^n palabras de memoria utilizando menos de n líneas.
24. Una computadora con un bus de datos de 32 bits emplea chips de memoria RAM dinámica $1M \times 1$. ¿Qué tamaño mínimo de memoria (en bytes) puede tener esta computadora?
25. Remitiéndonos al diagrama de temporización de la figura 3-37, suponga que frenara el reloj hasta tener un periodo de 40 ns en lugar de 25 ns como se muestra, pero que las restricciones de temporización no cambiaran. ¿Cuánto tiempo tendría la memoria para colocar los datos en el bus durante T_3 después de habilitarse $\overline{MREQ}$, en el peor de los casos?
26. Remítase otra vez a la figura 3-37 y suponga que el reloj sigue siendo de 40 MHz pero T_{AD} se aumentara a 16 ns. ¿Podrían seguir usándose chips de memoria de 40 ns?
27. En la figura 3-37(b) se especifica que T_{ML} es de por lo menos 6 ns. ¿Puede imaginar un chip en el que este tiempo sea negativo? En otras palabras, ¿la CPU podría habilitar $\overline{MREQ}$ antes de que la dirección se establezca? ¿Por qué sí o por qué no?
28. Suponga que la transferencia de bloque de la figura 3-41 se efectuará por el bus de la figura 3-37. ¿Qué ancho de banda adicional se obtendría usando una transferencia de bloque en lugar de transferencias individuales en el caso de bloques largos? Suponga ahora que el bus tiene una anchura de 32 bits en lugar de 8 bits. Conteste otra vez la pregunta.
29. Denote los tiempos de transición de las líneas de dirección de la figura 3-38 como T_{A1} y T_{A2} , y los tiempos de transición de $\overline{MREQ}$ como T_{MREQ1} y T_{MREQ2} , etc. Escriba todas las desigualdades implícitas en el saludo completo.
30. Casi todos los buses de 32 bits permiten lecturas y escrituras de 16 bits. ¿Existe ambigüedad respecto a dónde deben colocarse los datos? Comente.
31. Muchas CPU tienen un tipo de ciclo de bus especial para acusar recibo de interrupciones. ¿Por qué?
32. Una PC/AT de 10 MHz necesita cuatro ciclos para leer una palabra. ¿Qué ancho de banda de bus consume la CPU?
33. Una CPU de 32 bits con líneas de dirección A2-A31 requiere que todas las referencias a la memoria estén alineadas; es decir, las palabras tienen que direccionarse en múltiplos de 4 bytes, y las medias palabras tienen que direccionarse en bytes pares. Los bytes pueden estar en cualquier lugar. ¿Cuántas combinaciones válidas hay para las lecturas de memoria, y cuántas líneas se necesitan para expresarlas? Dé dos respuestas y justifique cada una.
34. ¿Por qué es imposible que el Pentium II funcione en un bus PCI de 32 bits sin perder funcionalidad? Después de todo, otras computadoras con un bus de datos de 64 bits pueden efectuar transferencias de 32 bits, 16 bits e incluso 8 bits.
35. Suponga que una CPU tiene una caché nivel 1 y una caché nivel 2, con tiempos de acceso de 5 ns y 10 ns, respectivamente. El tiempo de acceso a la memoria principal es de 50 ns. Si el 20% de los accesos son aciertos en la caché nivel 1 y el 60% son aciertos en la caché nivel 2, calcule el tiempo de acceso promedio.

36. ¿Es probable que un sistema picoJava II incorporado pequeño incluya un chip 8255A?
37. Calcule el ancho de banda de bus necesario para exhibir una película VGA (640 × 480) en color real a 30 cuadros/s. Suponga que los datos deben pasar por el bus dos veces, una vez del CD-ROM a la memoria y una vez de la memoria a la pantalla.
38. ¿Cuál señal del Pentium II cree usted que alimente la línea FRAME# del bus PCI?
39. ¿Cuál de las señales de la figura 3-53 no es estrictamente necesaria para que funcione el protocolo de bus?
40. Las instrucciones de cierta computadora requieren dos ciclos de bus cada una, uno para traer la instrucción y uno para traer los datos. Cada ciclo de bus tarda 250 ns y cada instrucción tarda 500 ns (es decir, el tiempo de procesamiento interno es insignificante). La computadora también tiene un disco con 16 sectores de 512 bytes por pista. El tiempo de rotación del disco es de 8.092 ms. ¿A qué porcentaje de su velocidad normal se reduce la computadora durante una transferencia de DMA si cada una de estas transferencias tarda un ciclo de bus? Considere dos casos: transferencias de bus de 8 bits y transferencias de bus de 16 bits.
41. La carga útil máxima de un paquete de datos isocrónico en el bus USB es de 1023 bytes. Si suponemos que un dispositivo sólo puede enviar un paquete de datos por trama, ¿qué ancho de banda máximo puede tener un solo dispositivo isocrónico?
42. ¿Qué efecto tendría añadir una tercera línea de entrada a la compuerta AND que selecciona el PIO de la figura 3-57(b) si dicha línea nueva se conectara a A13?
43. Escriba un programa que simule el comportamiento de una matriz $m \times n$ de compuertas NAND de dos entradas. Este circuito, contenido en un chip, tiene j terminales de entrada y k terminales de salida. Los valores de j , k , m y n son parámetros de la simulación que se fijan en el momento de la compilación. El programa debe iniciar con la lectura de una "lista de alambrado" en la que cada alambre especifica una entrada y una salida. Una entrada es una de las j terminales de entrada o bien la salida de alguna compuerta NAND. Una salida es una de las k terminales de salida o bien una entrada de alguna compuerta NAND. Las entradas que no se usan están en 1 lógico. Después de leer la lista de alambrado, el programa deberá imprimir la salida de cada una de las 2^j posibles entradas. Los chips con matrices de compuertas como éste se usan ampliamente para colocar circuitos personalizados en un chip porque casi todo el trabajo (depositar la matriz de compuertas en el chip) es independiente del circuito por implementar. Sólo el alambrado es específico para cada diseño.
44. Escriba un programa que lea dos expresiones booleanas arbitrarias y determine si representan la misma función. El lenguaje de entrada deberá incluir letras individuales como variables booleanas, los operandos AND, OR y NOT, y paréntesis. Cada expresión deberá caber en una línea de entrada. El programa deberá calcular las tablas de verdad de ambas funciones y compararlas.
45. Escriba un programa que lea una colección de expresiones booleanas y calcule las matrices 24×50 y 50×6 necesarias para implementar esas expresiones con el PLA de la figura 3-15. El lenguaje de entrada deberá ser el mismo que en el problema anterior. Imprima las matrices en la impresora de líneas.

4

EL NIVEL DE MICROARQUITECTURA

El nivel que está arriba del de lógica digital es el de la microarquitectura. Su tarea es implementar el nivel ISA (arquitectura de conjunto de instrucciones) que está arriba de él, como se ilustra en la figura 1-2. El diseño del nivel de microarquitectura depende de la ISA que se está implementando, así como de los objetivos de costo y desempeño de la computadora. Muchas ISA modernas, en especial los diseños RISC, tienen instrucciones sencillas que por lo regular pueden ejecutarse en un solo ciclo de reloj. Las ISA más complejas, como la de Pentium II, podrían requerir muchos ciclos para ejecutar una sola instrucción. Dicha ejecución podría requerir encontrar los operandos en la memoria, leerlos y almacenar resultados de vuelta en la memoria. La secuencia de operaciones dentro de una sola instrucción con frecuencia obliga a adoptar un diferente enfoque de control que con las ISA sencillas.

4.1 UN EJEMPLO DE MICROARQUITECTURA

Lo ideal sería comenzar explicando los principios generales del diseño de microarquitecturas. Lo malo es que no existen principios generales; cada una es un caso especial. Por tanto, optaremos por analizar un ejemplo detallado. Como ejemplo de ISA hemos escogido un subconjunto de la Máquina Virtual Java, como prometimos en el capítulo 1. Este subconjunto contiene sólo instrucciones para manejar enteros, por lo que lo hemos llamado **IJVM** (*Integer Java Virtual Machine*). Examinaremos la JVM completa en el capítulo 5.

Comenzaremos por describir la microarquitectura sobre la cual implementaremos IJVM. La IJVM tiene algunas instrucciones relativamente complejas. Muchas arquitecturas de este

tipo se han implementado con microprogramación, como vimos en el capítulo 1. Aunque IJVM es pequeña, es un buen punto de partida para describir el control y la secuencia de las instrucciones.

Nuestra microarquitectura contendrá un microprograma (en ROM) cuya tarea es buscar, decodificar y ejecutar instrucciones IJVM. No podemos usar el intérprete de JVM de Sun en vez del microprograma porque necesitamos un microprograma diminuto que alimente de forma eficiente las compuertas individuales del hardware real. El intérprete de JVM de Sun, en cambio, se escribió en C para ser transportable, y no puede controlar el hardware con el nivel de detalle que necesitamos. Puesto que el hardware real empleado consiste únicamente en los componentes básicos que describimos en el capítulo 3, en teoría, después de entender plenamente este capítulo, el lector deberá poder salir y comprar una gran bolsa llena de transistores, y construir este subconjunto de la máquina JVM. Los estudiantes que completen con éxito esta tarea recibirán crédito extra (y un examen psiquiátrico completo).

Un modelo conveniente para el diseño de la microarquitectura es ver el diseño como un problema de programación, en el que cada instrucción en el nivel ISA es una función que el programa maestro debe invocar. En este modelo, el programa maestro es un ciclo infinito simple que determina cuál función se invocará, llama a la función, y vuelve a comenzar, casi como se muestra en la figura 2-3.

El microprograma tiene un conjunto de variables, llamadas **estado** de la computadora, al que todas las funciones tienen acceso. Cada función modifica al menos algunas de las variables que constituyen el estado. Por ejemplo, el Contador de Programa (PC) forma parte del estado; indica la localidad de memoria que contiene la siguiente función (o sea, instrucción de ISA) que se ejecutará. Durante la ejecución de cada instrucción, el PC se incrementa para apuntar a la siguiente instrucción por ejecutar.

Las instrucciones de IJVM son cortas y precisas. Cada instrucción tiene unos cuantos campos, por lo regular uno o dos, cada uno de los cuales tiene un propósito específico. El primer campo de cada instrucción es el **código de operación (opcode)** que identifica la instrucción como, por ejemplo, ADD (sumar) o BRANCH (ramificar) o algo más. Muchas instrucciones tienen un campo adicional que especifica el operando. Por ejemplo, las instrucciones que accesan una variable local necesitan un campo para indicar *cuál* variable.

Este modelo de ejecución, llamado **ciclo de búsqueda-ejecución**, es útil en lo abstracto y también podría ser la base para implementar ISA como IJVM que tienen instrucciones complejas. A continuación describiremos cómo funciona, qué aspecto tiene la microarquitectura, y cómo se controla con las microinstrucciones, cada una de las cuales controla la trayectoria de los datos durante un ciclo. La lista de todas las microinstrucciones constituye el microprograma, que presentaremos y analizaremos con detalle.

4.1.1 La trayectoria de datos

La **trayectoria de datos** es la parte de la CPU que contiene la ALU, sus entradas y sus salidas. La trayectoria de datos de nuestro ejemplo de microarquitectura se muestra en la figura 4-1. Si bien esta trayectoria de datos se optimizó minuciosamente para interpretar programas en IJVM, es muy similar al que se usa en la mayor parte de las máquinas. Esta trayec-

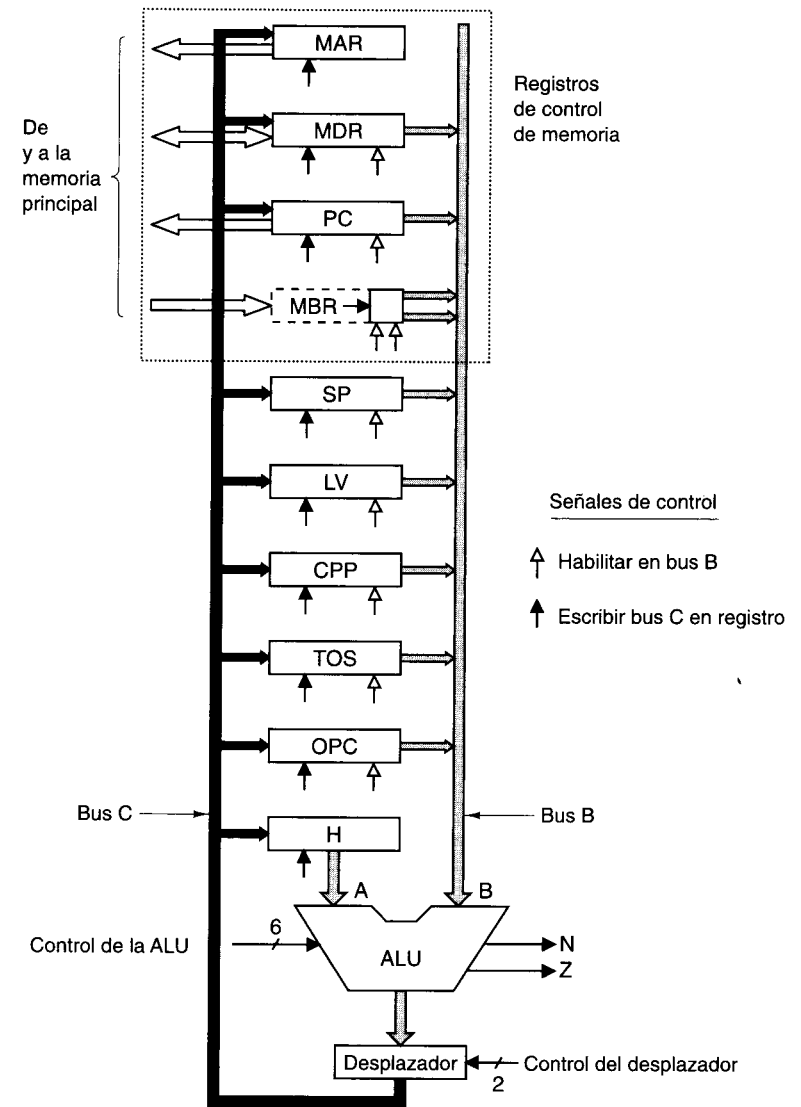


Figura 4-1. La trayectoria de datos de la microarquitectura que usaremos como ejemplo en este capítulo.

toria contiene varios registros de 32 bits, a los que hemos asignado nombres simbólicos como PC, SP y MDR. Aunque algunos de estos nombres son conocidos, es importante entender que estos registros pueden accersarse sólo en el nivel de microarquitectura (por el microprograma). Reciben estos nombres porque casi siempre contienen un valor que corresponde a la variable del mismo nombre en la arquitectura del nivel ISA. Casi todos los registros

pueden colocar su contenido en el bus B. La salida de la ALU alimenta al desplazador y luego al bus C, cuyo valor puede escribirse en uno o más registros al mismo tiempo. No existe un bus A por el momento; lo añadiremos después.

La ALU es idéntica a la que se muestra en la figura 3-19 y la figura 3-20. Su función está determinada por seis líneas de control. La línea diagonal corta rotulada con "6" en la figura 4-1 indica que hay seis líneas de control de la ALU. Éstas son F_0 y F_1 para determinar el funcionamiento de la ALU, ENA y ENB para habilitar individualmente las salidas, INVA para invertir la entrada izquierda, e INC para forzar un acarreo de entrada en el bit de orden bajo, lo que en realidad suma 1 al resultado. Sin embargo, no todas las 64 combinaciones de las líneas de control de la ALU hacen algo útil.

Algunas de las combinaciones más interesantes se muestran en la figura 4-2. No todas estas funciones se necesitan en la IJVM, pero muchas de ellas serían útiles en la JVM completa. En muchos casos hay varias posibilidades para obtener el mismo resultado. En esta tabla, + significa suma aritmética y - significa resta aritmética, de modo que, por ejemplo, -A indica el complemento a 2 de A.

F_0	F_1	ENA	ENB	INVA	INC	Función
0	1	1	0	0	0	A
0	1	0	1	0	0	B
0	1	1	0	1	0	$\bar{A}$
1	0	1	1	0	0	$\bar{B}$
1	1	1	1	0	0	A+B
1	1	1	1	0	1	A+B+1
1	1	1	0	0	1	A+1
1	1	0	1	0	1	B+1
1	1	1	1	1	1	B-A
1	1	0	1	1	1	B-1
1	1	1	0	1	1	-A
0	0	1	1	0	0	A AND B
0	1	1	1	0	0	A OR B
0	1	0	0	0	0	0
0	1	0	0	0	1	1
0	1	0	0	1	0	-1

Figura 4-2. Combinaciones útiles de señales de la ALU y la función que desempeñan.

La ALU de la figura 4-1 necesita dos entradas de datos: una entrada izquierda (A) y una entrada derecha (B). La entrada izquierda tiene conectado un registro de retención, H. La entrada derecha está conectada al bus B, que puede cargarse desde cualquiera de nueve orígenes, como indican las nueve flechas grises que lo tocan. Un diseño alternativo, con dos buses completos, tiene un conjunto diferente de ventajas y desventajas, mismas que veremos más adelante en este capítulo.

H se puede cargar inicialmente escogiendo una función de la ALU que se limite a pasar la entrada derecha (del bus B) hasta la salida de la ALU. Una función así sería sumar las entradas de la ALU, sólo que con ENA deshabilitada para que la entrada izquierda sea forzosamente cero. Sumar cero al valor del bus B produce ese mismo valor. Luego, este resultado puede pasarse por el desplazador sin modificación y guardarse en H.

Además de las funciones anteriores, se pueden usar otras dos líneas de control de forma independiente para controlar la salida de la ALU. SLL8 (desplazamiento lógico a la izquierda, *Shift Left Logical*) desplaza el contenido un byte a la izquierda, llenando los ocho bits menos significativos con ceros. SRA1 (desplazamiento aritmético a la derecha, *Shift Right Arithmetic*) desplaza el contenido un bit a la derecha, sin modificar el bit más significativo.

Es posible explícitamente leer y escribir el mismo registro en un ciclo. Por ejemplo, está permitido colocar SP en el bus B, deshabilitar la entrada izquierda de la ALU, habilitar la señal INC y guardar el resultado en SP con lo que se incrementa SP en 1 (vea la octava línea de la figura 4-2). ¿Cómo puede leerse y escribirse un registro en el mismo ciclo sin producir basura? La solución es que la lectura y la escritura se realizan realmente en diferentes momentos dentro del ciclo. Cuando un registro se selecciona como entrada derecha de la ALU, su valor se coloca en el bus B al principio del ciclo y se mantiene ahí continuamente durante todo el ciclo. Luego, la ALU efectúa su trabajo y produce un resultado que pasa al bus C a través del desplazador. Cerca del final del ciclo, cuando se sabe que las salidas de la ALU y del desplazador están estables, una señal de reloj activa el almacenamiento del bus C en uno más de los registros. Uno de estos registros bien podría ser el que proporcionó su entrada al bus B. La temporización precisa de la trayectoria de datos hace posible leer y escribir el mismo registro en un ciclo, como se describe en seguida.

Temporización de la trayectoria de datos

La temporización de estos sucesos se muestra en la figura 4-3. Aquí se produce una pulsación corta al principio de cada ciclo de reloj. La pulsación se puede sacar del reloj principal, como se muestra en la figura 3-21(c). En el flanco descendente de la pulsación, ya están preparados los bits que alimentarán todas las compuertas. Esto tarda un tiempo finito conocido, Δw . Luego se selecciona el registro que se necesita en el bus B y su valor se pasa a ese bus. Se requiere un tiempo Δx para que el valor se estabilice. Luego la ALU y el desplazador comienzan a operar con datos válidos. Después de un tiempo Δy adicional, las salidas de la ALU y el desplazador se han estabilizado. Después de otro intervalo Δz , los resultados se han propagado por el bus C hasta los registros, en los que pueden cargarse durante el flanco ascendente de la siguiente pulsación. La carga debe ser disparada por flanco y rápida, de modo que aun si algunos de los registros de entrada se modifican, los efectos no se harán sentir en el bus C sino hasta mucho después de haberse cargado todos los registros. También en el flanco ascendente de la pulsación, el registro que alimenta al bus B deja de hacerlo, en preparación del siguiente ciclo. MPC, MIR y la memoria se mencionan en la figura; exploraremos sus papeles en breve.

Es importante darse cuenta de que aunque no hay elementos de almacenamiento en la trayectoria de datos, hay un tiempo de propagación finito a través suyo. Modificar el valor

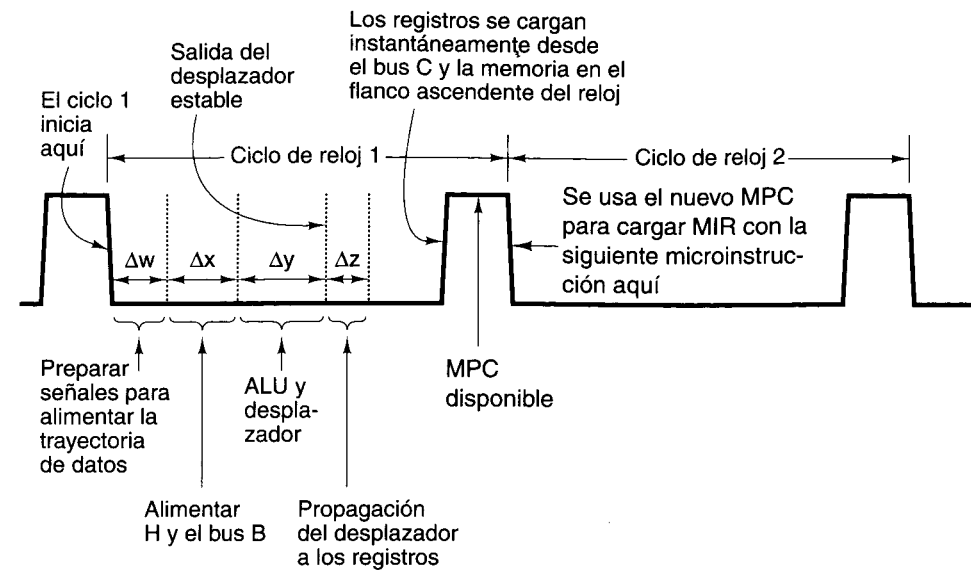


Figura 4-3. Diagrama de temporización de un ciclo de la trayectoria de datos.

que está en el bus B no hace que el bus C cambie sino hasta después de un tiempo finito (debido a los retrasos finitos en cada paso). Por consiguiente, aun si una operación de almacenamiento modifica uno de los registros de entrada, el valor ya estará guardado a salvo en el registro mucho antes de que el valor (ahora incorrecto) que se está colocando en el bus B (o en H) pueda llegar a la ALU.

Para que este diseño funcione se requiere una temporización rígida, un ciclo de reloj largo, un tiempo de propagación mínimo conocido a través de la ALU y un cargado rápido de los registros desde el bus C. Sin embargo, con una ingeniería cuidadosa se puede diseñar la trayectoria de datos de modo que funcione correctamente.

Una forma un tanto distinta de ver la trayectoria de datos es dividirlo conceptualmente en subciclos implícitos. El inicio del subciclo 1 se dispara por el flanco descendente del reloj. A continuación mostramos las actividades que ocurren durante los subciclos, junto con las longitudes de los subciclos (entre paréntesis).

1. Se preparan las señales de control (Δw).
2. Los registros se cargan en el bus B (Δx).
3. La ALU y el desplazador operan (Δy).
4. Los resultados se propagan por el bus C de regreso a los registros (Δz).

En el flanco ascendente del siguiente ciclo de reloj, los resultados se almacenan en los registros.

Dijimos que la mejor forma de ver los subciclos es como algo *implícito*. Con esto queremos decir que no hay pulsaciones de reloj ni otras señales implícitas que indiquen a la ALU cuándo debe operar o que digan a los resultados que entren en el bus C. En realidad, la ALU y

el desplazador están operando todo el tiempo. Sin embargo, sus entradas son basura hasta un tiempo $\Delta w + \Delta x$ después del flanco descendente del reloj. De igual manera, sus salidas son basura hasta que ha transcurrido $\Delta w + \Delta x + \Delta y$ después del flanco descendente del reloj. Las únicas señales explícitas que controlan la trayectoria de datos son el flanco descendente del reloj, que inicia el ciclo de la trayectoria de datos, y el flanco ascendente del reloj, que carga los registros a partir del bus C. Las otras fronteras de los subciclos están determinadas implícitamente por los tiempos de propagación inherentes a los circuitos participantes. Es responsabilidad de los ingenieros de diseño asegurarse de que el tiempo $\Delta w + \Delta x + \Delta y + \Delta z$ termine con tiempo de sobra antes del flanco ascendente del reloj, si se quiere que el cargado de los registros funcione siempre.

Operación de la memoria

Nuestra máquina tiene dos formas diferentes de comunicarse con la memoria: un puerto de memoria de 32 bits direccionable por palabra y un puerto de memoria de 8 bits direccionable por byte. El puerto de 32 bits está bajo el control de dos registros, **MAR (registro de dirección de memoria, Memory Address Register)** y **MDR (registro de datos de memoria, Memory Data Register)**, como se muestra en la figura 4-1. El puerto de ocho bits está bajo el control de un registro, **PC**, que lee un byte y lo coloca en los ocho bits de orden bajo de **MBR**. Este puerto sólo puede leer datos de la memoria; no puede escribir datos en la memoria.

Cada uno de estos registros (y todos los demás registros de la figura 4-1) operan con una o dos **señales de control**. Una flecha blanca bajo un registro indica una señal de control que habilita la colocación de la salida del registro en el bus B. Puesto que **MAR** no está conectado con el bus B, no tiene una señal de habilitación. **H** tampoco tiene una porque siempre está habilitado, al ser la única posible entrada izquierda de la ALU.

Una flecha negra bajo un registro indica una señal de control que escribe (es decir, carga) el registro a partir del bus C. Puesto que **MBR** no se puede cargar a partir del bus C, no tiene una señal de escritura (aunque sí tiene otras dos señales de habilitación, que se describirán más adelante). Para iniciar una lectura o escritura de memoria, es preciso cargar los registros de memoria apropiados, y luego emitir una señal de lectura o escritura a la memoria (no se muestra en la figura 4-1).

MAR contiene direcciones de *palabra*, de modo que los valores 0, 1, 2, etc., se refieren a palabras consecutivas. **PC** contiene direcciones de *byte*, de modo que los valores 0, 1, 2, etc., se refieren a bytes consecutivos. Así, colocar un 2 en **PC** e iniciar una lectura de memoria hará que se lea el byte 2 de la memoria y se coloque en los 8 bits de orden bajo de **MBR**. Colocar un 2 en **MAR** e iniciar una lectura de memoria hará que se lean los bytes 8-11 (o sea, la palabra 2) de la memoria y se coloquen en **MDR**.

Esta diferencia en funcionalidad es necesaria porque **MAR** y **PC** se usarán para hacer referencia a dos partes diferentes de la memoria. La necesidad de esta distinción se aclarará posteriormente. Por ahora, basta decir que la combinación **MAR/MDR** se usa para leer y escribir palabras de datos en el nivel ISA, y la combinación **PC/MBR** sirve para leer el programa ejecutable en el nivel ISA, que consiste en un flujo de bytes. Todos los demás registros que contienen direcciones emplean direcciones de palabra, como **MAR**.

En la implementación física real, sólo hay una memoria y está organizada en bytes. Un sencillo truco permite a MAR contar en palabras (lo cual es necesario debido a la forma en que se definió la JVM) mientras la memoria física cuenta en bytes. Cuando MAR se coloca en el bus de direcciones, sus 32 bits no se corresponden directamente con las 32 líneas de dirección, 0-31. En vez de ello, el bit 0 de MAR se conecta con la línea 2 del bus de direcciones, el bit 1 de MAR se conecta con la línea 3 del bus de direcciones, y así. Los 2 bits superiores de MAR se desechan porque sólo se necesitan para las direcciones de palabra mayores que 2^{32} , que no son válidas en nuestra máquina de 4 GB. Con esta transformación, cuando MAR es 1, se coloca la dirección 4 en el bus; cuando MAR es 2, se coloca la dirección 8 en el bus, y así. Este truco se ilustra en la figura 4-4.

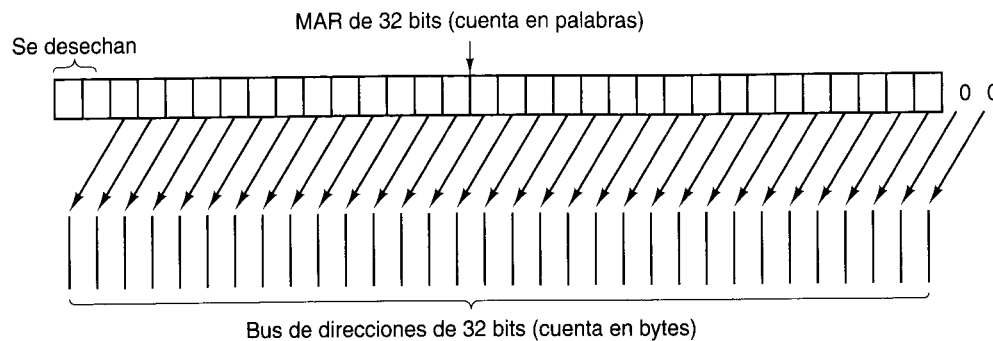


Figura 4-4. Correspondencia de los bits de MAR con el bus de direcciones.

Como ya se mencionó, los datos leídos de la memoria a través del puerto de memoria de ocho bits se devuelven en MBR, un registro de 8 bits. MBR puede copiarse en el bus B de una de dos formas: sin signo o con signo. Cuando se necesita un valor sin signo, la palabra de 32 bits que se coloca en el bus B contiene el valor de MBR en los 8 bits de orden bajo y ceros en los 24 bits superiores. Los valores sin signo son útiles como índices de tablas, o cuando es preciso armar un entero de 16 bits con dos bytes (sin signo) consecutivos del flujo de instrucciones.

La otra opción para convertir el MBR de 8 bits en una palabra de 32 bits es tratarlo como un valor con signo entre -128 y $+127$ y usar este valor para generar una palabra de 32 bits que tenga el mismo valor numérico. Esta conversión se efectúa copiando el bit de signo de MBR (el bit de la extrema izquierda) en las 24 posiciones de bit superiores del bus B, proceso que se llama **extensión del signo**. Cuando se escoge esta opción, los 24 bits superiores serán todos ceros o todos unos, dependiendo de si el bit de la izquierda del MBR es 0 o 1.

La decisión de convertir el MBR de 8 bits en un valor de 32 bits con o sin signo en el bus B depende de cuál de las dos líneas de control (flechas blancas debajo de MBR en la figura 4-1) está habilitada. Hay dos flechas porque se necesitan estas dos opciones. La capacidad de hacer que el MBR de 8 bits actúe como una fuente de 32 bits para el bus B se indica con el rectángulo punteado a la izquierda de MBR en la figura.

4.1.2 Microinstrucciones

Para controlar la trayectoria de datos de la figura 4-1 necesitamos 29 señales. Éstas se pueden dividir en cinco grupos funcionales, como se describe a continuación.

- 9 Señales para controlar la escritura de datos del bus C en registros.
- 9 Señales para controlar la habilitación de registros en el bus B para introducción en la ALU.
- 8 Señales para controlar las funciones de la ALU y el desplazador.
- 2 Señales (que no se muestran) para indicar lectura/escritura de memoria vía MAR/MDR.
- 1 Señal (que no se muestra) para indicar obtención de memoria vía PC/MBR.

Los valores de estas 29 señales de control especifican las operaciones durante un ciclo de la trayectoria de datos. Un ciclo consiste en colocar valores de los registros en el bus B, propagar las señales a través de la ALU y el desplazador, alimentarlas al bus C y por último escribir los resultados en el registro o registros apropiados. Además, si una señal de lectura de datos de memoria está habilitada, la operación de memoria se inicia al final del ciclo de la trayectoria de datos, una vez que se ha cargado MAR. Los datos de memoria están disponibles hasta el final del *siguiente* ciclo en MBR o MDR y se usa el ciclo *que sigue*. En otras palabras, una lectura de memoria por cualquiera de los puertos, iniciada al final del ciclo k , proporciona datos que no se usan en el ciclo $k + 1$, sino hasta el ciclo $k + 2$ o uno posterior.

Este comportamiento que aparentemente va contra la intuición se explica con la figura 4-3. Las señales de control de la memoria no se generan en el ciclo de reloj 1 sino hasta inmediatamente después de que MAR y PC se cargan en el flanco ascendente del reloj, hacia el final del ciclo de reloj 1. Supondremos que la memoria coloca sus resultados en los buses de memoria en el plazo de un ciclo, de modo que MBR y/o MDR se puedan cargar en el siguiente flanco ascendente del reloj, junto con los demás registros.

En otras palabras, cargamos MAR al final de un ciclo de la trayectoria de datos e iniciamos la lectura de memoria poco tiempo después. Por tanto, no podemos esperar realmente que los resultados de una operación de lectura estén en MDR al principio del siguiente ciclo, sobre todo si la pulsación de reloj es angosta. Si la memoria tarda un ciclo de reloj, simplemente no hay suficiente tiempo. Debe transcurrir un ciclo de la trayectoria de datos completa entre que se inicia una lectura de memoria y que se usa el resultado. Desde luego, se efectúan otras operaciones durante ese ciclo, pero no las que necesitan la palabra de la memoria.

El supuesto de que la memoria tarda un ciclo en operar equivale a suponer que la tasa de aciertos en caché es de 100%. Este supuesto nunca se cumple, pero la complejidad que implica tener un ciclo de memoria de longitud variable es mayor que con la que queremos lidiar aquí.

Puesto que MBR y MDR se cargan en el flanco ascendente del reloj, junto con todos los demás registros, podrían leerse durante ciclos en los que se está efectuando una nueva lectura de memoria. Esa lectura devuelve los valores antiguos, ya que todavía no ha habido tiempo de sobrescribirlos. No hay ambigüedad aquí; hasta que se carguen valores nuevos en MBR y MDR en el flanco ascendente del reloj, estos registros contendrán sus valores anteriores,

los cuales se podrán usar. Cabe señalar que es posible realizar lecturas una tras otra en dos ciclos consecutivos puesto que una lectura sólo tarda un ciclo. Además, ambas memorias podrían operar al mismo tiempo. Sin embargo, tratar de leer y escribir el mismo byte simultáneamente produce resultados no definidos.

Si bien podría ser deseable escribir la salida colocada en el bus C en más de un registro, nunca es conveniente habilitar más de un registro para el bus B al mismo tiempo. (De hecho, algunas implementaciones reales sufren daños físicos si se hace esto.) Si se agregan unos circuitos más, podemos reducir el número de bits necesarios para seleccionar una de las posibles fuentes que alimentan al bus B. Sólo hay nueve registros de entrada que pueden alimentar al bus B (si contamos por separado las versiones con signo y sin signo de MBR). Por tanto, podemos codificar la información del bus B en cuatro bits y usar un decodificador para generar las 16 señales de control, siete de las cuales no se necesitan. En un diseño comercial, los arquitectos estarían fuertemente tentados a deshacerse de uno de los registros para poder efectuar la tarea con sólo 3 bits. Como académicos, podemos darnos el lujo de desperdiciar un bit para contar con un diseño más limpio y sencillo.

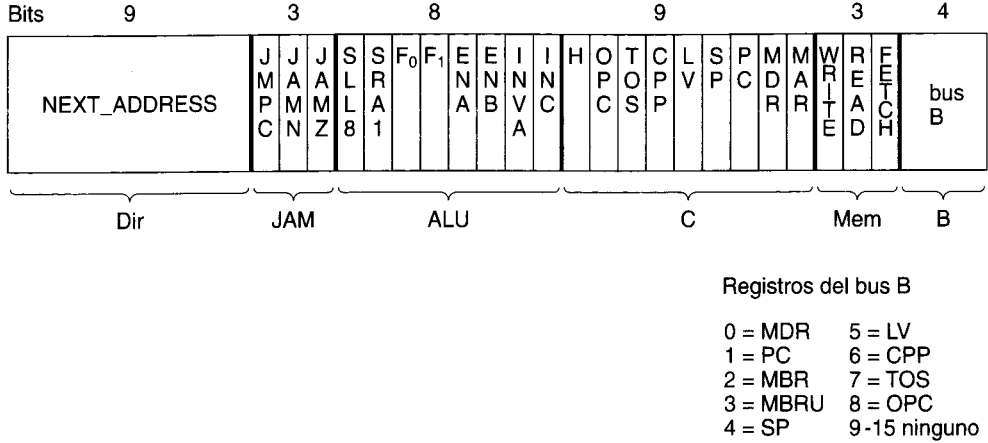


Figura 4-5. El formato de microinstrucciones para el Mic-1.

A estas alturas podemos controlar la trayectoria de datos con $9 + 4 + 8 + 2 + 1 = 24$ señales, y por tanto 24 bits. Sin embargo, estos 24 bits sólo controlan la trayectoria de datos durante un ciclo. La segunda parte del control consiste en determinar qué debe hacerse en el siguiente ciclo. Para incluir esto en el diseño del controlador, crearemos un formato para describir las operaciones que se efectuarán usando los 24 bits de control más dos campos adicionales: el campo NEXT_ADDRESS y el campo JAM. A continuación explicaremos el contenido de cada uno de estos campos. La figura 4-5 muestra un posible formato, dividido en los seis grupos, y que contiene las 36 señales siguientes:

- Dir – Contiene la dirección de una microinstrucción que podría ser la siguiente.
- JAM – Determina cómo se selecciona la siguiente microinstrucción.
- ALU – Funciones de la ALU y el desplazador.
- C – Selecciona cuáles registros se escriben desde el bus C.
- Mem – Funciones de memoria.
- B – Selecciona la fuente del bus B; se codifica como se muestra.

El ordenamiento de los grupos es arbitrario en principio, aunque en realidad se escogió con mucho cuidado a fin de minimizar los cruces de líneas en la figura 4-6. Los cruces de línea en diagramas esquemáticos como el de la figura 4-6 a menudo corresponden a cruces de hilos en chips, que pueden causar problemas en los diseños bidimensionales y deben evitarse.

4.1.3 Control de microinstrucciones: el Mic-1

Hasta aquí hemos descrito cómo se controla la trayectoria de datos, pero todavía no hemos explicado cómo se decide cuál de las señales de control debe estar habilitada en cada ciclo. Esto lo determina un **secuenciador** que se encarga de recorrer la secuencia de operaciones necesarias para ejecutar una sola instrucción ISA.

El secuenciador debe producir dos tipos de información en cada ciclo:

1. El estado de cada señal de control en el sistema.
2. La dirección de la microinstrucción que debe ejecutarse a continuación.

La figura 4-6 es un diagrama de bloques detallado de la microarquitectura completa de nuestra máquina de ejemplo, que llamaremos **Mic-1**. A primera vista este diagrama podría parecer abrumador, pero vale la pena estudiarlo con detenimiento. Una vez que entienda plenamente todos y cada uno de los bloques y líneas de esta figura, habrá adelantado mucho en la comprensión del nivel de microarquitectura. El diagrama de bloques tiene dos partes: la trayectoria de datos, a la izquierda, que ya vimos con detalle, y la sección de control, a la derecha, que examinaremos a continuación.

El elemento más grande e importante de la porción de control de la máquina es una memoria llamada **almacén de control**. Es recomendable ver este “almacén” como una memoria que contiene todo el microprograma, aunque a veces se implementa como un conjunto de compuertas lógicas. En general, nos referiremos a él como almacén de control para evitar confundirlo con la memoria principal, a la que se accede a través de MBR y MDR. Sin embargo, en lo funcional, el almacén de control no es más que una memoria que contiene microinstrucciones en lugar de instrucciones ISA. En nuestra máquina de ejemplo, el almacén contiene 512 palabras, cada una de las cuales consiste en una microinstrucción de 36 bits como las que se ilustran en la figura 4-5. En realidad, no se necesitan todas estas palabras pero (por razones que se explicarán en breve) necesitamos direcciones para 512 palabras distintas.

Es importante notar que el almacén de control es muy diferente de la memoria principal: las instrucciones de la memoria principal tienen la propiedad de que se ejecutan en orden

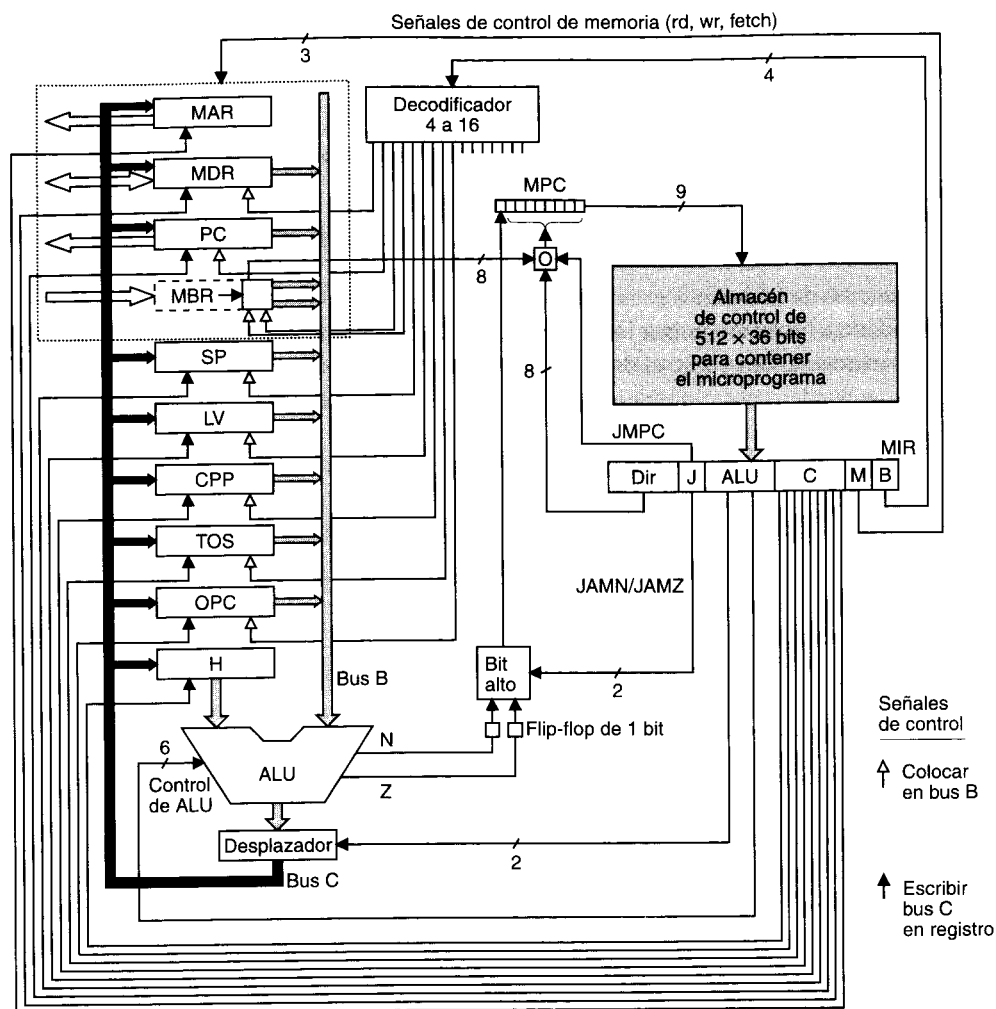


Figura 4-6. Diagrama de bloques completo de nuestra microarquitectura de ejemplo, Mic-1.

según su dirección (con excepción de las ramificaciones); las microinstrucciones no. El acto de incrementar el contador de programa en la figura 2-3 expresa el hecho de que la instrucción que se ejecutará por omisión después de la actual es la que sigue a la actual en la memoria. Los microprogramas necesitan mayor flexibilidad (porque las secuencias de microinstrucciones tienden a ser cortas), así que generalmente no tienen esta propiedad. En lugar de esto, cada microinstrucción especifica explícitamente su sucesora.

Puesto que el almacén de control es funcionalmente una memoria (sólo de lectura), necesita su propio registro de direcciones de memoria y su propio registro de datos de memoria. No necesita señales de leer y escribir, porque continuamente se está leyendo. Llamaremos al registro de dirección de memoria del almacén de control **MPC (contador de microprograma, MicroProgram Counter)**. Este nombre es irónico porque las localidades que contiene no están ordenadas, explícitamente, de modo que el concepto de contar no es útil (pero, ¿quiénes somos nosotros para discutir con la tradición?). El registro de datos de memoria se llama **MIR (registro de microinstrucción, MicroInstruction Register)**. Su función es almacenar la microinstrucción actual, cuyos bits alimentan las señales de control que operan la trayectoria de datos.

El registro **MIR** de la figura 4-6 contiene los mismos seis grupos de la figura 4-5. Los grupos **Dir** y **J** (por **JAM**) controlan la selección de la siguiente microinstrucción y se estudiarán más adelante. El grupo **ALU** contiene ocho bits que seleccionan la función de la ALU y controlan el desplazador. Los bits **C** hacen que registros individuales carguen la salida de la ALU desde el bus **C**. Los bits **M** controlan las operaciones de memoria.

Por último, los cuatro bits finales controlan el decodificador que determina qué se coloca en el bus B. En este caso hemos escogido un decodificador estándar 4 a 16, aunque sólo se requieren nueve posibilidades. En un diseño más afinado se usaría un decodificador de 4 a 9. La ventaja aquí es poder usar un circuito estándar tomado de una biblioteca de circuitos en lugar de diseñar uno a la medida. El uso del circuito estándar es más fácil y reduce la posibilidad de introducir errores. Crear circuitos propios ocupa menos área de chip pero tarda más y podríamos tener errores.

La operación de la figura 4-6 es como sigue. Al principio de cada ciclo de reloj (el flanco descendente del reloj de la figura 4-3), **MIR** se carga con la palabra del almacén de control a la que **MPC** apunta. El tiempo de carga de **MIR** se indica en la figura con Δw . Si pensamos en términos de subciclos, **MIR** se carga durante el primero.

Una vez que la microinstrucción está en **MIR**, las diversas señales se propagan a la trayectoria de datos. Se coloca un registro en el bus B, la ALU sabe qué instrucción debe ejecutar y ocurren muchas actividades más. Éste es el segundo subciclo. Después de un intervalo $\Delta w + \Delta x$ a partir del principio del ciclo, las entradas de la ALU se han estabilizado.

Después de transcurrir un tiempo Δy adicional, todo se ha calmado otra vez y las salidas de la ALU, N, Z y el desplazador están estables. Luego, los valores de N y Z se guardan en un par de flip-flops de un bit. Estos bits, al igual que todos los registros que se cargan a partir del bus C y la memoria, se guardan en el flanco ascendente del reloj, cerca del final del ciclo de la trayectoria de datos. La salida de la ALU no se almacena, sólo se alimenta al desplazador. La actividad de la ALU y el desplazador ocurre durante el subciclo 3.

Después de un intervalo adicional, Δz , la salida del desplazador ha llegado a los registros por el bus C. Luego los registros pueden cargarse cerca del final del ciclo (en el flanco ascendente de la pulsación de reloj de la figura 4-3). El subciclo 4 consiste en el cargado de los registros y los flip-flops N y Z; termina un poco después del flanco ascendente del reloj, cuando todos los resultados se han guardado y los resultados de las operaciones de memoria anteriores están disponibles, y se ha cargado MPC. Este proceso continúa hasta que alguien se aburre y apaga la máquina.

Al mismo tiempo que controla la trayectoria de datos, el microprograma tiene que determinar qué microinstrucción ejecutará a continuación, porque no se ejecutan en el orden en que aparecen en el almacén de control. El cálculo de la dirección de la siguiente microinstrucción se inicia una vez que MIR se ha cargado y estabilizado. Primero se copia el campo **NEXT_ADDRESS** de 9 bits en MPC. Mientras se está efectuando este copiado, se inspecciona el campo JAM. Si tiene el valor 000, no se hace nada más; una vez que concluye el copiado de **NEXT_ADDRESS**, MPC apunta a la siguiente microinstrucción.

Si uno o más de los bits de JAM son 1, hay que trabajar más. Si JAMN está activado, el flip-flop N de un bit se hace OR con el bit de orden alto de MPC. Del mismo modo, si JAMZ está establecido, el flip-flop Z de un bit realiza la operación OR. Si ambos están establecidos, se realiza la operación OR de ellos. Se necesitan los flip-flops N y Z porque después del flanco ascendente del reloj (mientras el reloj está alto), el B bus ya no está siendo controlado, y no puede suponerse que las salidas de la ALU sigan siendo correctas. Guardar las banderas de estado de la ALU en N y Z hace que los valores correctos se establezcan y estén disponibles para el cálculo de MPC, sin importar qué esté ocurriendo en torno de la ALU.

En la figura 4-6 la lógica que efectúa este cálculo se denomina “Bit alto”. La función booleana que calcula es

$$F = (\text{JAMZ AND Z}) \text{ OR } (\text{JAMN AND N}) \text{ OR } \text{NEXT_ADDRESS}[8]$$

Observe que, en todos los casos, MPC puede adoptar sólo uno de dos posibles valores:

1. El valor de **NEXT_ADDRESS**.
2. El valor de **NEXT_ADDRESS** después de que se realizó el OR del bit de orden alto con 1.

No existen otras posibilidades. Si el bit de orden alto de **NEXT_ADDRESS** era 1, no tiene sentido usar JAMN o JAMZ.

Observe que cuando todos los bits de JAM son ceros, la dirección de la siguiente microinstrucción a ejecutar es simplemente el número de nueve bits que está en su campo **NEXT_ADDRESS**. Si JAMN o JAMZ son 1, hay dos posibles sucesores, **NEXT_ADDRESS** y el resultado de la operación OR de **NEXT_ADDRESS** con 0x100 (suponiendo que $\text{NEXT_ADDRESS} \leq 0xFF$). (Cabe señalar que 0x indica que el número que sigue está en hexadecimal.) Este punto se ilustra en la figura 4-7. La microinstrucción en curso, en la posición 0x75, tiene **NEXT_ADDRESS** = 0x92 y **JAMZ** = 1. Por tanto, la siguiente dirección de la microinstrucción depende del bit Z almacenado en la operación de ALU previa. Si el bit Z es 0, la siguiente microinstrucción provendrá de 0x92. Si el bit Z es 1, la siguiente microinstrucción provendrá de 0x192.

El tercer bit del campo JAM es JMPC. Si está activo, se efectúa la operación OR bit por bit de los ocho bits de MBR con los 8 bits de orden bajo del campo **NEXT_ADDRESS** de la microinstrucción en curso. El resultado se envía a MPC. El cuadro rotulado “O” en la figura 4-6 efectúa un OR de MBR con **NEXT_ADDRESS** si JMPC es 1, pero se limita a pasar **NEXT_ADDRESS** directamente a MPC si JMPC es 0. Cuando JMPC es 1, los ocho bits de orden bajo de **NEXT_ADDRESS** normalmente son cero. El bit de orden alto puede ser 0 o 1, de modo que el valor de **NEXT_ADDRESS** que se usa con JMPC normalmente es 0x000 o 0x100. La razón para usar a veces 0x000 y a veces 0x100 se explicará más adelante.

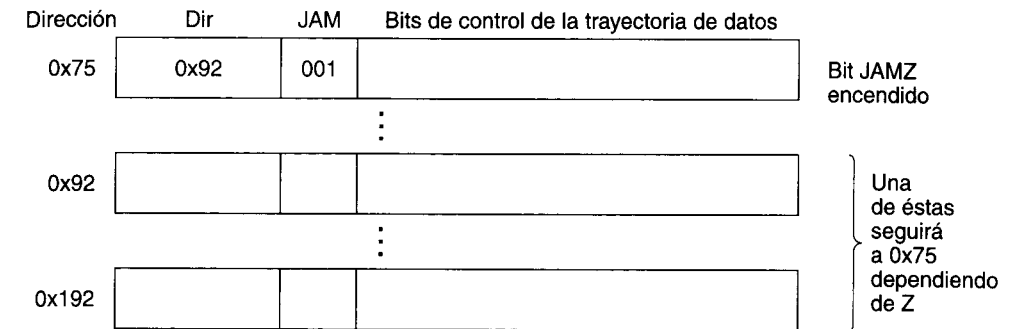


Figura 4-7. Una microinstrucción con JAMZ puesto en 1 tiene dos posibles sucesoras.

La capacidad para calcular el OR de MBR con **NEXT_ADDRESS** y almacenar el resultado en MPC hace posible una implementación eficiente de una ramificación (salto) de múltiples vías. Observe que es posible especificar cualquiera de 256 direcciones, determinada exclusivamente por los bits presentes en MBR. En un uso típico, MBR contiene un código de operación, de modo que el uso de JMPC hará que se seleccione para ejecutarse en seguida una microinstrucción única para cada posible código de operación. Este método es útil para saltar directamente a la función que corresponde al código de operación encontrado.

Para la explicación que sigue es crucial entender la temporización de la máquina, así que tal vez valga la pena repetirla. Lo haremos en términos de subciclos, ya que esto es más fácil de visualizar, los sucesos de reloj reales son el flanco descendente, que inicia el ciclo, y el flanco ascendente, que carga los registros y los flip-flops N y Z. Remítase por favor a la figura 4-3.

Durante el subciclo 1, iniciado por el flanco descendente del reloj, MIR se carga con la dirección que actualmente está en MPC. Durante el subciclo 2, las señales de MIR se propagan hacia afuera y el bus B se carga a partir del registro seleccionado. Durante el subciclo 3, la ALU y el desplazador operan y producen un resultado estable. Durante el subciclo 4, el bus C, los buses de memoria y los valores de la ALU se estabilizan. En el flanco ascendente del reloj los registros se cargan a partir del bus C, los flip-flops N y Z se cargan, y MBR y MDR obtienen sus resultados de la operación de memoria que se inició al final del ciclo de trayectoria de datos anterior (si se inició alguna de esas operaciones). Tan pronto como está disponible MBR, MPC se carga en preparación para la siguiente microinstrucción. Así MPC obtiene su valor en algún momento durante la parte media del intervalo en el que el reloj está alto pero después de que MBR/MDR están listos. Esto podría ser disparado por nivel (en lugar de por flanco) o disparado por flanco con un retraso fijo después del flanco ascendente del reloj. Lo único que importa es que MPC no se cargue sino hasta que los registros de los que depende (MBR, N y Z) estén listos. Tan pronto como el reloj baja, MPC puede direccionar el almacén de control y se puede iniciar un ciclo nuevo.

Cabe señalar que cada ciclo es autosuficiente: especifica qué se coloca en el bus B, qué deben hacer la ALU y el desplazador, dónde debe almacenarse el valor del bus C y, por último, qué valor debe tener MPC a continuación.

Vale la pena un último comentario acerca de la figura 4-6. Hemos estado tratando a MPC como un registro normal, con 9 bits de capacidad, que se carga mientras el reloj está alto. En realidad, no hay necesidad de tener un registro ahí. Todas sus entradas se pueden almacenar directamente hasta el almacén de control. En tanto estén presentes en el almacén de control en el flanco descendente del reloj cuando MIR se selecciona y lee, será suficiente. No hay necesidad de almacenarlas realmente en MPC. Por esta razón, MPC bien podría implementarse como un **registro virtual**, que no es más que un punto de reunión de señales, más parecido a un panel de conexiones electrónico que a un registro real. Hacer que MPC sea un registro virtual simplifica la temporización: ahora sólo ocurren sucesos en los flancos descendentes y ascendentes del reloj, y en ningún otro momento. Pero si para usted es más fácil pensar en MPC como un registro real, también es un punto de vista válido.

4.2 UN EJEMPLO DE ISA: IJVM

Continuemos nuestro ejemplo presentando el nivel ISA de la máquina para ser interpretada por el microprograma que se ejecuta en la microarquitectura de la figura 4-6 (IJVM). Por comodidad, algunas veces nos referiremos a la arquitectura del conjunto de instrucciones (ISA) como la **macroarquitectura**, a fin de contrastarla con la microarquitectura. Sin embargo, antes de describir IJVM haremos una breve digresión para motivar la descripción.

4.2.1 Pilas

Prácticamente todos los lenguajes de programación soportan el concepto de procedimientos (métodos) que tienen variables locales. Estas variables pueden accederse desde el interior del procedimiento pero dejan de ser accesibles una vez que el procedimiento ha regresado. Así, surge la pregunta: “¿en qué lugar de la memoria deben guardarse estas variables?”

La solución más sencilla, asignar a cada variable una dirección de memoria absoluta, no funciona. El problema es que un procedimiento puede llamarse a sí mismo. Estudiaremos estos procedimientos recursivos en el capítulo 5. Por el momento, basta decir que si un procedimiento se activa (es decir, se llama) dos veces, es imposible almacenar sus variables en posiciones de memoria absolutas porque la segunda invocación interferirá la primera.

En vez de ello, se usa una estrategia distinta. Se reserva un área de memoria, llamada **pila**, para variables, pero las variables individuales no reciben direcciones absolutas dentro de la pila. En vez de ello se ajusta un registro, digamos LV (*Local Variable*) de modo que apunte a la base de las variables locales del procedimiento en curso. En la figura 4-8(a) se ha invocado un procedimiento A, que tiene las variables locales *a1*, *a2* y *a3*, así que se ha reservado memoria para sus variables locales a partir de la posición a la que LV apunta. Otro registro, SP (*Stack Pointer*, apuntador de la pila), apunta a la palabra más alta de las variables locales de A. Si LV es 100 y las palabras son de 4 bytes, entonces SP será 108. Se hace referencia a las variables especificando a qué distancia están de LV. La estructura de datos entre LV y SP (que incluye las dos palabras a las que se apunta) es el **marco de variables locales** de A.

Consideremos ahora qué sucede si A llama a otro procedimiento, B. ¿Dónde deberán almacenarse las cuatro variables locales de B (*b1*, *b2*, *b3*, *b4*)? Respuesta: en la pila, encima de las de A, como se muestra en la figura 4-8(b). Observe que la llamada de procedimiento

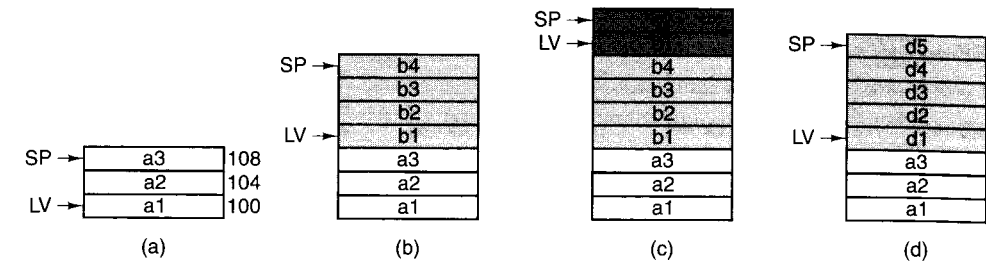


Figura 4-8. Uso de una pila para almacenar variables locales. (a) Mientras A está activo. (b) Después de que A llama a B. (c) Después de que B llama a C. (d) Después de que C y B regresan y A llama a D.

ajustó LV de modo que apunte a las variables locales de B en lugar de a las de A. Podemos hacer referencia a las variables locales de B dando su distancia respecto a LV. De igual manera, si B llama a C, LV y SP se ajustan nuevamente para asignar espacio a las dos variables locales de C, como se ilustra en la fig. 4-8(c).

Cuando C regresa, B se activa nuevamente, y la pila se vuelve a ajustar como en la figura 4-8(b) de modo que ahora LV apunte otra vez a las variables locales de B. De igual modo, cuando B regresa, volvemos a la situación de la figura 4-8(a). En todas las condiciones, LV apunta a la base del marco de pila del procedimiento que está activo, y SP apunta al tope del marco de pila.

Supongamos ahora que A llama a D, que tiene cinco variables locales. Llegamos a la situación de la figura 4-8(d), en la que las variables locales de D usan la misma memoria que usaron las de B, y también parte de las de C. Con esta organización de memoria, sólo se asigna espacio a los procedimientos que están activos. Cuando un procedimiento regresa, se libera la memoria que usaban sus variables locales.

Las pilas tienen otro uso, además de contener variables locales: pueden servir para retener operandos durante el cálculo de una expresión aritmética. Cuando se usa de este modo, la pila se llama **pila de operandos**. Suponga, por ejemplo, que antes de llamar a B, A tiene que efectuar el cálculo

$$a1 = a2 + a3;$$

Una forma de efectuar esta suma es meter *a2* en la pila, como se muestra en la figura 4-9(a). Aquí SP se incrementó tantos bytes como hay en una palabra, digamos 4, y el primer operando se almacenó en la dirección a la que ahora apunta SP. Luego se mete *a3* en la pila, como se muestra en la figura 4-9(b). Como acotación acerca de la notación, usaremos el tipo de letra **Helvética** para todos los fragmentos de programa como el anterior. También usaremos este tipo de letra para los códigos de operación de lenguaje ensamblador y los registros de la máquina, pero en el texto corrido las variables y procedimientos de los programas se darán en *cursivas*. La diferencia es que los nombres de variables y procedimientos son elegidos por el usuario; los códigos de operación y los nombres de registros están fijos.

Ahora puede efectuarse el cálculo ejecutando una instrucción que obtiene dos palabras de la pila, las suma y mete el resultado otra vez a la pila, como se muestra en la figura 4-9(c). Por último, saca la variable que está en la localidad más alta de la pila y se almacena en la variable local *a1*, como se ilustra en la figura 4-9(d).

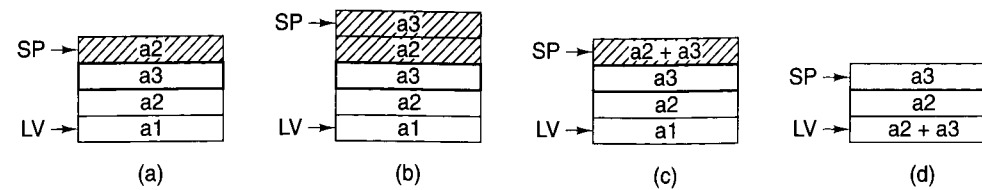


Figura 4-9. Uso de una pila de operandos para realizar un cálculo aritmético.

Los marcos de variables locales y las pilas de operandos se pueden entremezclar. Por ejemplo, al calcular una expresión como $x^2 + f(x)$, parte de la expresión (digamos x^2) podría estar en la pila de operandos cuando se invoca una función f . El resultado de la función se deja en la pila, encima de x^2 , y la siguiente instrucción los suma.

Vale la pena señalar que si bien todas las máquinas usan una pila para almacenar variables locales, no todas usan una pila de operandos como ésta para efectuar operaciones aritméticas. De hecho, pocas lo hacen, pero JVM e IJVM trabajan así, y es por ello que hemos presentado esta introducción a las operaciones de pila. Las estudiaremos con mayor detalle en el capítulo 5.

4.2.2 El modelo de memoria IJVM

Ya estamos listos para examinar la arquitectura de la IJVM. Básicamente, consiste en una memoria que se puede ver de dos maneras: como una formación de 4,294,967,296 bytes (4 GB) o como una formación de 1,073,741,824 palabras, cada una de 4 bytes. A diferencia de la mayor parte de las ISA, la Máquina Virtual Java no permite que el nivel ISA vea directamente las direcciones, pero hay varias direcciones implícitas que sirven de base para un apuntador. Las instrucciones de la IJVM sólo pueden acceder a la memoria sumando índices a tales apuntadores. En cualquier momento están definidas las siguientes áreas de memoria:

1. *La reserva de constantes.* Un programa IJVM no puede escribir en esta área que consta de constantes, cadenas y apuntadores a otras áreas de memoria a las que se hace referencia. Se carga cuando el programa se almacena en la memoria y no se modifica posteriormente. Existe un registro implícito, **CPP**, que contiene la dirección de la primera palabra de la reserva de constantes.
2. *El marco de variables locales.* Para cada invocación de un método, se asigna un área para almacenar variables durante la vigencia de la invocación. Esta área se llama **marco de variables locales**. Al principio de este marco residen los parámetros (también llamados argumentos) con que se invocó el método. El marco de variables locales no incluye la pila de operandos, que es aparte. Sin embargo, por razones de eficiencia, nuestra implementación coloca la pila de operandos inmediatamente arriba del marco de variables locales. Existe un registro implícito que contiene la dirección de la primera posición del marco de variables locales. Llamaremos a este registro **LV**. Los parámetros pasan en el momento de invocarse el método y se almacenan al principio del marco de variables locales.

3. *La pila de operandos.* Se garantiza que el marco de pila no excederá cierto tamaño, calculado con anticipación por el compilador de Java. El espacio de la pila de operandos se asigna directamente arriba del marco de variables locales, como se ilustra en la figura 4-10. En nuestra implementación es conveniente pensar en la pila de operandos como parte del marco de variables locales. En todo caso, existe un registro implícito que contiene la dirección de la palabra superior de la pila. Observe que, a diferencia de **CPP** y **LV**, este apuntador, **SP**, cambia durante la ejecución del método a medida que se meten operandos en la pila o se sacan de ella.
4. *El área de métodos.* Por último, hay una región de la memoria que contiene el programa y que se llama área de "texto" en el caso de un proceso UNIX. Existe un registro implícito que contiene la dirección de la instrucción que se traerá a continuación. Este apuntador es el Contador de Programa o **PC**. A diferencia de las demás regiones de memoria, el área de métodos se trata como una formación de bytes.

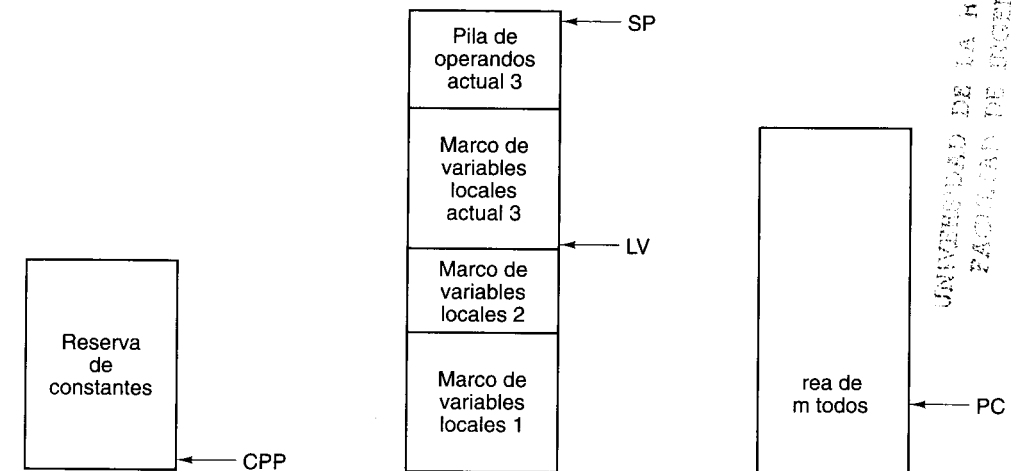


Figura 4-10. Las distintas partes de la memoria IJVM.

Debemos dejar en claro algo acerca de los apuntadores. Los registros **CPP**, **LV** y **SP** son apuntadores a *palabras*, no *bytes*, y las distancias se miden en palabras. Para el subconjunto entero que hemos escogido, todas las referencias a elementos de la reserva de constantes, el marco de variables locales y la pila son palabras, y todas las distancias empleadas como índices dentro de estos marcos se dan en palabras. Por ejemplo, **LV**, **LV + 1** y **LV + 2** se refieren a las primeras tres palabras del marco de variables locales. En contraste, **LV**, **LV + 4** y **LV + 8** se refieren a palabras situadas a intervalos de cuatro palabras (16 bytes).

En contraste, **PC** contiene una dirección de byte, y una resta o suma a **PC** altera la dirección en cierto número de bytes, no de palabras. El direccionamiento de **PC** es distinto

del de los demás registros, y esto se pone de manifiesto al haber un puerto de memoria especial en Mic-1 para PC. Recuerde que el puerto sólo tiene un byte de anchura. Incrementar PC en 1 e iniciar una lectura hace que se traiga el siguiente *byte*. Incrementar SP en 1 e iniciar una lectura hace que se traiga la siguiente *palabra*.

4.2.3 El conjunto de instrucciones IJVM

El conjunto de instrucciones de IJVM se muestra en la figura 4-11. Cada instrucción consiste en un código de operación y a veces un operando, como una distancia en la memoria o una constante. La primera columna da la codificación hexadecimal de la instrucción. La segunda da su representación mnemotécnica en lenguaje ensamblador. La tercera da una breve descripción de su efecto.

Hex	Mnemónico	Significado
0x10	BIPUSH <i>byte</i>	Almacenar byte en la pila
0x59	DUP	Copiar la palabra en el tope de la pila y almacenarla en la pila
0xA7	GOTO <i>distancia</i>	Ramificación incondicional
0x60	IADD	Sacar dos palabras de la pila; meter su suma
0x7E	IAND	Sacar dos palabras de la pila; meter su AND booleano
0x99	IFEQ <i>distancia</i>	Sacar palabra de la pila y saltar si es cero
0x9B	IFLT <i>distancia</i>	Sacar palabra de la pila y saltar si es menor que cero
0x9F	IF_ICMPEQ <i>distancia</i>	Sacar dos palabras de la pila; saltar si son iguales
0x84	IINC <i>númvar const</i>	Sumar una constante a una variable local
0x15	ILOAD <i>númvar</i>	Almacenar variable local en la pila
0xB6	INVOKEVIRTUAL <i>despl</i>	Invocar un método
0x80	IOR	Sacar dos palabras de la pila; almacenar su OR booleano
0xAC	IRETURN	Regresar de método con valor entero
0x36	ISTORE <i>númvar</i>	Sacar palabra de la pila y guardar en variable local
0x64	ISUB	Sacar dos palabras de la pila; almacenar su diferencia
0x13	LDC_W <i>índice</i>	Meter en la pila constante de la reserva de constantes
0x00	NOP	No hacer nada
0x57	POP	Quitar palabra del tope de la pila
0x5F	SWAP	Intercambiar las dos palabras en el tope de la pila
0xC4	WIDE	Prefijo de instrucción; la siguiente instrucción tiene un índice de 16 bits

Figura 4-11. El conjunto de instrucciones de IJVM. Los operandos *byte*, *const* y *númvar* son de 1 byte. Los operandos *despl*, *índice* y *distancia* son de 2 bytes.

Se proporcionan instrucciones para meter en la pila una palabra procedente de varios orígenes. Éstos incluyen la reserva de constantes (LDC_W), el marco de variables locales (ILOAD) y la instrucción misma (BIPUSH). También se puede sacar una variable de la pila

y guardarse en el marco de variables locales (ISTORE). Pueden efectuarse dos operaciones aritméticas (IADD e ISUB) así como dos operaciones lógicas (booleanas) (IAND y IOR) utilizando como operandos las dos palabras que están en el tope de la pila. En todas las operaciones aritméticas y lógicas se obtienen dos palabras de la pila y el resultado se almacena en la pila. Se proporcionan cuatro instrucciones de ramificación, una incondicional (GOTO) y tres condicionales (IFEQ, IFLT e IF_ICMPEQ). Todas las instrucciones de ramificación, si se siguen, ajustan el valor de PC según la distancia (de 16 bits con signo) que sigue al código de operación en la instrucción. Esta distancia se suma a la dirección del código de operación. También hay instrucciones IJVM para intercambiar las dos palabras que están en el tope de la pila (SWAP), duplicar la palabra que está en el tope (DUP) y sacarla (POP).

Algunas instrucciones tienen varios formatos, que incluyen una forma corta para las de uso común. Hemos incluido en IJVM dos de los diversos mecanismos que la JVM usa para implementar esto. En un caso hemos pasado por alto la forma corta en favor de la más general. En otro caso mostramos cómo se puede usar la instrucción prefijo WIDE para modificar la instrucción subsecuente.

Por último, hay una instrucción (INVOKEVIRTUAL) para invocar otro método, y otra instrucción (IRETURN) para salir de un método y devolver el control al método que lo invocó. Debido a la complejidad del mecanismo, hemos simplificado un poco la definición, y esto hizo posible crear un mecanismo sencillo para hacer una llamada y regresar. La restricción es que, a diferencia de Java, sólo permitimos a un método invocar un método que exista dentro de su propio objeto. Esta restricción obstaculiza severamente la orientación a objetos pero nos permite presentar un mecanismo mucho más sencillo, al evitar la necesidad de localizar el método dinámicamente. (Si no está familiarizado con la programación orientada a objetos, puede hacer caso omiso de este comentario sin problema. Lo que hemos hecho es convertir a Java en un lenguaje no orientado a objetos, como C o Pascal.) En todas las computadoras excepto JVM, la dirección de procedimiento al que se llamará la determina directamente la instrucción CALL, así que nuestro enfoque en realidad es el caso normal, no la excepción.

El mecanismo para invocar un método es el siguiente. Primero, el invocador mete en la pila una referencia (apuntador) al objeto que se llamará. (Esta referencia no se necesita en una IJVM porque no se puede especificar otro objeto, pero se conserva para mantener la consistencia con JVM.) En la figura 4-12(a) esta referencia se indica con OBJREF. Luego, el invocador mete los parámetros del método en la pila, en este ejemplo, *Parámetro 1*, *Parámetro 2* y *Parámetro 3*. Por último, se ejecuta INVOKEVIRTUAL.

La instrucción INVOKEVIRTUAL incluye un desplazamiento que indica la posición en la reserva de constantes que contiene la dirección, dentro del área de métodos, donde inicia el método que se está invocando. Sin embargo, si bien el código del método reside en la posición a la que apunta este apuntador, los primeros cuatro bytes en el área de métodos contienen datos especiales. Los primeros dos bytes se interpretan como un entero de 16 bits que indica el número de parámetros del método (los parámetros se almacenaron previamente en la pila). En lo referente a esta cifra, OBJREF cuenta como parámetro: el parámetro 0. Este entero de 16 bits, junto con el valor de SP, da la ubicación de OBJREF. Observe que LV

apunta a **OBJREF** y no al primer parámetro real. La decisión de a dónde apunta **LV** es un tanto arbitraria.

Los siguientes dos bytes del área de método se interpretan como otro entero de 16 bits que indica el tamaño del área de variables locales para el método que se está invocando. Esto es necesario porque se establecerá una nueva pila para el método, que comenzará inmediatamente arriba del marco de variables locales. Por último, el quinto byte del área de métodos contiene el primer código de operación a ejecutar.

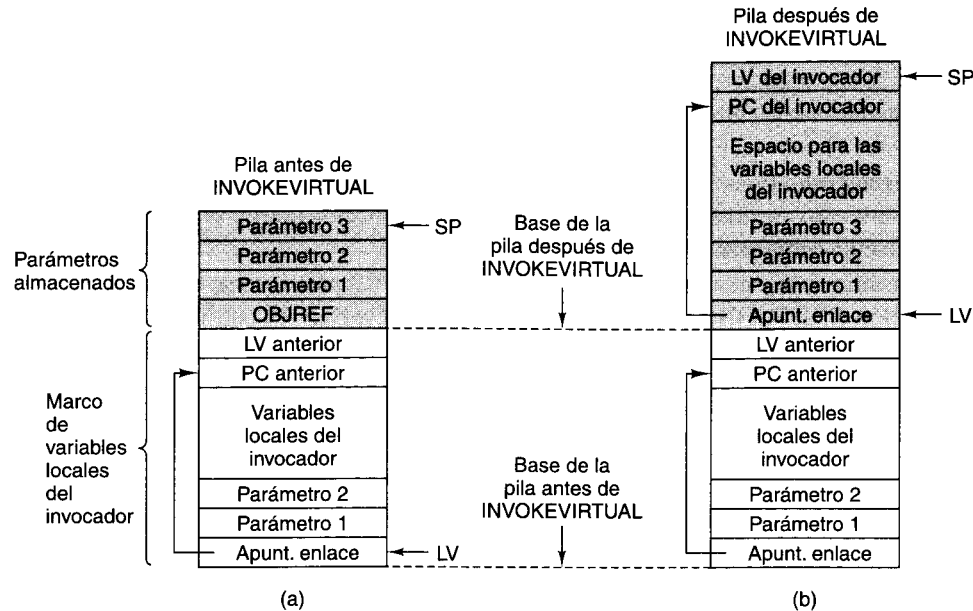


Figura 4-12. (a) Memoria antes de ejecutar **INVOKEVIRTUAL**. (b) Después de ejecutarlo.

La secuencia real que ocurre con **INVOKEVIRTUAL** es la que sigue y se muestra en la figura 4-12. Los dos bytes de índice sin signo que siguen al código de operación sirven para construir un índice para la tabla de la reserva de constantes (el primer byte es el byte de orden alto). La instrucción calcula la dirección base del nuevo marco de variables locales restando el número de parámetros al apuntador de pila y ajustando **LV** de modo que apunte a **OBJREF**. En este lugar, sobrescribiendo **OBJREF**, la implementación almacena la dirección de la posición donde se almacenará el **PC** antiguo. Esta dirección se calcula sumando el tamaño del marco de variables locales (parámetros + variables locales) a la dirección contenida en **LV**. Inmediatamente arriba de la dirección donde se guardará el **PC** viejo está la dirección donde se guardará el **LV** viejo. Inmediatamente arriba de esa dirección está el principio de la pila del procedimiento recién llamado. Se ajusta **SP** de modo que apunte al viejo **LV**, que es la dirección inmediatamente abajo de la primera posición vacía de la pila. Recuerde que **SP** siempre apunta a la palabra que está en el tope de la pila. Si la pila está vacía, **SP** apuntará a

la primera posición debajo del final de la pila porque nuestras pilas crecen hacia arriba, hacia direcciones más altas. En nuestras figuras las pilas siempre crecen hacia arriba, hacia la dirección más alta en la parte superior de la página.

La última operación que se requiere para ejecutar **INVOKEVIRTUAL** es hacer que **PC** apunte al quinto byte del espacio de código de métodos.

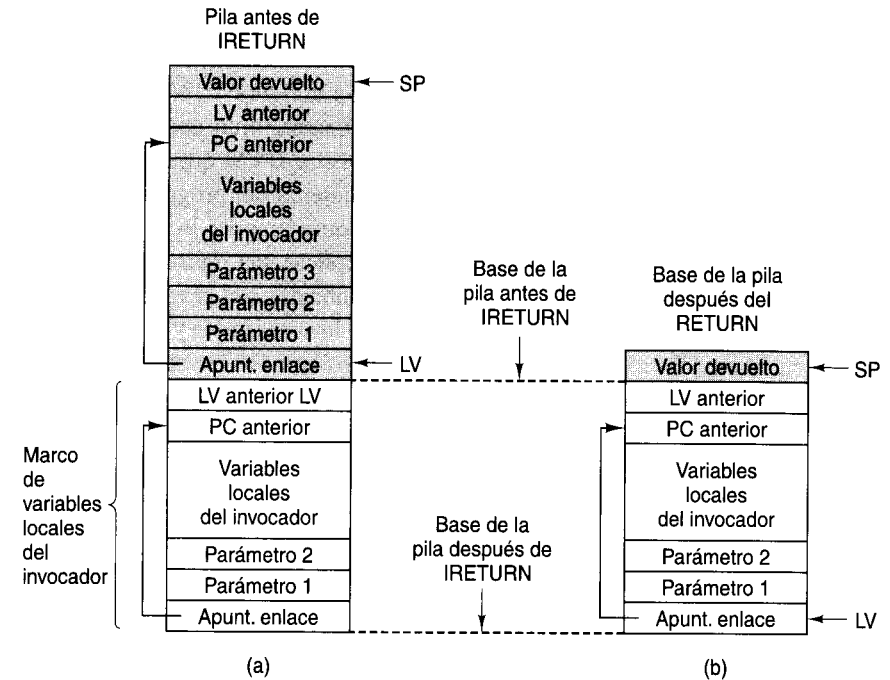


Figura 4-13. (a) Memoria antes de ejecutar **IRETURN**. (b) Después de ejecutarlo.

La instrucción **IRETURN** invierte las operaciones de la instrucción **INVOKEVIRTUAL**, como se muestra en la figura 4-13. Libera el espacio utilizado por el método que regresa y también restaura la pila a su estado anterior, excepto que (1) la palabra **OBJREF** (que ya se sobrescribió) y todos los parámetros se sacaron de la pila, y (2) el valor devuelto se colocó en el tope de la pila, en la posición que antes ocupaba **OBJREF**. Para restaurar el estado antiguo, la instrucción **IRETURN** debe restaurar los valores antiguos de **PC** y **LV**. Esto lo hace accediendo al apuntador de enlace (que es la palabra identificada por el apuntador **LV** actual). Recuerde que en esta posición, donde originalmente estaba almacenado **OBJREF**, la instrucción **INVOKEVIRTUAL** guardó la dirección que contiene el antiguo **PC**. Esta palabra y la que está arriba de ella se recuperan para restaurar **PC** y **LV**, respectivamente, a sus antiguos valores. El valor devuelto, que se guarda en el tope de la pila del método que termina, se copia en la posición en que se había guardado originalmente **OBJREF**, y **SP** se restaura de modo que apunte a esta posición. Así, el control se devuelve a la instrucción que sigue inmediatamente a la instrucción **INVOKEVIRTUAL**.

Hasta aquí, nuestra máquina no tiene instrucciones de entrada/salida. Y no las vamos a agregar, pues las necesita tanto como las necesita la Máquina Virtual Java, en cuya especificación oficial ni siquiera se menciona E/S. La teoría es que una máquina que no efectúa entrada ni salida es “segura”. (La lectura y escritura se efectúan en JVM mediante llamadas a métodos especiales, que realizan la E/S.)

4.2.4 Compilación de Java a IJVM

Veamos ahora la relación que hay entre Java y la IJVM. En la figura 4-14(a) mostramos un sencillo fragmento de código en Java. Cuando éste se alimenta a un compilador Java, el compilador probablemente producirá el lenguaje ensamblador IJVM que se muestra en la figura 4-14(b). Los números de línea del 1 al 15 a la izquierda del programa en lenguaje ensamblador no forman parte de la salida del compilador, como tampoco lo hacen los comentarios (que comienzan con //); están ahí para ayudar a explicar una figura subsecuente. El ensamblador de Java traduciría entonces el programa ensamblado en el programa binario que se muestra en la figura 4-14(c). (En realidad, el compilador Java ensambla y genera el programa binario directamente.) Para este ejemplo hemos supuesto que *i* es la variable local 1, *j* es la variable local 2 y *k* es la variable local 3.

<i>i</i> = <i>j</i> + <i>k</i> ;	1	ILOAD <i>j</i>	// <i>i</i> = <i>j</i> + <i>k</i>	0x15 0x02
if (<i>i</i> == 3)	2	ILOAD <i>k</i>		0x15 0x03
<i>k</i> = 0;	3	IADD		0x60
else	4	ISTORE <i>i</i>		0x36 0x01
<i>j</i> = <i>j</i> - 1;	5	ILOAD <i>i</i>	// if (<i>i</i> < 3)	0x15 0x01
	6	BIPUSH 3		0x10 0x03
	7	IF_ICMPEQ L1		0x9F 0x00 0x0D
	8	ILOAD <i>j</i>	// <i>j</i> = <i>j</i> - 1	0x15 0x02
	9	BIPUSH 1		0x10 0x01
	10	ISUB		0x64
	11	ISTORE <i>j</i>		0x36 0x02
	12	GOTO L2		0xA7 0x00 0x07
13 L1:	13	BIPUSH 0	// <i>k</i> = 0	0x10 0x00
14	14	ISTORE <i>k</i>		0x36 0x03
15 L2:	15			

(a)

(b)

(c)

Figura 4-14. (a) Fragmento en Java. (b) El lenguaje de ensamblador Java correspondiente. (c) El programa IJVM en hexadecimal.

El código compilado es sencillo. Primero *j* y *k* se meten en la pila, se suman y el resultado se guarda en *i*. Luego *i* y la constante 3 se meten en la pila y se comparan. Si son iguales, se sigue una ramificación a L1, donde *k* se pone en 0. Si son distintas, la comparación falla y se ejecuta el código que sigue a IF_ICMPEQ. Cuando el programa termina de hacer esto, salta a L2, donde se fusionan las partes then y else.

La pila de operandos para el programa IJVM de la figura 4-14(b) se muestra en la figura 4-15. Antes de que el código comience a ejecutarse, la pila está vacía, lo que se indica con la línea horizontal arriba del 0. Después del primer ILOAD, *j* está en la pila, como indica la *j* en un rectángulo arriba del 1 (que indica que ya se ejecutó la instrucción 1). Después del segundo ILOAD hay dos palabras en la pila, como se muestra arriba del 2. Después del IADD sólo queda una palabra en la pila, y contiene la suma *j* + *k*. Cuando se saca la palabra del tope de la pila y se guarda en *i*, la pila queda vacía, como se muestra arriba del 4.

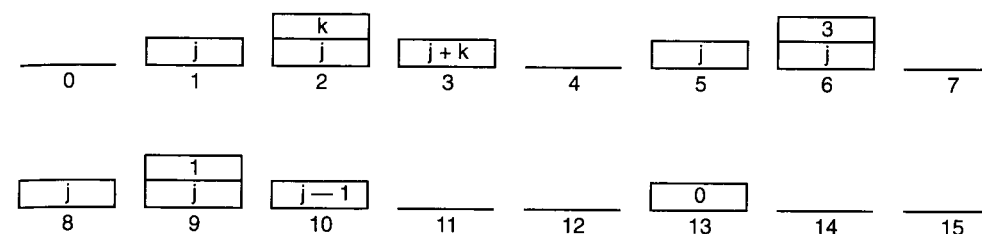


Figura 4-15. La pila después de cada instrucción de la figura 4-14(b).

La instrucción 5 (ILOAD) inicia el enunciado if metiendo *i* en la pila (en 5). A continuación viene la constante 3 (en 6). Después de la comparación, la pila queda vacía otra vez (7). La instrucción 8 es el principio de la parte else del fragmento de programa Java. La parte else continúa hasta la instrucción 12, donde se salta la parte then y llega al rótulo L2.

4.3 UN EJEMPLO DE IMPLEMENTACIÓN

Ahora que hemos especificado tanto la microarquitectura como la macroarquitectura con detalle, el aspecto que nos falta es la implementación. En otras palabras, ¿qué aspecto tiene un programa que se ejecuta en la primera y que interpreta la segunda, y cómo funciona? Antes de poder contestar estas preguntas, debemos considerar con detenimiento la notación que usaremos para describir la implementación.

4.3.1 Microinstrucciones y notación

En principio, podríamos describir el almacén de control en binario, 36 bits por palabra. Sin embargo, al igual que en los lenguajes de programación convencionales, es muy provechoso introducir una notación que comunique la esencia de los aspectos que necesitamos tratar y al mismo tiempo oculte los detalles que carecen de importancia o que se manejan mejor automáticamente. Es importante darse cuenta aquí de que el lenguaje que hemos escogido pretende ilustrar los conceptos más que facilitar diseños eficientes. Si esto último fuera nuestra meta, usaríamos una notación muy diferente para maximizar la flexibilidad de la que dispone el diseñador. Un aspecto en el que esta cuestión es importante es cómo escoger las

direcciones. Puesto que la memoria no está ordenada lógicamente, no existe una “siguiente instrucción” natural que esté implícita cuando especificamos una secuencia de operaciones. Gran parte de la potencialidad de esta organización de control proviene de la capacidad del diseñador (o del ensamblador) para seleccionar direcciones de forma eficiente. Por tanto, comenzaremos por introducir un lenguaje simbólico sencillo que describe cabalmente cada operación sin explicar plenamente cómo podrían haberse determinado todas las direcciones.

Nuestra notación especifica todas las actividades que ocurren en un solo ciclo de reloj en una sola línea. En teoría, podríamos usar un lenguaje de alto nivel para describir las operaciones. Sin embargo, el control ciclo por ciclo es muy importante porque ofrece la oportunidad de realizar varias operaciones de forma concurrente y es necesario poder analizar cada ciclo para entender y verificar las operaciones. Si la meta es una implementación rápida y eficiente (si los demás factores son iguales, rápido y eficiente siempre es mejor que lento e ineficiente), entonces cada ciclo cuenta. En una implementación real, el programa oculta muchos trucos sutiles, y se emplean secuencias u operaciones poco claras con objeto de ahorrar un solo ciclo. La recompensa por ahorrar ciclos es atractiva: si podemos reducir en dos ciclos una instrucción que tarda cuatro ciclos, se ejecutará en la mitad del tiempo, y esta aceleración ocurre cada vez que ejecutamos la instrucción.

Una posible estrategia sería simplemente hacer una lista de las señales que deben activarse en cada ciclo del reloj. Supongamos que en un ciclo queremos incrementar el valor de SP. También queremos iniciar una operación de lectura y queremos que la siguiente instrucción sea la que reside en la posición 122 del almacén de control. Podríamos escribir

`ReadRegister = SP, ALU = INC, WSP, Read, NEXT_ADDRESS = 122`

donde WSP significa “escribir el registro SP”. Esta notación es completa, pero difícil de entender. En vez de ello, combinaremos las operaciones de una forma natural e intuitiva para capturar el efecto de lo que está sucediendo:

`SP = SP + 1; rd`

Llamemos a nuestro microlenguaje ensamblador “MAL” (*Micro Assembly Language*) (que es como usted se va a sentir si escribe demasiado código en él). MAL está hecho a propósito para que refleje las características de la microarquitectura. Durante cada ciclo se puede escribir cualquiera de los registros, aunque normalmente sólo se escribe uno. Sólo se puede colocar el contenido de un registro en el lado B de la ALU. En el lado A las opciones son +1, 0, -1 y el registro H. Así, podemos usar un sencillo enunciado de asignación, como en Java, para indicar la operación a ejecutar. Por ejemplo, si queremos copiar algo de SP a MDR, podemos decir

`MDR = SP`

Si queremos indicar el uso de las funciones de la ALU aparte de dejar pasar simplemente el contenido del bus B, podemos escribir, por ejemplo,

`MDR = H + SP`

que suma el contenido del registro H a SP y escribe el resultado en MDR. El operador + es conmutativo (lo que implica que el orden de los operandos no importa), así que el enunciado anterior también puede escribirse como

`MDR = SP + H`

y generar la misma microinstrucción de 36 bits, aunque en términos estrictos H deba ser el operando izquierdo de la ALU.

Debemos tener cuidado de usar sólo operaciones permitidas. Las operaciones permitidas más importantes se muestran en la figura 4-16, donde SOURCE (origen) puede ser cualquiera de MDR, PC, MBR, MBRU, SP, LV, CPP, TOS u OPC (MBRU se refiere a la versión sin signo de MBR). Todos estos registros pueden actuar como fuentes de la ALU en el bus B. De igual manera, DEST puede ser MAR, MDR, PC, SP, LV, CPP, TOS, OPC o H, los cuales son posibles destinos de la salida de la ALU en el bus C. Este formato es engañoso porque muchos enunciados aparentemente razonables no están permitidos. Por ejemplo,

`MDR = SP + MDR`

parece perfectamente razonable, pero no es posible ejecutarlo en la trayectoria de datos de la figura 4-6 en un solo ciclo. Esta restricción se debe a que en una suma (fuera del incremento o decremento) uno de los operandos debe ser el registro H. De igual modo,

`H = H - MDR`

podría ser útil, pero también es imposible porque el único origen posible de un sustraendo (el valor que se resta) es el registro H. Corresponde al ensamblador rechazar enunciados que parezcan válidos pero que en realidad están prohibidos.

Extenderemos la notación para permitir múltiples asignaciones empleando varios signos de igual. Por ejemplo, si queremos incrementar SP y guardar el nuevo valor en SP y también en MDR, podemos escribir

`SP = MDR = SP + 1`

Para indicar lecturas y escrituras de palabras de datos de 4 bytes en la memoria simplemente incluiremos rd (leer) y wr (escribir) en la microinstrucción. La obtención de un byte a través del puerto de un byte se indica con fetch (buscar). Pueden ocurrir asignaciones y operaciones de memoria en el mismo ciclo. Esto se indica escribiéndolas en la misma línea.

Para evitar confusiones, repitiremos que Mic-1 tiene dos formas de acceder a la memoria. Las lecturas y escrituras de palabras de datos de 4 bytes usan MAR/MDR y se indican en las microinstrucciones con rd y wr, respectivamente. Las lecturas de códigos de operación de un byte del flujo de instrucciones usan PC/MBR y se indican con fetch en las microinstrucciones. Ambos tipos de operaciones de memoria pueden efectuarse simultáneamente.

Sin embargo, el mismo registro no puede recibir un valor de la memoria y de la trayectoria de datos en el mismo ciclo. Considere el código

`MAR = SP; rd`

`MDR = H`

DEST = H
DEST = SOURCE
DEST = $\overline{H}$
DEST = $\overline{SOURCE}$
DEST = H + SOURCE
DEST = H + SOURCE + 1
DEST = H + 1
DEST = SOURCE + 1
DEST = SOURCE - H
DEST = SOURCE - 1
DEST = - H
DEST = H AND SOURCE
DEST = H OR SOURCE
DEST = 0
DEST = 1
DEST = - 1

Figura 4-16. Todas las operaciones permitidas. Cualquiera de estas operaciones se puede extender añadiéndole "<<8" para desplazar el resultado un byte a la izquierda. Por ejemplo, una operación común es $H = MBR << 8$.

El efecto de la primera microinstrucción es asignar a MDR un valor de la memoria al final de la segunda microinstrucción. Sin embargo, la segunda microinstrucción también asigna un valor a MDR al mismo tiempo. Estas dos asignaciones están en conflicto y no se permiten porque el resultado no está definido.

Recuerde que cada microinstrucción debe proporcionar explícitamente la dirección de la siguiente microinstrucción que se ejecutará. No obstante, a menudo ocurre que sólo hay una microinstrucción que puede invocar cierta microinstrucción: la que está en la línea inmediatamente anterior a ésta. Para facilitar la tarea del microprogramador, el microensamblador normalmente asigna una dirección a cada microinstrucción (no necesariamente consecutivas en el almacén de control) y llena el campo NEXT_ADDRESS de modo que microinstrucciones escritas en líneas consecutivas se ejecuten una tras otra.

Sin embargo, hay ocasiones en que el microprogramador necesita efectuar un salto, sea incondicional o condicional. La notación para los saltos incondicionales es fácil:

goto etiqueta

y se puede incluir en cualquier microinstrucción para nombrar explícitamente a su sucesora. Por ejemplo, casi todas las secuencias de microinstrucciones terminan con un regreso a la primera instrucción del ciclo principal, de modo que la última instrucción de una secuencia semejante por lo regular incluye

goto Main1

Cabe señalar que la trayectoria de datos está disponible para operaciones normales aun durante una microinstrucción que contiene un goto. Después de todo, cada microinstrucción

contiene un campo NEXT_ADDRESS. Lo único que goto hace es ordenar al microensamblador que coloque ahí un valor específico en lugar de la dirección donde el microensamblador decidió colocar la microinstrucción de la siguiente línea. En principio, toda línea debería tener un enunciado goto sólo cuando es útil para el microprogramador; si la dirección objetivo es la siguiente línea, puede omitirse.

Necesitamos una notación distinta para los saltos condicionales. Recuerde que JAMN y JAMZ usan los bits N y Z. A veces es necesario probar un registro para ver si es cero, por ejemplo. Una forma de hacerlo sería pasar el contenido del registro por la ALU y almacenarlo de vuelta en el mismo registro. El enunciado

TOS = TOS

parece extraño, aunque hace lo que buscamos (establecer el flip-flop Z con base en TOS). Sin embargo, con objeto de que los microprogramas sean más elegantes, extenderemos MAL añadiendo dos registros imaginarios, N y Z a los cuales podemos hacer asignaciones. Por ejemplo,

Z = TOS

pasa el contenido de TOS por la ALU y establece los valores de los flip-flops Z y N, pero no almacena un valor en un registro. Lo que hacemos realmente al usar Z o N como destino es decirle al microensamblador que ponga en 0 todos los bits del campo C de la figura 4-5. La trayectoria de datos ejecuta un ciclo normal, con todas las operaciones normales permitidas, pero no se escribe en ningún registro. Observe que no importa si el destino es N o Z; la microinstrucción generada por el microensamblador es idéntica. A los programadores que escojan intencionalmente el destino "incorrecto" se les debería obligar a trabajar con una IBM PC original de 4.77 MHz durante una semana como castigo.

La sintaxis para indicarle al microensamblador que establezca el bit JAMZ es

si (Z) goto L1; else goto L2

Puesto que el hardware exige que estas dos direcciones sean idénticas en sus 8 bits de orden bajo, corresponde al microensamblador asignarles tales direcciones. Por otra parte, dado que L2 puede estar en cualquier lugar dentro de las 256 primeras palabras del almacén de control, el microensamblador tiene total libertad para encontrar un par disponible.

Normalmente, estos dos enunciados se combinarán; por ejemplo,

Z = TOS; if (Z) goto L1; else goto L2

El efecto de este enunciado es que MAL genera una microinstrucción en la que TOS se hace pasar por la ALU (pero no se almacena en ningún lado) para que su valor establezca el bit Z. Poco después de que Z se ha cargado con el valor del bit de condición de la ALU, se hace OR con el bit de orden alto de MPC, lo que obliga a traer la dirección de la siguiente microinstrucción de L2 o bien de L1 (que debe ser exactamente 256 más que L2). MPC estará estable y listo para usarse en la obtención de la siguiente microinstrucción.

Por último, necesitamos una notación para usar el bit JMPC. La que usaremos es **goto (MBR OR valor)**

Esta sintaxis dice al microensamblador que use *valor* para **NEXT_ADDRESS** y ajuste el bit **JMPC** de modo que el resultado de **MBR OR NEXT_ADDRESS** se coloque en **MPC**. Si *valor* es 0, como sucede normalmente, basta con escribir

goto (MBR)

Observe que sólo los 8 bits de orden bajo de **MBR** están conectados permanentemente con **MPC** (vea la figura 4-6), por lo que en este caso no surge la cuestión de la extensión de signo (es decir, **MBR** versus **MBRU**). Observe también que el **MBR** que está disponible al final del ciclo actual es el mismo que se usó. Una obtención iniciada en *esta* microinstrucción llega demasiado tarde para afectar la selección de la siguiente microinstrucción.

4.3.2 Implementación de IJVM utilizando el Mic-1

Por fin hemos llegado a un punto en el que podemos armar todas las piezas. La figura 4-17 es el microprograma que se ejecuta en Mic-1 e interpreta IJVM. Es un programa sorprendentemente corto: 112 microinstrucciones en total. Se dan tres columnas para cada microinstrucción: el rótulo simbólico, el microcódigo real y un comentario. Cabe señalar que microinstrucciones consecutivas no necesariamente se encuentran en direcciones consecutivas en el almacén de control, como ya mencionamos.

A estas alturas los nombres que escogimos para casi todos los registros de la figura 4-1 deberán tener un significado obvio: **CPP** (*Constant Pool Pointer*), **LV** (*Local Variables*) y **SP** (*Stack Pointer*) sirven para almacenar los apuntadores a la reserva de constantes, a las variables locales y al tope de la pila, respectivamente, mientras que **PC** (*Program Counter*) contiene la dirección del siguiente byte que se obtendrá del flujo de instrucciones. **MBR** es un registro de un byte que contiene los bytes del flujo de instrucciones que llegan de la memoria para interpretarse. **TOS** (*Top Of Stack*) y **OPC** (*OPCode*) son registros extra. Su uso se describe a continuación.

En ciertos momentos, se garantiza que cada uno de estos registros contiene cierto valor, pero todos pueden usarse como registros temporales si es necesario. Al principio y al final de cada instrucción, **TOS** contiene el valor que está en la posición de memoria a la que **SP** apunta, la palabra que está en el tope de la pila. Este valor es redundante porque siempre se puede leer de la memoria, pero tenerlo en un registro a menudo ahorra una referencia a la memoria. En el caso de unas cuantas instrucciones, mantener **TOS** implica *más* operaciones de memoria. Por ejemplo, la instrucción **POP** desecha la palabra que está en el tope de la pila, y por tanto debe traer de la memoria la nueva palabra que ahora está en el tope y colocarla en **TOS**.

El registro **OPC** es un registro temporal; no tiene un uso preasignado. Se usa, por ejemplo, para guardar la dirección del código de operación de una operación de ramificación mientras **PC** se incrementa para acceder a parámetros. **OPC** también se usa como registro temporal en las instrucciones de ramificación condicional de IJVM.

Al igual que todos los intérpretes, el microprograma de la figura 4-17 tiene un ciclo principal que busca, decodifica y ejecuta instrucciones del programa que se está interpretando, en este caso, instrucciones IJVM. Su ciclo principal comienza en la línea rotulada **Main1**, con la invariante de que **PC** ya se cargó previamente con la dirección de una posición de memoria que contiene un código de operación. Además, ese código de operación ya se colocó en **MBR**. Cabe señalar que esto implica que cuando regresemos a este punto deberemos

asegurarnos de que **PC** se haya actualizado de modo que apunte al siguiente código de operación a interpretar y que el byte del código de operación mismo ya se haya traído y se haya colocado en **MBR**.

Esta sucesión inicial de instrucciones se ejecuta al principio de cada instrucción, por lo que es importante que sea lo más corta posible. Gracias a un diseño muy cuidadoso del hardware y software del Mic-1 hemos logrado reducir el ciclo principal a una sola microinstrucción. Una vez que la máquina ha iniciado, cada vez que se ejecuta esta microinstrucción el código de operación IJVM que se va a ejecutar ya está presente en **MBR**. Lo que la microinstrucción hace es ramificar al microcódigo que ejecuta esta instrucción IJVM y también comenzar a traer el byte que sigue al código de operación, que podría ser un byte de operando o el siguiente código de operación.

Ahora podemos revelar la verdadera razón por la que cada microinstrucción nombra explícitamente a su sucesora, en lugar de dejar que se ejecuten en sucesión. Todas las direcciones del almacén de control que corresponden a códigos de operación deben reservarse para la primera palabra del intérprete de la instrucción correspondiente. Así, en la figura 4-11 vemos que el código que interpreta **POP** inicia en 0x57 y el código que interpreta **DUP** inicia en 0x59. (Cómo **MAL** supo que debía poner **POP** en 0x57 es uno de los misterios del universo; cabe suponer que en algún lugar hay un archivo que lo especifica.)

Lo malo es que el código de **POP** tiene una longitud de tres microinstrucciones, así que si lo colocáramos en palabras consecutivas interferiría el principio de **DUP**. Puesto que todas las direcciones del almacén de control que corresponden a códigos de operación están reservadas de hecho, las microinstrucciones que siguen a la primera de cada secuencia deben colocarse en los huecos que hay entre las direcciones reservadas. Por esta razón hay una ramificación constante, y tener una microrramificación explícita (una microinstrucción que ramifica) cada cierto número de instrucción para saltar de un hueco a otro implicaría un gran desperdicio.

Para ver cómo funciona el intérprete, supongamos, por ejemplo, que **MBR** contiene el valor 0x60, es decir, el código de operación de **IADD** (vea la figura 4-11). En el ciclo principal, que abarca una sola microinstrucción, realizamos tres cosas:

1. Incrementamos **PC**, de modo que ahora contiene la dirección del primer byte después del código de operación.
2. Iniciamos la búsqueda del siguiente byte para colocarlo en **MBR**. Este byte se necesitará tarde o temprano, sea como operando para la instrucción IJVM en curso o como siguiente código de operación (como en el caso de la instrucción **IADD**, que no tiene bytes de operandos).
3. Realizar una ramificación multivías a la dirección contenida en **MBR** al principio de **Main1**. Esta dirección es igual al valor numérico del código de operación que se está ejecutando. La microinstrucción anterior la colocó ahí. Tome nota de que el valor que se está obteniendo en esta microinstrucción no desempeña ningún papel en la ramificación multivías.

La obtención del siguiente byte se inicia aquí para que esté disponible cuando inicie la tercera microinstrucción. Podría necesitarse entonces o no, pero se usará tarde o temprano, por lo que iniciar la obtención ahora en nada podría perjudicar.

Mnemónico	Operaciones	Comentarios
Main1	PC = PC + 1; fetch; goto (MBR)	MBR contiene código de operación; obtener siguiente byte; despachar
nop1	goto Main1	No hacer nada
iadd1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
iadd2	H = TOS	H = tope de la pila
iadd3	MDR = TOS = MDR + H; wr; goto Main1	Sumar dos palabras del tope; escribir en tope de pila
isub1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
isub2	H = TOS	H = tope de la pila
isub3	MDR = TOS = MDR - H; wr; goto Main1	Restar; escribir en tope de pila
iand1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
iand2	H = TOS	H = tope de la pila
iand3	MDR = TOS = MDR AND H; wr; goto Main1	Hacer AND; escribir en tope de pila
ior1	MAR = SP = SP + 1; rd	Leer 2a. palabra de la pila
ior2	H = TOS	H = tope de la pila
ior3	MDR = TOS = MDR OR H; wr; goto Main1	Hacer OR; escribir en tope de pila
dup1	MAR = SP = SP + 1	Incrementar SP y copiar en MAR
dup2	MDR = TOS; wr; goto Main1	Escribir nueva palabra de pila
pop1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
pop2		Esperar que se lea nuevo TOS de la memoria
pop3	TOS = MDR; goto Main1	Copiar nueva palabra en TOS
swap1	MAR = SP - 1; rd	Asignar SP-1 a MAR; leer 2a. palabra de pila
swap2	MAR = SP	Asignar palabra en tope a MAR
swap3	H = MDR; wr	Guardar TOS en H; escribir 2a. pal. en tope de pila
swap4	MDR = TOS	Copiar TOS viejo en MDR
swap5	MAR = SP - 1; wr	Asignar SP-1 a MAR; escribir 2a. pal. en pila
swap6	TOS = H; goto Main1	Actualizar TOS
bipush1	SP = MAR = SP + 1	MBR = byte que se meterá en la pila
bipush2	PC = PC + 1; fetch	Incrementar PC, traer siguiente cód. op.
bipush3	MDR = TOS = MDR; wr; goto Main1	Extender signo de constante y meter en pila
iload1	H = LV	MBR contiene índice; copiar LV en H
iload2	MAR = MBRU + H; rd	MAR = dir. de variable local que se meterá
iload3	MAR = SP = SP + 1	SP apunta a nuevo tope de pila; preparar escrit.
iload4	PC = PC + 1; fetch; wr	Inc. PC; obt. sig. cód. op.; escribir en tope de pila
iload5	TOS = MDR; goto Main1	Actualizar TOS
istore1	H = LV	MBR contiene índice; copiar LV en H
istore2	MAR = MBRU + H	MAR = dir. de var. local en la que se guardará
istore3	MDR = TOS; wr	Copiar TOS en MDR; escribir palabra
istore4	SP = MAR = SP - 1; rd	Leer 2a. palabra de la pila
istore5	PC = PC + 1; fetch	Incrementar PC; traer siguiente cód. op.
istore6	TOS = MDR; goto Main1	Actualizar TOS
wide1	PC = PC + 1; fetch; goto (MBR OR 0x100)	Ramif. multivías con bit alto encendido
wide_iload1	PC = PC + 1; fetch	MBR contiene 1er. byte índice; traer 2o.
wide_iload2	H = MBRU << 8	H = 1er. byte índice desplazado 8 bits a la izq.
wide_iload3	H = MBRU OR H	H = índice de 16 bits de variable local
wide_iload4	MAR = LV + H; rd; goto iload3	MAR = dir. de variable local que se meterá
wide_istore1	PC = PC + 1; fetch	MBR contiene 1er. byte índice; traer 2o.
wide_istore2	H = MBRU << 8	H = 1er. byte índice desplazado 8 bits a la izq.
wide_istore3	H = MBRU OR H	H = índice de 16 bits de variable local
wide_istore4	MAR = LV + H; goto istore3	MAR = dir. de variable local en que se guardará
idc_w1	PC = PC + 1; fetch	MBR contiene 1er. byte índice; traer 2o.
idc_w2	H = MBRU << 8	H = 1er. byte de índice << 8
idc_w3	H = MBRU OR H	H = índice de 16 bits en reserva de constantes
idc_w4	MAR = H + CPP; rd; goto iload3	MAR = dir. de la constante en la reserva

Figura 4-17. El microprograma del Mic-1 (parte 1 de 3).

Mnemónico	Operaciones	Comentarios
iinc1	H = LV	MBR contiene índice; copiar LV en H
iinc2	MAR = MBRU + H; rd	Copiar LV + índice en MAR; leer variable
iinc3	PC = PC + 1; fetch	Obtener constante
iinc4	H = MDR	Copiar variable en H
iinc5	PC = PC + 1; fetch	Obtener siguiente código de operación
iinc6	MDR = MBR + H; wr; goto Main1	Almacenar suma en MDR; actualizar variable
goto1	OPC = PC - 1	Guardar dirección de código de op.
goto2	PC = PC + 1; fetch	MBR = 1er. byte de distancia; traer 2o.
goto3	H = MBR << 8	Desplazar y guardar 1er. byte c/signo en H
goto4	H = MBRU OR H	H = distancia de salto de 16 bits
goto5	PC = OPC + H; fetch	Sumar distancia a OPC
goto6	goto Main1	Esperar que traigan siguiente cód. op.
ifft1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
ifft2	OPC = TOS	Guardar TOS en OPC temporalmente
ifft3	TOS = MDR	Poner nuevo tope de pila en TOS
ifft4	N = OPC; if (N) goto T; else goto F	Ramificar según bit N
ifeq1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
ifeq2	OPC = TOS	Guardar TOS en OPC temporalmente
ifeq3	TOS = MDR	Poner nuevo tope de pila en TOS
ifeq4	Z = OPC; if (Z) goto T; else goto F	Ramificar según bit Z
if_jcmpeq1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
if_jcmpeq2	MAR = SP = SP - 1	Ajustar MAR para leer nuevo tope de pila
if_jcmpeq3	H = MDR; rd	Copiar 2a. palabra de pila en H
if_jcmpeq4	OPC = TOS	Guardar TOS en OPC temporalmente
if_jcmpeq5	TOS = MDR	Poner nuevo tope de pila en TOS
if_jcmpeq6	Z = OPC - H; if (Z) goto T; else goto F	Si 2 pal. en tope iguales, ir a T; si no, a F
T	OPC = PC - 1; fetch; goto goto2	Mismo que goto1; necesario p/dir. objetivo
F	PC = PC + 1	Saltar primer byte de distancia
F2	PC = PC + 1; fetch	PC ahora apunta a siguiente cód. op.
F3	goto Main1	Esperar que traigan código de operación
invokevirtual1	PC = PC + 1; fetch	MBR = byte índice 1; inc. PC, obt. 2o. byte
invokevirtual2	H = MBRU << 8	Desplazar y guardar primer byte en H
invokevirtual3	H = MBRU OR H	H = dist. de apunt. a método, a CPP
invokevirtual4	MAR = CPP + H; rd	Obt. apunt. al método del área de CPP
invokevirtual5	OPC = PC + 1	Guardar temporalmente PC de retorno en OPC
invokevirtual6	PC = MDR; fetch	PC apunta a nuevo método; obt. cuenta de parám.
invokevirtual7	PC = PC + 1; fetch	Traer 2o. byte de cuenta de parámetros
invokevirtual8	H = MBRU << 8	Desplazar y guardar primer byte en H
invokevirtual9	H = MBRU OR H	H = número de parámetros
invokevirtual10	PC = PC + 1; fetch	Traer primer byte de # locales
invokevirtual11	TOS = SP - H	TOS = dirección de OBJREF - 1
invokevirtual12	TOS = MAR = TOS + 1	TOS = dirección de OBJREF (nuevo LV)
invokevirtual13	PC = PC + 1; fetch	Traer 2o. byte de # locales
invokevirtual14	H = MBRU << 8	Desplazar y guardar primer byte en H
invokevirtual15	H = MBRU OR H	H = # locales
invokevirtual16	MDR = SP + H + 1; wr	Sobreescribir OBJREF con apunt. enlace
invokevirtual17	MAR = SP = MDR;	Asignar a SP, MAR, pos. p/guardar viejo PC
invokevirtual18	MDR = OPC; wr	Guardar viejo PC arriba de variables locales
invokevirtual19	MAR = SP = SP + 1	SP apunta a pos. p/guardar viejo LV
invokevirtual20	MDR = LV; wr	Guardar viejo LV arriba de PC guardado
invokevirtual21	PC = PC + 1; fetch	Traer 1er. cód. op. del nuevo método
invokevirtual22	LV = TOS; goto Main1	Hacer que LV apunte a marco de var. loc.

Figura 4-17. El microprograma del Mic-1 (parte 2 de 3).

Mnemónico	Operaciones	Comentarios
ireturn1	MAR = SP = LV; rd	Restab. SP, MAR, p/obtener apunt. enlace
ireturn2		Esperar lectura
ireturn3	LV = MAR = MDR; rd	Asignar apunt. enlace a LV; obt. viejo PC
ireturn4	MAR = LV + 1	Ajustar MAR para leer viejo LV
ireturn5	PC = MDR; rd; fetch	Restaurar PC; traer siguiente cód. op.
ireturn6	MAR = SP	Ajustar MAR p/escribir TOS
ireturn7	LV = MDR	Restaurar LV
ireturn8	MDR = TOS; wr; goto Main1	Guardar valor de retorno en tope original

Figura 4-17. El microprograma del Mic-1 (parte 3 de 3).

Si sucede que el byte en MBR sólo contiene ceros, el código de una instrucción NOP, la siguiente microinstrucción es la rotulada `nop1`, que se obtiene de la posición 0. Puesto que esta instrucción no hace nada, simplemente salta de nuevo al inicio del ciclo principal, donde la secuencia se repite, sólo que ya se había obtenido un nuevo código de operación y se colocó en MBR.

Hacemos hincapié una vez más en que las microinstrucciones de la figura 4-17 no son consecutivas en la memoria y que `Main1` no está en la dirección 0 del almacén de control (porque `nop1` tiene que estar en la dirección 0). Corresponde al microensamblador colocar cada microinstrucción en una dirección apropiada y vincularlas en secuencias cortas empleando el campo `NEXT_ADDRESS`. Cada secuencia inicia en la dirección que corresponde al valor numérico del código de operación IJVM que interpreta (por ejemplo, `POP` inicia en 0x57), pero el resto de la secuencia puede estar en cualquier lugar del almacén de control, y no necesariamente en direcciones consecutivas.

Consideremos ahora la instrucción `IADD` de IJVM. La microinstrucción a la que se ramifica desde el ciclo principal es la rotulada `iadd1`. Esta instrucción inicia las tareas específicas de `IADD`:

1. `TOS` ya está presente, pero se debe obtener de la memoria la segunda palabra desde el tope de la pila hacia abajo.
2. `TOS` se debe sumar a la segunda palabra obtenida de la memoria.
3. El resultado, que se colocará en la pila, se debe almacenar en la memoria y también en el registro `TOS`.

Para obtener el operando almacenado en la memoria es necesario decrementar el apuntador de la pila y escribirlo en `MAR`. Observe que, por suerte, esta dirección también es la que se usará para la escritura subsecuente. Además, puesto que esta posición será el nuevo tope de la pila, hay que asignar este valor a `SP`. Por tanto, una sola operación puede determinar el nuevo valor de `SP` y `MAR`, decrementar `SP`, y escribirlo en ambos registros.

Estas cosas se logran en el primer ciclo, `iadd1`, y se inicia la operación de lectura. Además, `MPC` recibe el valor del campo `NEXT_ADDRESS` de `iadd1`, que es la dirección de `iadd2`, donde sea que esté. Luego se lee `iadd2` del almacén de control. En el segundo ciclo, mientras esperamos que se lea el operando de la memoria, copiamos la palabra del tope de la pila, de `TOS` a `H`, donde estará disponible para la suma cuando la lectura termine.

Al principio del tercer ciclo, `iadd3`, `MDR` contiene el sumando que se obtuvo de la memoria. En este ciclo ese sumando se suma al contenido de `H`, y el resultado se guarda en `MDR` y también en `TOS`. De igual manera, se inicia una operación de escritura que almacena la nueva palabra del tope de la pila en la memoria. En este ciclo el `goto` tiene el efecto de asignar la dirección de `Main1` a `MPC`, lo que nos regresa al punto de partida para ejecutar la siguiente instrucción.

Si el código de operación IJVM subsecuente, que ahora está contenido en `MBR`, es 0x64 (`ISUB`), ocurre otra vez una serie de sucesos casi exactamente igual. Una vez que se ejecuta `Main1`, el control se transfiere a la microinstrucción que está en 0x64 (`isub1`). Esta microinstrucción va seguida de `isub2` e `isub3`, y luego de `Main1` otra vez. La única diferencia entre esta secuencia y la anterior es que en `isub3` el contenido de `H` se resta de `MDR` en lugar de sumársele.

La interpretación de `IAND` es casi idéntica a la de `IADD` e `ISUB`, excepto que se obtiene el `AND` bit por bit de las dos palabras del tope de la pila, en lugar de la suma o la diferencia. Algo similar ocurre con `IOR`.

Si el código de operación IJVM es `DUP`, `POP` o `SWAP`, es preciso ajustar la pila. La instrucción `DUP` simplemente duplica la palabra que está en el tope de la pila. Puesto que el valor de esta palabra ya está almacenado en `TOS`, la operación sólo requiere incrementar `SP` de modo que apunte a la nueva posición, y guardar `TOS` en esa posición. La instrucción `POP` es casi igual de sencilla, pues sólo decrementa `SP` para desechar la palabra que estaba en el tope de la pila. Sin embargo, para mantener en `TOS` la palabra que está en el tope de la pila ahora es necesario leer esa palabra de la memoria y escribirla en `TOS`. Por último, la instrucción `SWAP` implica intercambiar los valores de dos localidades de memoria: las dos palabras que están hasta arriba en la pila. Esto se facilita un poco por el hecho de que `TOS` ya contiene uno de esos valores, de modo que no se tiene que leer de la memoria. Examinaremos con mayor detalle esta instrucción más adelante.

La instrucción `BIPUSH` es un poco más complicada porque el código de operación va seguido de un solo byte, como se muestra en la figura 4-18. El byte se debe interpretar como un entero con signo. Este byte, que ya se obtuvo y se colocó en `MBR` en `Main1`, se debe extender con signo a 32 bits y meter en el tope de la pila. Por tanto, esta secuencia debe extender a 32 bits el signo del byte que está en `MBR` y copiarlo en `MDR`. Por último, `SP` se incrementa y se copia en `MAR`, lo que permite escribir el operando en el tope de la pila. Además, este operando debe copiarse en `TOS`. Cabe señalar que, antes de regresar al programa principal, se debe incrementar `PC` para que el siguiente código de operación esté disponible en `Main1`.

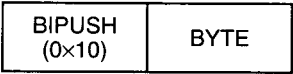


Figura 4-18. El formato de instrucción BIPUSH.

Consideremos ahora la instrucción `ILOAD`. Ésta también tiene un byte después del código de operación, como se muestra en la figura 4-19(a), pero este byte es un índice (sin signo)

que identifica la palabra del espacio de variables locales que debe almacenarse en la pila. Puesto que sólo hay un byte, sólo pueden distinguirse $2^8 = 256$ palabras, a saber, las primeras 256 palabras del espacio de variables locales. La instrucción **ILOAD** requiere tanto una lectura (para obtener la palabra) como una escritura (para almacenarla en el tope de la pila). Sin embargo, para determinar la dirección de lectura es preciso sumar al contenido de **LV** la distancia, contenida en **MBR**. Puesto que tanto **MBR** como **LV** sólo pueden accederse a través del bus **B**, primero se copia **LV** en **H** (en **iload1**), y luego se le suma **MBR**. El resultado de esta suma se copia en **MAR** y se inicia una lectura (en **iload2**).

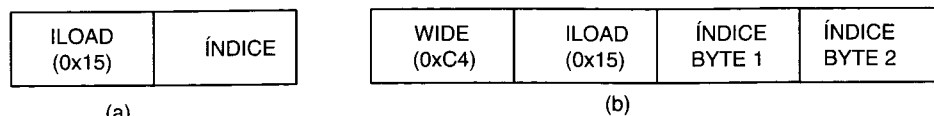


Figura 4-19. (a) **ILOAD** con un índice de un byte. (b) **WIDE ILOAD** con un índice de 2 bytes.

Aquí el uso de **MBR** como índice es un poco diferente que en **BIPUSH**, donde se extendía su signo. En el caso de un índice, la distancia siempre es positiva, por lo que la distancia en bytes debe interpretarse como un entero sin signo, a diferencia de en **BIPUSH**, donde se interpretaba como un entero de 8 bits con signo. La interfaz entre **MBR** y el bus **B** se diseñó con cuidado para hacer posibles ambas operaciones. En el caso de **BIPUSH** (entero de 8 bits con signo), la operación correcta es la extensión de signo, es decir, el copiado del bit de la extrema izquierda de **MBR** (que tiene un byte de longitud) en los 24 bits de orden alto del bus **B**. En el caso de **ILOAD** (entero de 8 bits sin signo), la operación correcta es el llenado con ceros. Aquí los 24 bits de orden alto del bus **B** simplemente se ponen en 0. Estas dos operaciones se distinguen por señales individuales que indican cuál operación debe efectuarse (vea la figura 4-6). En el microcódigo, esto se indica con **MBR** (extensión de signo, como en **BIPUSH3**) o **MBRU** (como en **iload2**).

Mientras se espera que la memoria proporcione el operando (en **iload3**), **SP** se incrementa para que indique dónde se guardará el resultado, el nuevo tope de pila. Este valor también se copia en **MAR** como preparación a escribir el operando en el tope de la pila. Se debe incrementar **PC** otra vez para traer el siguiente código de operación (en **iload4**). Por último, se copia **MDR** en **TOS** para que éste refleje el nuevo tope de la pila (en **iload5**).

ISTORE es la operación opuesta a **ILOAD**, es decir, se saca una palabra del tope de la pila y se almacena en la posición especificada por la suma de **LV** y el índice contenido en la instrucción. **ISTORE** emplea el mismo formato que **ILOAD** (figura 4-19(a)), excepto que con el código de operación 0x36 en lugar de 0x15. Esta instrucción es un poco diferente de lo que cabría esperar porque ya se conoce la palabra que está en el tope de la pila (está en **TOS**), así que puede almacenarse inmediatamente. Sin embargo, es preciso traer la palabra que será el nuevo tope de la pila, así que se requieren una lectura y una escritura, aunque pueden efectuarse en cualquier orden (o incluso en paralelo, si ello fuera posible).

Tanto **ILOAD** como **ISTORE** están restringidas en cuanto a que sólo accesan las primeras 256 variables locales. Si bien para la mayor parte de los programas este espacio de varia-

bles locales es más que suficiente, es desde luego necesario poder acceder a una variable en cualquier lugar que esté dentro del espacio de variables locales. Para lograr esto, **IJVM** usa el mismo mecanismo empleado en **JVM**: con código de operación especial **WIDE**, llamado **byte prefijo**, seguido del código de operación de **ILOAD** o **ISTORE**. Cuando ocurre esta secuencia, las definiciones de **ILOAD** e **ISTORE** se modifican, y un índice de 16 bits sigue al código de operación en lugar de un índice de 8 bits, como se muestra en la figura 4-19(b).

WIDE se decodifica de la forma acostumbrada y causa una ramificación a **wide1**, que maneja el código de operación **WIDE**. Aunque el código de operación para ensanchar ya está en **MBR**, **wide1** trae el primer byte que sigue al código de operación porque la lógica del microprograma siempre espera que esté ahí. Luego se efectúa una segunda ramificación multivías, esta vez empleando el byte que sigue a **WIDE** para despachar. Sin embargo, dado que **WIDE ILOAD** requiere diferente microcódigo que **ILOAD**, y **WIDE ISTORE** requiere diferente microcódigo que **ISTORE**, etc., la segunda ramificación multivías no puede utilizar simplemente el código de operación como dirección objetivo, que es lo que hace **Main1**.

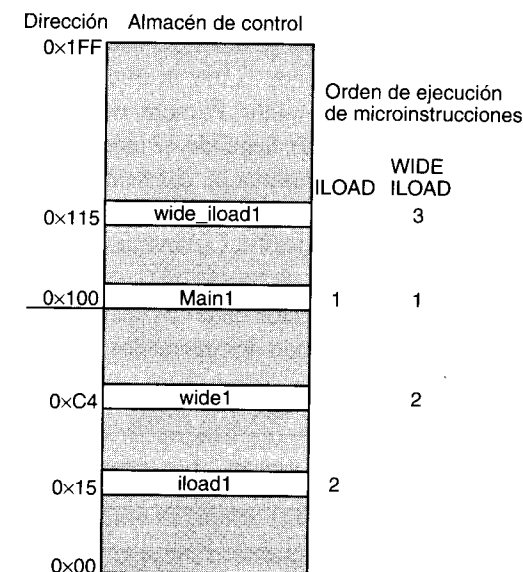


Figura 4-20. La secuencia de microinstrucciones inicial para **ILOAD** y **WIDE ILOAD**. Las direcciones son ejemplos.

En vez de ello, **wide1** hace un OR de 0x100 con el código de operación mientras lo coloca en **MPC**. El resultado es que la interpretación de **WIDE ILOAD** inicia en 0x115 (en lugar de 0x15), la interpretación de **WIDE ISTORE** inicia en 0x136 (en lugar de 0x36), y así. De este modo, todos los códigos de operación **WIDE** inician en una dir 256 (o sea, 0x100) palabras más alta en el almacén de control que el código de operación normal correspondiente. La secuencia inicial de microinstrucciones para **ILOAD** y para **WIDE ILOAD** se muestra en la figura 4-20.

Una vez que se llega al código para implementar una **WIDE ILOAD** (0x115), el código difiere del **ILOAD** normal sólo en cuanto a que el índice debe construirse concatenando dos bytes de índice y no extendiendo el signo de un solo byte. La concatenación y la suma subsecuente deben efectuarse en etapas, copiando primero en **H** el primer byte del índice desplazado 8 bits a la izquierda. Puesto que el índice es un entero sin signo, **MBR** se extiende con ceros empleando **MBRU**. Ahora se suma el segundo byte del índice (la operación de suma es idéntica a la concatenación porque el byte de orden bajo de **H** ahora es cero, lo que garantiza que no habrá acarreo entre los bytes) y el resultado se vuelve a guardar en **H**. De aquí en adelante la operación puede proceder exactamente como si se tratara de un **ILOAD** estándar. En lugar de repetir las instrucciones finales de **ILOAD** (iload3 a iload5), simplemente saltamos de **wide_load4** a **iload3**. Observe, sin embargo, que es preciso incrementar **PC** dos veces durante la ejecución de la instrucción para que al final esté apuntando al siguiente código de operación. La instrucción **ILOAD** lo incrementa una vez; la secuencia **WIDE_ILOAD** también lo incrementa una vez.

La misma situación ocurre con **WIDE_ISTORE**: después de ejecutarse las primeras cuatro microinstrucciones (**wide_istore1** a **wide_istore4**), la secuencia es la misma que en el caso de **ISTORE** después de las primeras dos instrucciones, así que **wide_istore4** salta a **istore3**.

El siguiente ejemplo que consideraremos es una instrucción **LDC_W**. Este código de operación es diferente de **ILOAD** en dos sentidos. Primero, tiene una distancia de 16 bits sin signo (igual que la versión ancha de **ILOAD**). Segundo, se indexa respecto a **CPP**, no respecto a **LV**, ya que su función es leer de la reserva de constantes, no del marco de variables locales. (En realidad, hay una forma corta de **LDC_W** (**LDC**), pero no la incluimos en **IJVM** porque la forma larga incluye todas las posibles variaciones de la forma corta pero ocupa 3 bytes en lugar de 2.)

La instrucción **IINC** es la única instrucción de **IJVM** aparte de **ISTORE** que puede modificar una variable local. Lo hace incluyendo dos operandos, cada uno de un byte, como se muestra en la figura 4-21.

IINC (0x84)	ÍNDICE	CONST
----------------	--------	-------

Figura 4-21. La instrucción **IINC** tiene dos campos de operando distintos.

La instrucción **IINC** usa **ÍNDICE** para especificar la distancia desde el principio del marco de variables locales. **IINC** lee esa variable, la incrementa en **CONST**, un valor contenido en la instrucción, y la vuelve a guardar en la misma localidad. Cabe señalar que esta instrucción puede incrementar negativamente, es decir, **CONST** es una constante de 8 bits con signo en el intervalo -128 a $+127$. La JVM completa incluye una versión ancha de **IINC** en la que cada operando ocupa 2 bytes.

Ahora llegamos a la primera instrucción de ramificación de **IJVM**, **GOTO**. La única función de esta instrucción es modificar el valor de **PC**, de modo que la siguiente instrucción **IJVM** que se ejecute sea la que está en la dirección que se calcula sumando la distancia de 16 bits (con signo) a la dirección del código de operación de ramificar. Una complicación aquí

es que la distancia es relativa al valor que **PC** tenía cuando comenzó a decodificarse la instrucción, no el valor que tiene una vez que se han traído los dos bytes que indican la distancia.

Para aclarar bien esto, en la figura 4-22(a) vemos la situación al inicio de **Main1**. El código de operación ya está en **MBR**, pero todavía no se ha incrementado **PC**. En la figura 4-22(b) vemos la situación al principio de **goto1**. A estas alturas ya se incrementó **PC** y el primer byte de la distancia se trajo y se colocó en **MBR**. Una microinstrucción después tenemos la situación de la figura 4-22(c), en la que el antiguo **PC**, que apunta al código de operación, se ha guardado en **OPC**. Este valor es necesario porque la distancia de la instrucción **GOTO** de **IJVM** es relativa a él, no al valor actual de **PC**. De hecho, ésta es la razón por la que necesitamos el registro **OPC** en primera instancia.

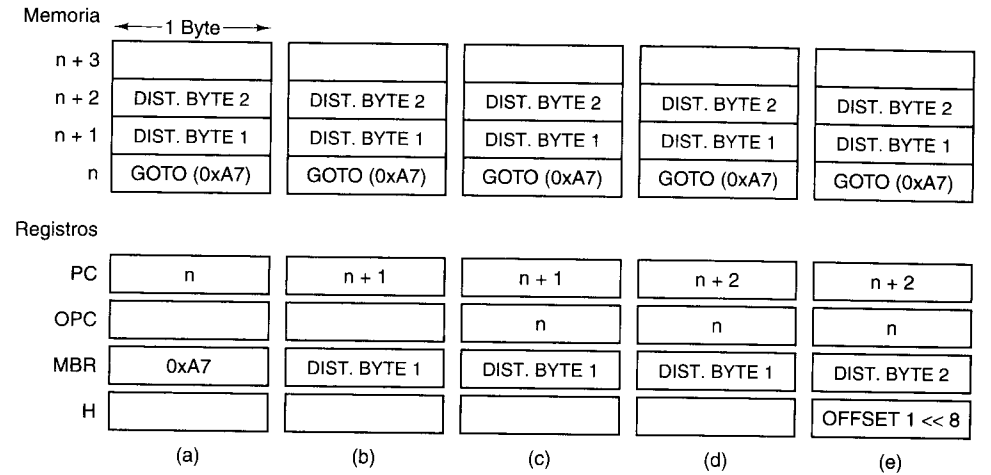


Figura 4-22. La situación al principio de diversas microinstrucciones. (a) **Main1**. (b) **goto1**. (c) **goto2**. (d) **goto3**. (e) **goto4**.

La microinstrucción en **goto2** inicia la búsqueda del segundo byte de distancia, lo que da pie a la figura 4-22(d) al principio de **goto3**. Una vez que el primer byte de distancia se ha desplazado 8 bits a la izquierda y se ha copiado en **H**, llegamos a **goto4** y a la figura 4-22(e). Ahora tenemos el primer byte de distancia desplazado a la izquierda en **H**, el segundo byte de distancia en **MBR**, y la base en **OPC**. Si construimos la distancia completa de 16 bits en **H** y la sumamos después a la base, obtenemos la nueva dirección que debe almacenarse en **PC**, en **goto5**. Tome nota de que en **goto4** usamos **MBRU** en lugar de **MBR** porque no queremos extender el signo del segundo byte. De hecho, la distancia de 16 bits se construye haciendo el OR de las dos mitades. Por último, debemos traer el siguiente código de operación antes de regresar a **Main1** porque ese código espera que el siguiente código de operación esté en **MBR**. El último ciclo, **goto6**, es necesario porque debemos traer los datos de la memoria con tiempo para que aparezcan en **MBR** durante **Main1**.

Las distancias empleadas en la instrucción **goto** de **IJVM** son valores de 16 bits con signo, con un mínimo de -32768 y un máximo de $+32767$. Esto implica que las ramificacio-

nes no son posibles en ninguna dirección a puntos más distantes que estos valores. Esta propiedad puede considerarse como un defecto o bien como una característica de IJVM (y también de JVM). Quienes la ven como defecto dirían que la definición de JVM no debería restringir su estilo de programación. Los que la ven como característica afirmarían que el trabajo de muchos programadores mejoraría drásticamente si tuvieran pesadillas acerca del temido mensaje del compilador

Programa demasiado grande y ramificado. Debe reescribirlo. Se aborta la compilación.

Lamentablemente (en nuestra opinión), este mensaje sólo aparece cuando una cláusula `then` o `else` excede 32 KB, que por lo regular equivale a unas 50 páginas de Java.

Consideremos ahora las tres instrucciones de ramificación condicional de IJVM: `IFLT`, `IFEQ` e `IF_ICMPEQ`. Las primeras dos sacan la palabra que está en el tope de la pila y toman la rama si la palabra es menor que cero o igual a cero, respectivamente. `IF_ICMPEQ` saca las dos palabras del tope de la pila y toma la rama si y sólo si son iguales. En los tres casos es necesario leer de la memoria una nueva palabra de tope de la pila para guardarla en `TOS`.

El control de estas tres instrucciones es similar: el o los operandos se colocan primero en registros, luego se lee el nuevo valor de tope de la pila y se coloca en `TOS`, y por último se efectúa la prueba y la ramificación. Consideremos primero `IFLT`. La palabra que vamos a probar ya está en `TOS`, pero como `IFLT` saca una palabra de la pila es necesario leer el nuevo tope de pila para guardarlo en `TOS`. Esta lectura se inicia en `ift1`. En `ift2` la palabra que se va a probar se guarda en `OPC` por el momento a fin de poder colocar el nuevo valor en `TOS` en breve sin perder el valor actual. En `ift3` la nueva palabra de tope de pila ya está disponible en `MDR`, así que se copia en `TOS`. Por último, en `ift4` la palabra que se va a probar, que ahora está en `OPC`, se pasa por la `ALU` sin almacenarse y se fija y prueba el bit `N`. Esta microinstrucción también contiene una ramificación que escoge `T` si la prueba tuvo éxito, o `F` en caso contrario.

Si se tiene éxito, el resto de la operación es básicamente igual al principio de la instrucción `GOTO`, y la ejecución simplemente continúa a la mitad de la secuencia de `GOTO`, con `goto2`. Si no se tiene éxito se requiere una secuencia corta (`F`, `F2` y `F3`) para saltarse el resto de la instrucción (la distancia) antes de regresar a `Main1` para continuar con la siguiente instrucción.

El código de `ifeq2` e `ifeq3` sigue la misma lógica, sólo que usa el bit `Z` en lugar del bit `N`. En ambos casos, corresponde al ensamblador de `MAL` reconocer que las direcciones de `T` y `F` son especiales y asegurarse de que se coloquen en direcciones del almacén de control que difieran sólo en el bit de la extrema izquierda.

La lógica para `IF_ICMPEQ` es parecida a la de `IFLT` excepto que aquí es necesario leer también el segundo operando. Dicho operando se guarda en `H` en `if_icmpeq3`, donde se inicia la lectura de la palabra que será el nuevo tope de pila. En este caso también la palabra que era el tope de pila se guarda en `OPC` y la nueva se instala en `TOS`. Por último, la prueba que se efectúa en `if_icmpeq6` es similar a la de `ifeq4`.

Consideremos ahora la implementación de `INVOKEVIRTUAL` y de `RETURN`, las instrucciones para efectuar una llamada de procedimiento y para regresar, como se describió en la sección 4.2.3. `INVOKEVIRTUAL` es una secuencia de 22 microinstrucciones, y es la instrucción IJVM más compleja que se implementó. Su funcionamiento se mostró en la figura

4.12. La instrucción utiliza su distancia de 16 bits para determinar la dirección del método que se invocará. En nuestra implementación, la distancia no es más que una distancia dentro de la Reserva de Constantes. Esta posición dentro de la Reserva de Constantes apunta al método que se invocará. Recuerde, empero, que los primeros 4 bytes de cada método *no* son instrucciones. Más bien, son dos apuntadores de 16 bits. El primero proporciona el número de palabras de parámetros (incluido `OBJREF`; vea la figura 4-12). El segundo proporciona el tamaño del área de variables locales en palabras. Estos campos se obtienen a través del puerto de 8 bits y se arman como si fueran dos distancias de 16 bits dentro de una instrucción.

Luego, la información de enlace necesaria para restablecer la máquina a su estado anterior —la dirección del inicio de la antigua área de variables locales y el antiguo `PC`— se guarda inmediatamente arriba del área de variables locales recién creada y debajo de la nueva pila. Por último, se obtiene el código de operación de la siguiente instrucción y `PC` se incrementa antes de regresar a `Main` para iniciar la siguiente instrucción.

`IRETURN` es una instrucción sencilla que no contiene operandos; simplemente usa la dirección almacenada en la primera palabra del área de variables locales para recuperar la información de enlace. Luego restaura `SP`, `LV` y `PC` a sus valores anteriores y copia el valor devuelto, del tope de la pila actual al tope de la pila original, como se muestra en la figura 4-13.

4.4 DISEÑO DEL NIVEL DE MICROARQUITECTURA

Como casi todo en computación, el diseño del nivel de microarquitectura está lleno de concesiones. Las computadoras tienen muchas características deseables, como rapidez, bajo costo, confiabilidad, facilidad de uso, bajo consumo de energía y tamaño físico. Sin embargo, el equilibrio que determina las decisiones más importantes que el diseñador de una CPU debe hacer es el de rapidez contra costo. En esta sección examinaremos este aspecto con detalle para ver qué puede sacrificarse para obtener qué, cuál es el máximo desempeño que puede alcanzarse, y cuál es su precio en términos de hardware y complejidad.

4.4.1 Rapidez versus costo

Si bien la creación de circuitos que operan a mayor velocidad ha sido el factor más importante para lograr computadoras más rápidas, ello rebasa el alcance de este texto. Los aumentos en rapidez logrados gracias a una mejor organización, aunque son menos espectaculares, también han sido impresionantes. Existen varias formas de medir la rapidez, pero dada una tecnología de circuitos y una ISA, hay tres estrategias básicas para aumentar la velocidad de ejecución:

1. Reducir el número de ciclos de reloj necesarios para ejecutar una instrucción.
2. Simplificar la organización para que el ciclo de reloj pueda ser más corto.
3. Traslapar la ejecución de instrucciones.

Las dos primeras son obvias, pero existe una sorprendente variedad de decisiones de diseño que pueden afectar drásticamente ya sea el número de ciclos de reloj, el periodo del reloj o, lo

que es más común, ambas cosas. En esta sección daremos un ejemplo de cómo la codificación y decodificación de una operación pueden afectar el ciclo de reloj.

El número de ciclos de reloj necesarios para ejecutar un conjunto de operaciones se denomina **longitud de trayectoria**. En ocasiones, la longitud de la trayectoria puede acortarse añadiendo hardware especializado. Por ejemplo, si agregamos un incrementador (que en lo conceptual es un sumador con un lado conectado permanentemente de modo que sume 1) a PC, ya no será necesario usar la ALU para incrementar PC, y en consecuencia se eliminan ciclos. El precio que se paga es un aumento en el hardware. Sin embargo, esta capacidad no ayuda tanto como cabría esperar. En casi todas las instrucciones los ciclos que se consumen incrementando PC son también ciclos durante los cuales se está efectuando una operación de lectura. De todos modos no sería posible ejecutar la siguiente instrucción antes porque depende de los datos que se están trayendo de la memoria.

Reducir el número de ciclos de instrucción necesarios para traer instrucciones requiere algo más que un circuito adicional para incrementar el contador de programa. Si queremos acelerar la obtención de instrucciones de forma apreciable es preciso aprovechar la tercera técnica: traslapar la ejecución de instrucciones. La separación de los circuitos que traen las instrucciones —el puerto de memoria de 8 bits y los registros MBR y PC— es más eficaz si se hace que la unidad sea funcionalmente independiente de la trayectoria de datos principal. Así, esos circuitos pueden traer el siguiente código de operación u operando por sí solos, tal vez operando asincrónicamente respecto al resto de la CPU y obteniendo una o más instrucciones por adelantado.

Una de las fases de la ejecución de muchas instrucciones que más tiempo consume es traer una distancia de dos bytes, extenderla de la forma apropiada, y acumularla en el registro H como preparación para una suma, por ejemplo, en una ramificación a $PC \pm n$ bytes. Una posible solución —hacer que el puerto de memoria tenga un ancho de 16 bits— complica considerablemente la operación, porque la memoria en realidad tiene una anchura de 32 bits. Los 16 bits requeridos podrían cruzar la frontera entre dos palabras, de modo que ni siquiera una sola lectura de 32 bits necesariamente traería los dos bytes que se necesitan.

Traslapar la ejecución de las instrucciones es por mucho el método más interesante y el que más oportunidades ofrece para aumentar drásticamente la rapidez. El simple traslapo de la búsqueda y la ejecución de las instrucciones es sorprendentemente eficaz. Sin embargo, las técnicas más sofisticadas van mucho más lejos, pues traslapan la ejecución de muchas instrucciones. De hecho, esta idea es fundamental para el diseño de computadoras modernas. A continuación analizaremos algunas técnicas básicas para traslapar la ejecución de instrucciones y delinearemos algunas de las más complejas.

La rapidez es sólo la mitad de la historia; la otra mitad es el costo. Además, hay varias formas de medir el costo, y definir éste con precisión es problemático. Algunas medidas son tan sencillas como un recuento del número de componentes. Esto era válido sobre todo en los tiempos en que los procesadores se construían con componentes discretos que se compraban y armaban. Hoy día todo el procesador existe en un solo chip, pero los chips grandes y complejos son más caros que los pequeños y sencillos. Podemos contar los componentes individuales, como transistores, compuertas o unidades funcionales, pero a menudo esa cuenta no es tan importante como el área que se requiere en el circuito integrado. Entre mayor sea el

área que se requiera para las funciones incluidas, más grande será el chip. Y el costo de fabricación del chip crece con mucha mayor rapidez que su área. Por esta razón, los diseñadores a menudo hablan del costo en términos de “terreno”, es decir, el área requerida para un circuito (¿quizá medida en picohectáreas?).

Uno de los circuitos más minuciosamente estudiados en la historia es el sumador binario. Se han creado miles de diseños, y los más rápidos son mucho más veloces que los más lentos, aunque también son mucho más complejos. El diseñador de sistemas tiene que decidir si el aumento en la velocidad justifica ocupar más “terreno”.

Los sumadores no son el único componente que ofrece muchas opciones. Casi todos los componentes del sistema se pueden diseñar de modo que operen con mayor o menor velocidad, pero con diferente costo. El reto del diseñador es identificar los componentes del sistema que pueden mejorarlo más si se hacen más rápidos. Resulta interesante que muchos componentes individuales se pueden sustituir por un componente mucho más rápido sin que se observe un efecto apreciable sobre la rapidez del sistema. En las secciones que siguen examinaremos algunas cuestiones de diseño y los equilibrios costo-rapidez que implican.

Uno de los factores clave para determinar con qué rapidez puede operar el reloj es la cantidad de trabajo que debe efectuarse en cada ciclo de reloj. Obviamente, cuanto más trabajo haya que realizar, más largo será el ciclo de reloj. Claro que la cosa no es tan sencilla, porque el hardware es muy bueno para hacer cosas en paralelo, así que es realmente la secuencia de operaciones que deben efectuarse *en serie* en un solo ciclo de reloj lo que determina la duración que debe tener el ciclo.

Un aspecto que puede controlarse es la cantidad de decodificación que hay que realizar. Recuerde, por ejemplo, que en la figura 4-6 vimos que si bien era posible introducir cualquiera de nueve registros en la ALU por medio del bus B, sólo requeríamos 4 bits de la palabra de la microinstrucción para especificar cuál registro había que seleccionar. Lo malo es que tales ahorros tienen un precio. El circuito decodificador aumenta el retraso en la ruta crítica. Esto implica que el registro que colocará sus datos en el bus B recibirá esa orden y después pondrá sus datos en el bus. Este efecto se propaga, pues la ALU recibe sus entradas un poco más tarde y produce sus resultados un poco más tarde. Por último, el resultado está disponible en el bus C para escribirse en los registros un poco más tarde. Puesto que este retraso en muchos casos es el factor que determina la longitud que debe tener el ciclo de reloj, podría implicar que el reloj no puede operar tan rápidamente como podría y que toda la computadora debe ser un poco más lenta. Así pues, hay un equilibrio entre rapidez y costo. Reducir el almacén de control en 5 bits por palabra tiene el costo de frenar el reloj. El ingeniero de diseño debe tener en cuenta los objetivos de diseño para decidir qué es lo que más conviene. En el caso de una implementación de alto desempeño, probablemente no es recomendable usar un decodificador; en una de bajo costo tal vez sería lo mejor.

4.4.2 Reducción de la longitud de la trayectoria de ejecución

El Mic-1 se diseñó tratando de hacerlo moderadamente sencillo y rápido, aunque hemos de admitir que existe una tensión enorme entre estas dos metas. En pocas palabras, las máquinas sencillas no son rápidas y las máquinas rápidas no son sencillas. Además, la CPU del Mic-1

emplea un mínimo de hardware: 10 registros, la sencilla ALU de la figura 3-19 repetida 32 veces, un desplazador, un decodificador, un almacén de control y un poco de adhesivo aquí y allá. Todo el sistema podría construirse con menos de 5000 transistores más lo que necesiten el almacén de control (ROM) y la memoria principal (RAM).

Habiendo visto cómo puede implementarse IJVM de forma directa en microcódigo con poco hardware, es momento de examinar otras implementaciones más rápidas. A continuación estudiaremos formas de reducir el número de microinstrucciones por cada instrucción ISA (es decir, reducir la longitud de la trayectoria de ejecución). Después consideraremos otras estrategias.

Fusión del ciclo del intérprete con el microcódigo

En el Mic-1, el ciclo principal consiste en una microinstrucción que debe ejecutarse al principio de cada instrucción IJVM. En algunos casos es posible traslaparla con la instrucción anterior. De hecho, esto ya se logró parcialmente. Observe que cuando se ejecuta Main1 el código de operación que se va a interpretar ya está en MBR. Está ahí porque el ciclo principal anterior la trajo (si la instrucción anterior no tenía operandos) o bien porque se trajo durante la ejecución de la instrucción anterior.

Este concepto de traslapar el principio de la instrucción puede llevarse más lejos, y en algunos casos el ciclo principal puede reducirse a la nada. Esto puede ocurrir como sigue. Consideremos las secuencias de microinstrucciones que terminan con un salto a Main1. En cada uno de estos puntos, la microinstrucción del ciclo principal puede anexarse al final de la sucesión (en lugar de estar al principio de la sucesión siguiente) de modo que la ramificación multivías esté repetida en muchos lugares (pero siempre con el mismo conjunto de objetivos). En algunos casos la microinstrucción Main1 se puede fusionar con microinstrucciones previas, ya que tales instrucciones no siempre se aprovechan plenamente.

En la figura 4-23 se muestra la sucesión dinámica de instrucciones para una instrucción POP. El ciclo principal ocurre antes y después de cada instrucción; en la figura mostramos sólo la ocurrencia que sigue a la instrucción POP. Observe que la ejecución de esta instrucción tarda cuatro ciclos: tres para las microinstrucciones específicas de POP y una para el ciclo principal.

Mnemónico	Operaciones	Comentarios
pop1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
pop2		Esperar lectura de nueva TOS de la memoria
pop3	TOS = MDR; goto Main 1	Copiar nueva palabra en TOS
Main	PC = PC + 1; fetch; goto (MDR)	MDR tiene cód. op.; obt. sig. byte. despachar

Figura 4-23. Nueva secuencia de microprograma para ejecutar POP.

En la figura 4-24 hemos reducido la sucesión a tres instrucciones fusionando las instrucciones del ciclo principal, aprovechando un ciclo de reloj en el que la ALU no se usa en pop2 para ahorrar un ciclo, y otra vez en Main1. Observe que el final de esta sucesión salta direc-

tamente al código específico de la instrucción subsecuente, de modo que sólo se requiere un total de tres ciclos. Este pequeño truco reduce el tiempo de ejecución de la siguiente microinstrucción en un ciclo, de modo que, por ejemplo, una IADD subsecuente baja de cuatro ciclos a tres, y esto equivale a acelerar el reloj de 250 MHz (microinstrucciones de 4 ns) a 333 MHz (microinstrucciones de 3 ns) gratis.

Mnemónico	Operaciones	Comentarios
pop1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
Main1.pop	PC = PC + 1; fetch	MDR contiene cód. op.; traer siguiente byte
pop3	TOS = MDR; goto (MDR)	Copiar nva. pal. en TOS; despachcar según cód. op.

Figura 4-24. Secuencia de microprograma mejorada para ejecutar POP.

La instrucción POP se presta mucho para este tratamiento porque tiene un ciclo muerto a la mitad que no usa la ALU. El ciclo principal, en cambio, sí usa la ALU. Por tanto, para reducir la longitud de las instrucciones en un ciclo es necesario encontrar un ciclo de la instrucción en el que la ALU no se esté usando. Tales ciclos muertos no son comunes, pero sí ocurren, así que vale la pena fusionar Main1 con el final de cada sucesión de microinstrucciones. Lo único que cuesta es un poco de espacio en el almacén de control. Así pues, ya tenemos nuestra primera técnica para reducir la longitud de la trayectoria:

Fusionar el ciclo del intérprete con el final de cada sucesión de microcódigo.

Una arquitectura de tres buses

¿Qué otra cosa podemos hacer para reducir la longitud de la trayectoria de ejecución? Otra solución fácil es que la ALU tenga dos buses de entrada completos, un bus A y un bus B, para un total de tres buses. Todos (o al menos casi todos) los registros deberán tener acceso a ambos buses de entrada. La ventaja de tener dos buses de entrada es que hace posible sumar cualesquier dos registros en un solo ciclo. Para ver el valor de esta característica, consideremos la implementación de ILOAD en Mic-1, que se muestra otra vez en la figura 4-25.

Mnemónico	Operaciones	Comentarios
iload1	H = LV	MDR contiene índice; copiar LV en H
iload2	MAR = MBRU + H; rd	MAR = dir. de variable local que se meterá
iload3	MAR = SP = SP + 1	SP apunta a nuevo tope de pila; preparar escrit.
iload4	PC = PC + 1; fetch; wr	Inc. PC; obt. sig. cód. op.; escribir en tope de pila
iload5	TOS = MDR; goto Main1	Actualizar TOS
Main1	PC = PC + 1; fetch; goto (MDR)	MDR tiene cód. op.; obt. sig. byte; despachar

Figura 4-25. Código de Mic-1 para ejecutar ILOAD.

Aquí vemos que en iload1 se copia LV en H. La única razón por la que se hace esto es para sumarlo a MBRU en iload2. En nuestro diseño original de dos buses, no hay forma de

De forma similar, MBR2 ofrece la misma funcionalidad pero contiene los dos bytes siguientes, y también tiene dos interfaces con el bus B: MBR2 y MBR2U, que guardan los valores extendidos a 32 bits con el signo y con ceros, respectivamente.

La IFU se encarga de traer una serie de bytes. Para ello, usa un puerto de memoria convencional de 4 bytes, obtiene por adelantado palabras completas de 4 bytes y carga los bytes consecutivos en un registro de desplazamiento que los proporciona de uno en uno o dos a la vez, en el orden en que se obtuvieron de la memoria. La función del registro de desplazamiento es mantener una cola de bytes de la memoria para alimentarlos a MBR1 y MBR2.

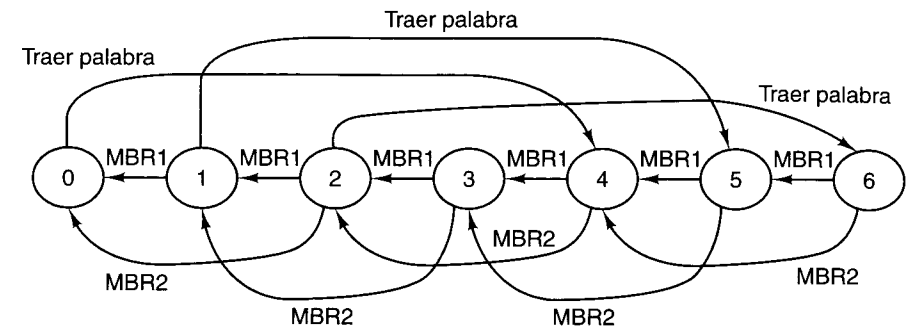
En todo momento MBR1 contiene el byte menos reciente de los que están en el registro de desplazamiento y MBR2 contiene los 2 bytes menos recientes (con el menos reciente a la izquierda) para formar un entero de 16 bits [vea la figura 4-19(b)]. Los dos bytes que están en MBR2 podrían provenir de diferentes palabras de la memoria, porque las instrucciones IJVM no se alinean según las fronteras de palabra en la memoria.

Cada vez que se lee MBR1, el registro de desplazamiento corre su contenido un byte a la derecha. Cada vez que se lee MBR2, el desplazamiento es de dos bytes a la derecha. Luego se vuelven a cargar MBR1 y MBR2 con el byte y el par de bytes menos recientes, respectivamente. Si existe suficiente espacio en el registro de desplazamiento para otra palabra entera, la IFU inicia un ciclo de memoria para leerla. Suponemos que cuando se lee cualquiera de los registros MBR, se ha vuelto a llenar para cuando inicia el siguiente ciclo, así que pueden leerse en ciclos consecutivos.

El diseño de la IFU se puede modelar con una **máquina de estados finitos (FSM, Finite State Machine)** como se muestra en la figura 4-28. Todas las FSM constan de dos partes: **estados**, que se indican con círculos, y **transiciones**, que se indican con flechas de un estado a otro. Cada estado representa una posible situación en que la FSM puede estar. Esta FSM en particular tiene siete estados, que corresponden a los siete estados del registro de desplazamiento de la figura 4-27. Los siete estados corresponden al número de bytes que están actualmente en el registro de desplazamiento, un número entre 0 y 6, inclusive.

Cada flecha representa un suceso que puede ocurrir. Aquí pueden ocurrir tres sucesos distintos. El primero es que se lea un byte de MBR1. Este suceso hace que el registro de desplazamiento se active y desplace su contenido un byte hacia la derecha, “expulsando” un byte por su extremo derecho y reduciendo su estado en 1. El segundo suceso es que se lean dos bytes de MBR2, lo que reduce el estado en 2. Ambas transiciones hacen que MBR1 y MBR2 se vuelvan a cargar. Cuando la FSM pasa a los estados 0, 1 o 2, se inicia una referencia a la memoria para leer una nueva palabra (suponiendo que la memoria no está ocupada leyendo una palabra). La llegada de la palabra adelanta el estado en 4.

Para que funcione correctamente, la IFU debe bloquearse cuando se le pide que haga algo que no puede hacer, como proporcionar el valor de MBR2 cuando sólo hay un byte en el registro de desplazamiento y la memoria sigue ocupada trayendo una nueva palabra. Además, sólo puede hacer una cosa a la vez, de modo que los sucesos deben colocarse en serie. Por último, cada vez que se modifique PC, habrá que actualizar la IFU. Estos detalles hacen que la IFU sea más complicada de lo que se muestra. No obstante, muchos dispositivos de hardware se construyen como máquinas de estados finitos.



Transiciones

MBR1: Ocurre cuando se lee MBR1

MBR2: Ocurre cuando se lee MBR2

Traer palabra: Ocurre cuando se lee una palabra de memoria y se colocan 4 bytes en el registro de desplazamiento

Figura 4-28. Máquina de estados finitos para implementar la IFU.

La IFU tiene su propio registro de dirección de memoria, llamado **IMAR**, que se usa para direccionar la memoria cuando es preciso traer una nueva palabra. Este registro tiene su propio incrementador dedicado, así que no se necesita la ALU principal para incrementarlo y así obtener la siguiente palabra. La IFU debe vigilar el bus C de modo que, cada vez que se cargue PC, el nuevo valor de PC también se copie en IMAR. Puesto que el nuevo valor que está en PC no necesariamente está en una frontera de palabra, la IFU tiene que traer la palabra necesaria y ajustar el registro de desplazamiento de manera acorde.

Con la IFU, la unidad de ejecución principal escribe en PC sólo cuando es necesario alterar la naturaleza secuencial del flujo de bytes de las instrucciones. Esto ocurre cuando una instrucción de ramificación condicional tiene éxito y también con **INVOKEVIRTUAL** y **RETURN**.

Puesto que el microprograma ya no incrementa explícitamente PC cuando se leen códigos de operación, la IFU debe mantener PC al día. Para ello, detecta cuándo se ha consumido un byte del flujo de instrucciones, es decir, cuándo se han leído MBR1 o MBR2 (o sus versiones sin signo). PC tiene asociado un incrementador propio, capaz de incrementar en 1 o en 2, dependiendo de cuántos bytes se hayan consumido. Así, el PC siempre contiene la dirección del primer byte que no se ha consumido. Al principio de cada instrucción MBR contiene la dirección del código de operación de esa instrucción.

Observe que hay dos incrementadores distintos y que tienen diferentes funciones. PC cuenta *bytes* y se incrementa en 1 o 2. IMAR cuenta *palabras* y se incrementa sólo en 1 (para obtener 4 bytes nuevos). Al igual que MAR, IMAR está conectado al bus de direcciones con sesgo (el bit 0 de IMAR conectado a la línea de dirección 2, etc.) para realizar una conversión implícita de direcciones de palabras en direcciones de bytes.

Como veremos en breve, no tener que incrementar PC en el ciclo principal representa un ahorro considerable, ya que la microinstrucción en la que PC se aumenta no hace mucho más

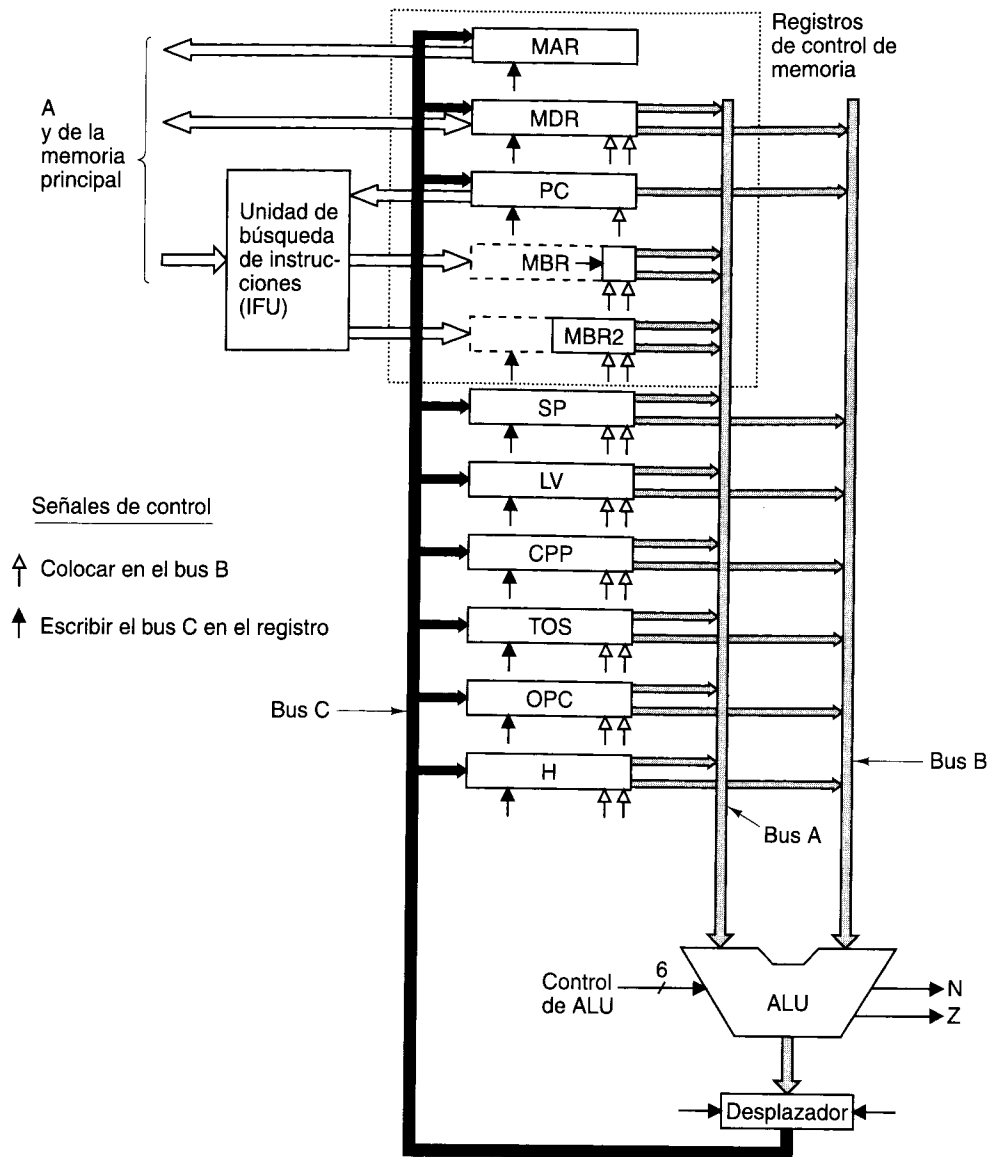


Figura 4-29. Trayectoria de datos del Mic-2.

que incrementar PC. Si podemos eliminar esta microinstrucción, se reducirá la trayectoria de ejecución. El precio aquí es más hardware a cambio de una máquina más rápida, así que nuestra tercera técnica para reducir la longitud de la trayectoria es

Hacer que una unidad especializada traiga las instrucciones de la memoria.

4.4.3 Un diseño con prebúsqueda: el Mic-2

La IFU puede reducir considerablemente la longitud de trayectoria de una instrucción típica. Primero, elimina el ciclo principal, puesto que el final de cada instrucción salta directamente a la siguiente instrucción. Segundo, evita ocupar la ALU en incrementar PC. Tercero, reduce la longitud de trayectoria cada vez que se calcula un índice o distancia de 16 bits, porque arma el valor de 16 bits y lo proporciona directamente a la ALU como valor de 32 bits, con lo que se evita la necesidad de armarlo en H. La figura 4-29 muestra el Mic-2, una versión mejorada del Mic-1 en la que se ha añadido la IFU de la figura 4-27. El microcódigo de la máquina mejorada se muestra en la figura 4-30.

Como ejemplo del funcionamiento del Mic-2, estudiemos IADD. Esta instrucción trae la segunda palabra de la pila y realiza la suma igual que antes, sólo que ahora no tiene que ir a Main1 cuando termina para incrementar PC y saltar a la siguiente microinstrucción. Cuando la IFU ve que se hizo referencia a MBR1 en iadd3, su registro de desplazamiento interno recorre todo a la derecha y vuelve a cargar MBR1 y MBR2; además, efectúa una transición a un estado uno más abajo que su estado actual. Si el nuevo estado es 2, la IFU comienza a leer una palabra de la memoria. Todo esto se hace en hardware. El microprograma no tiene que hacer nada. Es por ello que IADD puede reducirse de cuatro microinstrucciones a tres.

El Mic-2 mejora algunas instrucciones más que otras. LDC_W baja de nueve microinstrucciones a sólo tres, lo que reduce su tiempo de ejecución por un factor de 3. En cambio, SWAP sólo baja de ocho a seis microinstrucciones. En lo que respecta al desempeño global, lo que cuenta realmente es la ganancia que se obtiene en las instrucciones más comunes. Éstas incluyen ILOAD (antes 6, ahora 3), IADD (antes 4, ahora 3) y IF_ICMPEQ (antes 13, ahora 10 si se ramifica; antes 10, ahora 8 si no se ramifica). Para medir la mejora sería necesario escoger algunos programas de referencia y ejecutarlos, pero es evidente que las ganancias aquí son importantes.

4.4.4 Un diseño con filas de procesamiento: el Mic-3

Es obvio que el Mic-2 es mejor que el Mic-1: es más rápido y ocupa menos espacio en el almacén de control, aunque el costo de la IFU sin duda será mayor que lo que se ahorra en "terreno" al tener un almacén de control más pequeño. Por tanto, tenemos una máquina considerablemente más rápida por un precio un poco más alto. Veamos si podemos hacerla más rápida aún.

¿Qué tal si tratamos de reducir el tiempo de ciclo? En gran parte, el tiempo de ciclo está determinado por la tecnología subyacente. Cuanto más pequeños sean los transistores y menores sean las distancias físicas entre ellos, más rápidamente podrá operar el reloj. Para una tecnología dada, el tiempo requerido para efectuar una operación completa de trayectoria de datos está fijo (al menos desde nuestro punto de vista). No obstante, disponemos de cierta libertad que dentro de muy poco aprovecharemos al máximo.

Nuestra otra opción es introducir más paralelismo en la máquina. De momento, el Mic-2 es altamente secuencial: coloca registros en sus buses, espera que la ALU y el desplazador

Mnemónico	Operaciones	Comentarios
nop1	goto (MBR)	Saltar a la siguiente instrucción
iadd1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
iadd2	H = TOS	H = tope de la pila
iadd3	MDR = TOS = MDR+H; wr; goto (MBR1)	Sumar dos palabras del tope; escribir en nvo. tope de la pila
isub1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
isub2	H = TOS	H = tope de la pila
isub3	MDR = TOS = MDR-H; wr; goto (MBR1)	Restar TOS del TOS-1 traído
iand1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
iand2	H = TOS	H = tope de la pila
iand3	MDR = TOS = MDR AND H; wr; goto (MBR1)	Hacer AND del TOS-1 traído y TOS
ior1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
ior2	H = TOS	H = tope de la pila
ior3	MDR = TOS = MDR OR H; wr; goto (MBR1)	Hacer OR del TOS-1 traído y TOS
dup1	MAR = SP = SP + 1	Incrementar SP; copiar en MAR
dup2	MDR = TOS; wr; goto (MBR1)	Escribir nueva palabra de la pila
pop1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
pop2		Esperar lectura
pop3	TOS = MDR; goto (MBR1)	Copiar nueva palabra en TOS
swap1	MAR = SP - 1; rd	Leer 2a. palabra de la pila; asignar SP a MAR
swap2	MAR = SP	Repararse para escribir nueva 2a. palabra
swap3	H = MDR; wr	Guardar nuevo TOS; escribir 2a. pal. en pila
swap4	MDR = TOS	Copiar TOS viejo en MDR
swap5	MAR = SP - 1; wr	Escribir TOS viejo en 2o. lugar de la pila
swap6	TOS = H; goto (MBR1)	Actualizar TOS
bipush1	SP = MAR = SP + 1	Reparar MAR p/escribir en nuevo tope de pila
bipush2	MDR = TOS = MBR1; wr; goto (MBR1)	Actualizar pila en TOS y memoria
iload1	MAR = LV + MBR1U; rd	Pasar LV + índice a MAR; leer operando
iload2	MAR = SP = SP + 1	Incrementar SP; pasar nuevo SP a MAR
iload3	TOS = MDR; wr; goto (MBR1)	Actualizar pila en TOS y memoria
istore1	MAR = LV + MBR1U	Asignar LV + índice a MAR
istore2	MDR = TOS; wr	Copiar TOS para guardarlo
istore3	MAR = SP = SP - 1; rd	Decrementar SP; leer nuevo TOS
istore4		Esperar lectura
istore5	TOS = MDR; goto (MBR1)	Actualizar TOS
wide1	goto (MBR1 OR 0x100)	Sig. dir. es 0x100 OR código de operación
wide_iload1	MAR = LV + MBR2U; rd; goto iload2	Idéntico a iload1 pero usa índice de 2 bytes
wide_istore1	MAR = LV + MBR2U; goto istore2	Idéntico a iload1 pero usa índice de 2 bytes
idc_w1	MAR = CPP + MBR2U; rd; goto iload2	Igual que wide_iload1 pero indiza en CPP
iinc1	MAR = LV + MBR1U; rd	Asignar LV + índice a MAR para lectura
iinc2	H = MBR1	Asignar constante a H
iinc3	MDR = MDR + H; wr; goto (MBR1)	Incrementar constante y actualizar
goto1	H = PC - 1	Copiar PC en H
goto2	PC = H + MBR2	Sumar distancia y actualizar PC
goto3		Esperar que IFU traiga nuevo cód. op.
goto4	goto (MBR1)	Despachar a siguiente instrucción
iflt1	MAR = SP = SP - 1; rd	Leer segunda palabra de la pila
iflt2	OPC = TOS	Guardar TOS en OPC temporalmente
iflt3	TOS = MDR	Poner nuevo tope de pila en TOS
iflt4	N = OPC; if (N) goto T; else goto F	Ramificar según bit N

Figura 4-30. El microprograma del Mic-2 (parte 1 de 2).

Mnemónico	Operaciones	Comentarios
ifeq1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
ifeq2	OPC = TOS	Guardar TOS en OPC temporalmente
ifeq3	TOS = MDR	Poner nuevo tope de pila en TOS
ifeq4	Z = OPC; if (Z) goto T; else goto F	Ramificar según bit Z
if_icmpeq1	MAR = SP = SP - 1; rd	Leer 2a. palabra de la pila
if_icmpeq2	MAR = SP = SP - 1	Ajustar MAR para leer nuevo tope de pila
if_icmpeq3	H = MDR; rd	Copiar 2a. palabra de la pila en H
if_icmpeq4	OPC = TOS	Guardar TOS en OPC temporalmente
if_icmpeq5	TOS = MDR	Poner nuevo tope de pila en TOS
if_icmpeq6	Z = H - OPC; if (Z) goto T; else goto F	Si 2 pal. en tope iguales, ir a T; si no, a F
T	H = PC - 1; goto goto2	Igual que goto1
F	H = MBR2	Tocar bytes de MBR2 que se desecharán
F2	goto (MBR1)	
invokevirtual1	MAR = CPP + MBR2U; rd	Poner dir. de apunt. a método en MAR
invokevirtual2	OPC = PC	Guardar PC de retorno en OPC
invokevirtual3	PC = MDR	Hacer que PC apunte a 1er. byte de método
invokevirtual4	TOS = SP - MBR2U	TOS = dirección de OBJREF - 1
invokevirtual5	TOS = MAR = H = TOS + 1	TOS = dirección de OBJREF
invokevirtual6	MDR = SP + MBR2U + 1; wr	Sobreescribir OBJREF con apunt. enlace
invokevirtual7	MAR = SP = MDR	Asignar a SP, MAR, pos. p/guardar viejo PC
invokevirtual8	MDR = OPC; wr	Prepararse para guardar viejo PC
invokevirtual9	MAR = SP = SP + 1	Inc. SP p/que apunte a donde se guardará viejo LV
invokevirtual10	MDR = LV; wr	Guardar viejo LV
invokevirtual11	LV = TOS; goto (MBR1)	Hacer que LV apunte a parámetro 0
ireturn1	MAR = SP = LV; rd	Restab. SP, MAR, p/leer apunt. de enlace
ireturn2		Esperar apuntador de enlace
ireturn3	LV = MAR = MDR; rd	Asignar apunt. enlace a LV, MAR; leer viejo PC
ireturn4	MAR = LV + 1	Hacer que MAR apunte a viejo LV; leer viejo LV
ireturn5	PC = MDR; rd	Restaurar PC
ireturn6	MAR = SP	
ireturn7	LV = MDR	Restaurar LV
ireturn8	MDR = TOS; wr; goto (MBR1)	Guardar valor de retorno en tope de pila original

Figura 4-30. El microprograma del Mic-2 (parte 2 de 2).

los procesen y luego escribe los resultados de vuelta en los registros. Fuera de la IFU, casi no hay paralelismo. La adición de paralelismo es una gran oportunidad.

Como dijimos antes, el ciclo de reloj está limitado por el tiempo que se requiere para que las señales se propaguen por la trayectoria de datos. La figura 4-3 muestra un desglose del retardo a través de los diversos componentes durante cada ciclo. El ciclo de la trayectoria de datos real tiene tres componentes principales:

1. El tiempo que toma colocar los registros seleccionados en los buses A y B.
2. El tiempo que tardan la ALU y el desplazador en realizar su trabajo.
3. El tiempo que tardan los resultados en regresar a los registros y guardarse.

En la figura 4-31 mostramos una nueva arquitectura de tres buses que incluye la IFU pero tiene tres latches (registros) adicionales, cada uno insertado en medio de un bus. Se

escribe en los latches en cada ciclo. De hecho, los latches dividen la trayectoria de datos en partes discretas que ahora pueden operar de forma independiente. Llamaremos a este modelo **Mic-3**, o modelo **con filas de procesamiento**.

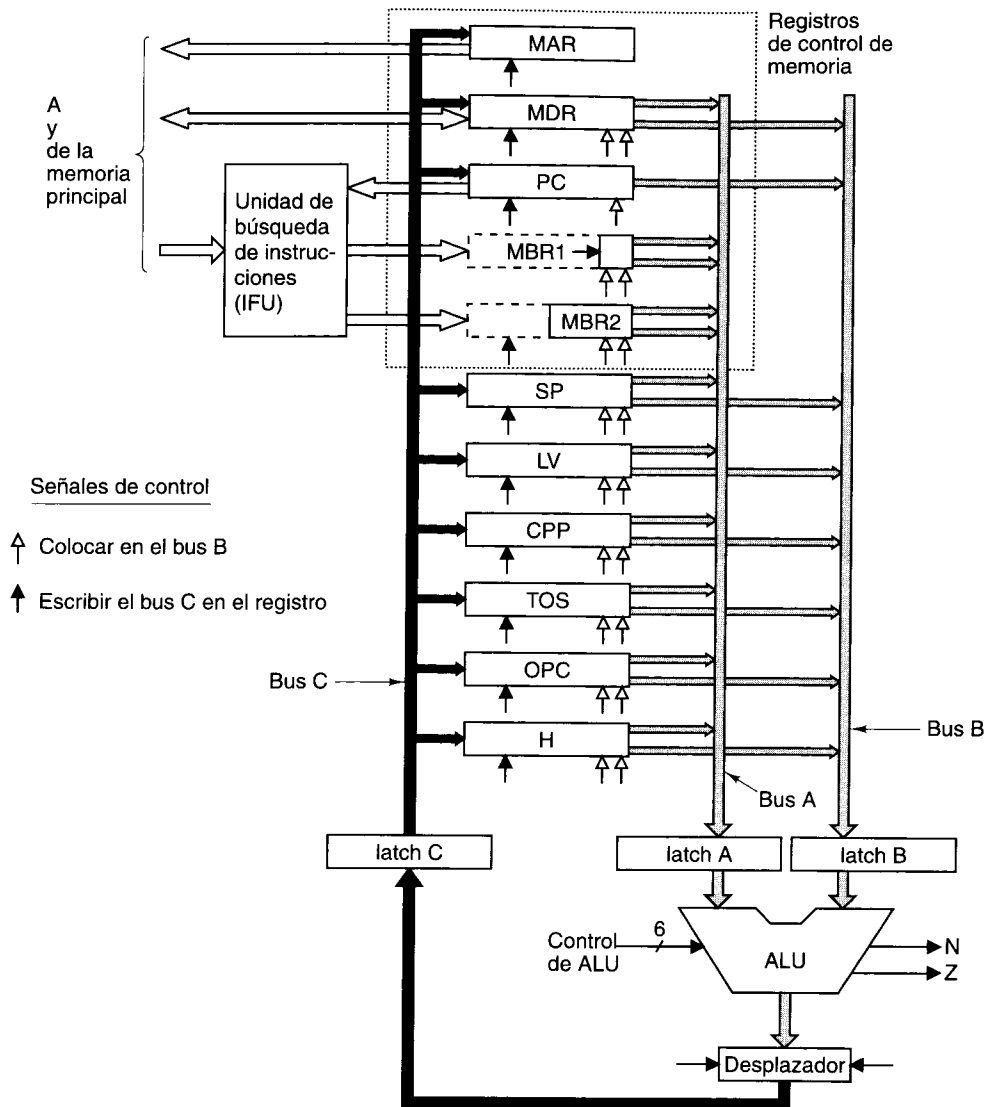


Figura 4-31. La trayectoria de datos de tres buses empleado en el Mic-3.

¿Cómo es posible que estos registros adicionales sirvan de algo? Ahora se requieren tres ciclos de reloj para usar la trayectoria de datos: uno para cargar los latches A y B, uno para

operar la ALU y el desplazador, además de cargar el latch C, y uno para almacenar el contenido del latch C de vuelta en los registros. ¿Acaso estamos locos? (Sugerencia: No.) La ventaja de insertar los latches es doble:

1. Podemos acelerar el reloj porque el retraso máximo ahora es más corto.
2. Podemos usar todas las partes de la trayectoria de datos durante cada ciclo.

Al descomponer la trayectoria de datos en tres partes, redujimos el retardo máximo, y el resultado es que la frecuencia del reloj puede ser más alta. Supongamos que al dividir el ciclo de la trayectoria de datos en tres intervalos de tiempo hacemos que cada uno tenga una duración de aproximadamente 1/3 de la original, así que podemos triplicar la velocidad del reloj. (Esto no es del todo realista porque también añadimos dos registros más a la trayectoria de datos, pero sirve como primera aproximación.)

Puesto que hemos estado suponiendo que todas las lecturas y escrituras de la memoria pueden satisfacerse con el contenido de la memoria caché de nivel 1, y esta caché está hecha con el mismo material que los registros, seguiremos suponiendo que una operación de memoria tarda un ciclo. En la práctica, empero, esto podría ser difícil de lograr.

El segundo punto tiene que ver con el rendimiento más que con la rapidez de una instrucción individual. En el Mic-2, durante la primera y la tercera partes de cada ciclo de reloj, la ALU está ociosa. Si dividimos la trayectoria de datos en tres partes, podremos usar la ALU en cada ciclo y obtener hasta tres veces más trabajo de la máquina.

Veamos ahora cómo funciona la trayectoria de datos del Mic-3. Antes de comenzar, necesitamos una notación para manejar los latches. La obvia es llamar a los latches A, B y C y tratarlos como registros, teniendo presentes las restricciones de la trayectoria de datos. La figura 4-32 muestra un ejemplo de secuencia de código, la implementación de **SWAP** para el Mic-2.

Mnemónico	Operaciones	Comentarios
swap1	MAR = SP - 1; rd	Leer 2a. palabra de la pila; asignar SP a MAR
swap2	MAR = SP	Prepararse para escribir nueva 2a. palabra
swap3	H = MDR; wr	Guardar nuevo TOS; escribir 2a. pal. en la pila
swap4	MDR = TOS	Copiar TOS viejo en MDR
swap5	MAR = SP - 1; wr	Escribir TOS viejo en 2o. lugar de la pila
swap6	TOS = H; goto (MBR1)	Actualizar TOS

Figura 4-32. El código de SWAP en el Mic-2.

Reimplementemos ahora esta secuencia en el Mic-3. Recuerde que la operación de la trayectoria de datos ahora requiere tres ciclos: uno para cargar A y B, uno para efectuar la operación y cargar C, y uno para escribir los resultados de vuelta en los registros. Llamaremos a cada una de estas partes un **micropaso**.

La implementación de **SWAP** para el Mic-3 se muestra en la figura 4-33. En el ciclo 1 iniciamos en **swap1** copiando SP en B. No importa lo que suceda en A porque para restar 1 a B ENA se invalida (vea la figura 4-2). Para simplificar, no mostraremos asignaciones que no

se usen. En el ciclo 2 realizamos la resta. En el ciclo 3 el resultado se almacena en MAR y se inicia la operación de lectura al final del ciclo 3 (una vez que se ha almacenado MAR). Puesto que las lecturas de memoria ahora tardan un ciclo, ésta no se completará sino hasta el final del ciclo 4, lo que se indica con la asignación a MDR en el ciclo 4. El valor en MDR no puede leerse antes del ciclo 5.

	Swap1	Swap2	Swap3	Swap4	Swap5	Swap6
Cic	MAR=SP-1;rd	MAR=SP	H=MDR;wr	MDR=TOS	MAR=SP-1wr	TOS=H;goto (MBR1)
1	B=SP					
2	C=B-1	B=SP				
3	MAR=C; rd	C=B				
4	MDR=mem	MAR=C				
5			B=MDR			
6			C=B	B=TOS		
7			H=C; wr	C=B	B=SP	
8			Mem=MDR	MDR=C	C=B-1	B=H
9					MAR=C; wr	C=B
10					Mem=MDR	TOS=C
11						goto (MBR1)

Figura 4-33. Implementación de SWAP en el Mic-3.

Volvamos ahora al ciclo 2. Ya podemos comenzar a dividir **swap2** en micropasos e iniciarlos. En el ciclo 2 podemos copiar **SP** en **B**, luego pasarlo por la ALU en el ciclo 3 y por último guardarlo en **MAR** en el ciclo 4. Hasta aquí todo va bien. Debe ser obvio que si podemos seguir a este ritmo, iniciando una nueva microinstrucción en cada ciclo, habremos triplicado la velocidad de la máquina. Esta ganancia se debe al hecho de que podemos emitir una nueva microinstrucción en cada ciclo de reloj, y el Mic-3 tiene tres veces más ciclos de reloj por segundo que el Mic-2. De hecho, hemos construido una CPU con conductos.

Lo malo es que nos topamos con un obstáculo en el ciclo 3. Nos gustaría comenzar a trabajar con **swap3**, pero lo primero que hace es pasar **MDR** por la ALU, y **MDR** no se habrá cargado desde la memoria antes del inicio del ciclo 5. La situación de que un micropaso no puede iniciarse porque está esperando un resultado que un micropaso anterior todavía no ha producido se llama **dependencia verdadera o dependencia RAW**. Es común llamar **peligros** a las dependencias. RAW significa "lectura después de escritura" (*Read After Write*) e indica que un micropaso quiere leer un registro que todavía no se escribe. La única acción sensata aquí es retrasar el inicio de **swap3** hasta que **MDR** esté disponible, en el ciclo 5. Detenerse para esperar un valor necesario se denomina **parar (stalling)**. Después, podremos seguir iniciando microinstrucciones en cada ciclo porque no hay más dependencias, aunque **swap6** apenas lo logra, ya que lee **H** en el ciclo después de que **swap3** lo escribe. Si **swap5** hubiera tratado de leer **H**, habría parado durante un ciclo.

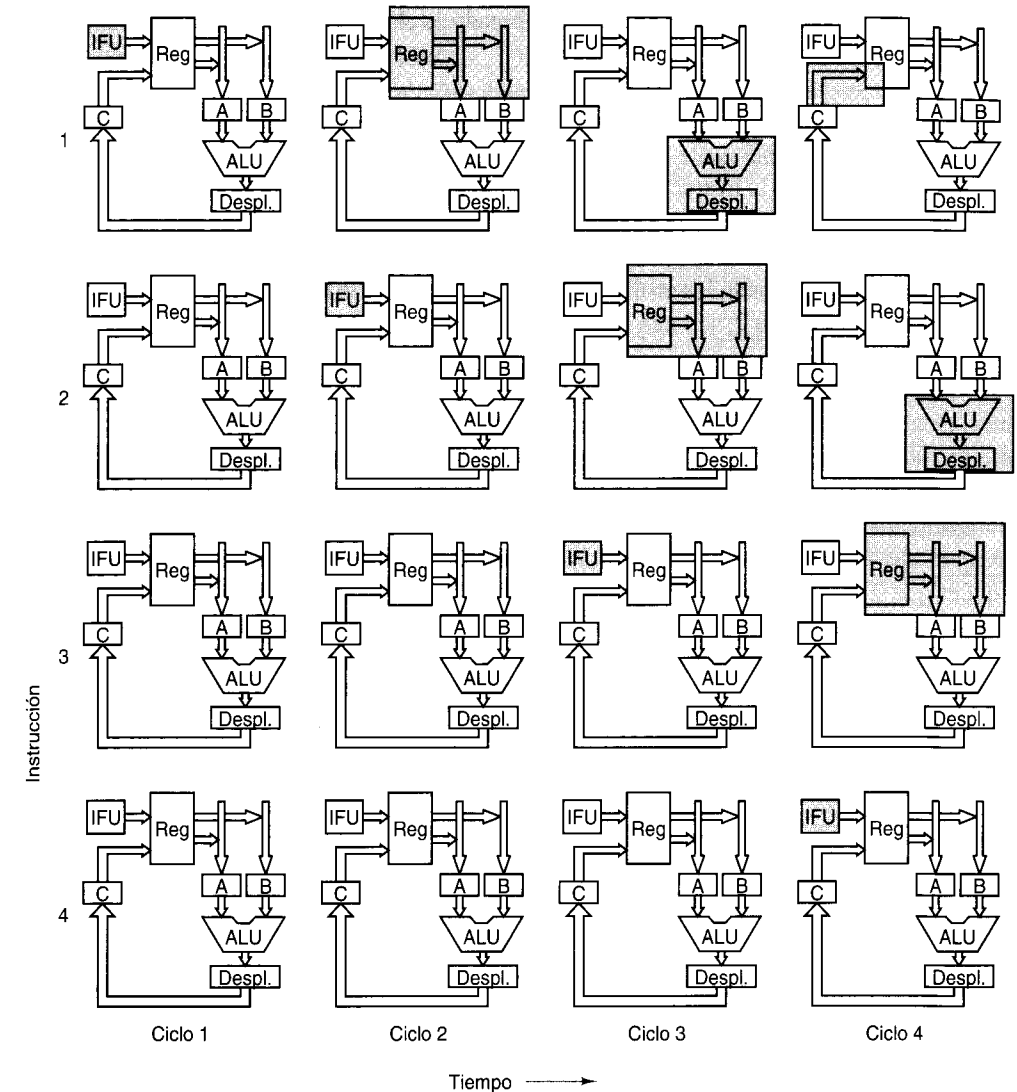


Figura 4-34. Ilustración gráfica del funcionamiento de una fila de procesamiento.

Aunque el programa Mic-3 tarda más ciclos que el Mic-2, se ejecuta más rápidamente. Si llamamos al tiempo de ciclo de Mic-3 ΔT ns, entonces el Mic-3 requiere 11 ΔT ns para ejecutar **SWAP**. En cambio, el Mic-2 tarda 6 ciclos de 3 ΔT cada uno, para un total de 18 ΔT . Los conductos han hecho más rápida la máquina, aunque tuvimos que parar una vez para evitar una dependencia.

El uso de filas de procesamiento es una técnica fundamental en todas las CPU modernas, así que es importante entenderla bien. En la figura 4-34 vemos a la trayectoria de datos de la figura 4-31 ilustrada gráficamente como conducto o interconexión. La primera columna representa lo que sucede durante el ciclo 1, la segunda columna representa el ciclo 2, y así

seguido de inmediato por otro código de operación. Cuando se detecta el prefijo **WIDE**, el siguiente byte se transforma en un código de operación ancho especial; por ejemplo **WIDE + ILOAD** se convierte en **WIDE_ILOAD**.

La unidad decodificadora envía el índice para la ROM de microoperaciones, que encontró en su tabla, al siguiente componente, la **unidad de cola de espera**. Esta unidad contiene algo de lógica y dos tablas internas, una en ROM y una en RAM. La ROM contiene el microprograma, y cada instrucción **IJVM** tiene cierto número de entradas consecutivas llamadas **microoperaciones**. Las entradas deben estar en orden, por lo que no se permiten trucos como que **wide_iloadd2** salte a **iloadd2** como en Mic-2. Cada secuencia **IJVM** debe detallarse totalmente, duplicando secuencias en algunos casos.

Las microoperaciones son similares a las microinstrucciones de la figura 4-5 excepto que no existen los campos **NEXT_ADDRESS** ni **JAM**, y se necesita un nuevo campo codificado para especificar la entrada del bus A. También se incluyen dos nuevos bits: **Final** y **Goto**. El bit **Final** se enciende en la última microoperación de cada secuencia de microoperaciones **IJVM** para marcarla. El bit **Goto** se enciende para marcar las microoperaciones que son microrramificaciones condicionales. Su formato es diferente del de las microoperaciones normales, pues consiste en los bits **JAM** y un índice para la ROM de microoperaciones. Las microinstrucciones que antes hacían algo con la trayectoria de datos y también efectuaban una microrramificación condicional (como **iflt4**) ahora tienen que dividirse en dos microoperaciones.

La unidad de cola de espera funciona como sigue. Recibe de la unidad decodificadora un índice para la ROM de microoperaciones. Con ese índice, busca la microoperación en la ROM y la copia en una cola interna. Luego copia en la cola la siguiente microoperación, y también la que sigue a ésta. Sigue haciendo esto hasta que llega a una que tiene el bit **Final** encendido (en 1). Copia esa microoperación y se detiene. Suponiendo que no encontró una microoperación con el bit **Goto** encendido y todavía tenga mucho espacio en la cola, la unidad de cola de espera enviará una señal de acuse a la unidad decodificadora. Cuando la unidad decodificadora detecta este acuse, envía el índice de la siguiente instrucción **IJVM** a la unidad de cola de espera.

De este modo, la sucesión de instrucciones **IJVM** en la memoria se convierte en última instancia en una sucesión de microoperaciones en una cola. Estas microoperaciones alimentan a los **MIR**, que envían las señales que controlan la trayectoria de datos. Sin embargo, ahora hay otro factor que es preciso considerar: los campos de cada microoperación no están activos al mismo tiempo. Los campos **A** y **B** están activos durante el primer ciclo. El campo **ALU** está activo durante el segundo ciclo, el campo **C** está activo durante el tercer ciclo, y las operaciones de memoria, si se requieren, se efectúan en el cuarto ciclo.

Para que esto funcione correctamente, hemos introducido cuatro **MIR** independientes en la figura 4-35. Al principio de cada ciclo de reloj (el tiempo Δw en la figura 4-3), **MIR3** se copia en **MIR4**, **MIR2** se copia en **MIR3**, **MIR1** se copia en **MIR2** y **MIR1** se carga con una nueva microoperación de la cola de microoperaciones. Luego cada **MIR** saca sus señales de control, pero sólo se usan algunas de ellas. Los campos **A** y **B** de **MIR1** sirven para seleccionar los registros que alimentan los latches **A** y **B**, pero el campo **ALU** de **MIR1** no se usa y no está conectado a ninguna otra cosa en la trayectoria de datos.

Un ciclo de reloj después, esta microoperación ha avanzado a **MIR2** y los registros que seleccionó ya están a salvo en los latches **A** y **B**, esperando nuevas aventuras. Su campo **ALU** se usa ahora para alimentar la **ALU**. En el siguiente ciclo, el campo **C** escribirá los resultados de vuelta en los registros. Después, pasará a **MIR4** e iniciará cualesquier operaciones de memoria que necesite utilizando el **MAR** que ya se cargó (y **MDR**, si se trata de una escritura).

Hay un último aspecto del Mic-4 que debemos tratar aquí: las microrramificaciones. Algunas instrucciones **IJVM**, como **IFLT**, necesitan ramificar condicionalmente con base en, digamos, el bit **N**. Cuando ocurre una microrramificación, la fila de procesamiento no puede continuar. Para resolver este problema, hemos añadido el bit **Goto** a cada microoperación. Cuando la unidad de cola de espera encuentra una microoperación con este bit encendido al copiarla en la cola, se da cuenta de que hay problemas en el futuro y no envía un acuse a la unidad decodificadora. El resultado es que la máquina se para en este punto hasta que se resuelve la microrramificación.

Es concebible que algunas instrucciones **IJVM** posteriores a la ramificación ya se hayan alimentado a la unidad decodificadora (pero no a la de cola de espera), ya que no devuelve una señal de acuse (o sea, de continuar) cuando encuentra una microoperación que tiene el bit **Goto** encendido. Se requieren hardware y mecanismos especiales para arreglar las cosas y volver al carril, pero este tema rebasa el alcance del presente libro. Cuando Edsger Dijkstra escribió su famosa carta "GOTO Statement Considered Harmful" ("El enunciado **GOTO** es perjudicial") (Dijkstra, 1968a), no sabía cuánta razón tenía.

Hemos llegado muy lejos desde el Mic-1. Éste era un sistema de hardware muy sencillo, con casi todo el control en software. El Mic-4 es un diseño con uso intensivo de filas de procesamiento, con siete etapas y hardware mucho más complejo. La fila de procesamiento se muestra de forma esquemática en la figura 4-36, donde los números en círculos corresponden a los de la figura 4-35. El Mic-4 prebusca automáticamente un flujo de bytes de la memoria, los decodifica como instrucciones **IJVM**, convierte éstas en una sucesión de microoperaciones mediante una ROM, y las encola para usarlas conforme se vayan necesitando. Las primeras tres etapas del conducto se pueden vincular con el reloj de la trayectoria de datos si se desea, pero no siempre habrá trabajo que efectuar. Por ejemplo, la **IFU** no puede alimentar un nuevo código de operación **IJVM** a la unidad decodificadora en cada ciclo porque las instrucciones **IJVM** tardan varios ciclos en ejecutarse y la cola se desbordaría rápidamente.

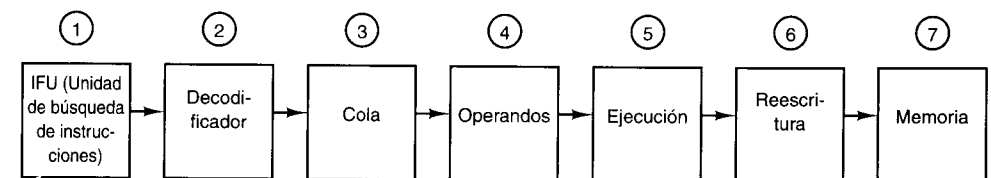


Figura 4-36. Fila de procesamiento del Mic-4.

En cada ciclo de reloj los **MIR** se desplazan hacia adelante y la microoperación que está al fondo de la cola se copia en **MIR1** para iniciar su ejecución. Luego las señales de control de

los cuatro MIR se distribuyen por la trayectoria de datos y hacen que se efectúen ciertas acciones. Cada MIR controla una parte distinta de la trayectoria de datos y por tanto diferentes micropasos.

En este diseño tenemos una CPU con gran uso de filas de procesamiento que permite pasos individuales muy cortos y por tanto una frecuencia de reloj alta. Muchas CPU se diseñan casi de la misma manera, sobre todo las que tienen que implementar un conjunto de instrucciones más viejo (CISC). Por ejemplo, la implementación del Pentium II es conceptualmente similar al Mic-4 en algunos aspectos, como veremos más adelante en este capítulo.

4.5 MEJORAMIENTO DEL DESEMPEÑO

Todos los fabricantes de computadoras quieren que sus sistemas operen con la mayor rapidez posible. En esta sección examinaremos varias técnicas avanzadas que se están investigando con miras a mejorar el desempeño de los sistemas (primordialmente de la CPU y la memoria). Debido a la naturaleza tan competitiva de la industria de las computadoras, el retraso entre que surgen ideas nuevas para hacer más rápida una computadora y que se incorporan en productos es sorprendentemente corto. Por tanto, casi todas las ideas que veremos aquí ya se están usando en una amplia variedad de productos.

Las ideas que trataremos se pueden clasificar a grandes rasgos en dos categorías: mejoras en la implementación y mejoras en la arquitectura. Las mejoras en la implementación son formas de construir una nueva CPU o memoria para hacer que el sistema opere más rápidamente sin cambiar la arquitectura. Modificar la implementación sin alterar la arquitectura implica que los programas viejos podrán ejecutarse en la nueva máquina, lo cual es un atractivo que eleva las ventas. Una forma de mejorar la implementación es usar un reloj más rápido, pero no es el único camino. Las mejoras en el desempeño logradas del 80386 al 80486, al Pentium, al Pentium Pro, hasta el Pentium II se deben a mejores implementaciones, pues la arquitectura se ha mantenido básicamente sin cambios en todos ellos.

Algunos tipos de mejoras sólo pueden lograrse modificando la arquitectura. A veces tales cambios son incrementales, como añadir nuevas instrucciones o registros, de modo que los programas viejos puedan seguir funcionando en los modelos nuevos. En este caso, para obtener el desempeño óptimo es preciso modificar el software, o al menos recompilarlo con un compilador nuevo que aproveche las nuevas características.

No obstante, cada cierto número de décadas los diseñadores se dan cuenta de que la vieja arquitectura ha dejado de ser útil y que la única forma de progresar es comenzar otra vez desde el principio. La revolución RISC de la década de los ochenta fue uno de esos avances; otro se está gestando actualmente. Examinaremos un ejemplo (el Intel IA-64) en el capítulo 5.

En el resto de esta sección estudiaremos cuatro técnicas para mejorar el desempeño de la CPU. Comenzaremos con tres mejoras de la implementación bien establecidas y luego pasaremos a uno que necesita un poco de apoyo arquitectónico para trabajar óptimamente. Las técnicas en cuestión son memoria caché, predicción de ramas, ejecución no en orden con cambio de nombre de registros y ejecución especulativa.

4.5.1 Memoria caché

Uno de los desafíos del diseño de computadoras en toda su historia ha sido proporcionar un sistema de memoria capaz de alimentar operandos al procesador tan rápidamente como éste puede procesarlos. El considerable aumento en la velocidad de los procesadores a últimas fechas no ha ido acompañado de un aumento correspondiente en la velocidad de las memorias. En comparación con las CPU, las memorias se han estado volviendo más lentas desde hace décadas. Dada la enorme importancia de la memoria primaria, esta situación ha limitado severamente el desarrollo de sistemas de alto desempeño, y ha estimulado investigaciones sobre formas de resolver el problema de memorias que son mucho más lentas que las CPU, problema que, en términos relativos, empeora cada año.

Los procesadores modernos someten a los sistemas de memoria a demandas abrumadoras, tanto en términos de latencia (lo que tarda en proporcionar un operando) como de ancho de banda (la cantidad de datos que se proporciona por unidad de tiempo). Lamentablemente, estos dos aspectos de un sistema de memoria son en gran medida opuestos. Muchas técnicas para aumentar el ancho de banda lo logran únicamente aumentando la latencia. Por ejemplo, las técnicas de filas de procesamiento que usamos en el Mic-3 se pueden aplicar a un sistema de memoria, manejando de forma eficiente múltiples solicitudes de memoria que se traslapan. Lo malo es que, como sucede con el Mic-3, el resultado es una mayor latencia para operaciones de memoria individuales. A medida que aumenta la velocidad de reloj de los procesadores, se vuelve más difícil crear un sistema de memoria capaz de proporcionar operandos en uno o dos ciclos de reloj.

Una forma de atacar este problema es incluir cachés. Como vimos en la sección 2.2.5, una caché contiene las palabras de memoria que se usaron más recientemente en una memoria pequeña y rápida, con lo que se agiliza el acceso a ellas. Si un porcentaje suficientemente grande de las palabras de memoria que se necesitan están en la caché, la latencia de memoria efectiva se puede reducir enormemente.

Una de las técnicas más eficaces para mejorar tanto el ancho de banda como la latencia se basa en el empleo de múltiples cachés. Una técnica fundamental que resulta muy eficaz es introducir cachés individuales para instrucciones y para datos, lo que se conoce como **caché dividida**. Esto tiene varias ventajas. Primera, las operaciones de memoria se pueden iniciar de forma independiente en cada caché, lo que duplica efectivamente el ancho de banda del sistema de memoria. Ésta es la razón por la que conviene tener dos puertos de memoria distintos, como hicimos en el Mic-1: cada puerto tiene su propia caché. Cabe señalar que cada caché tiene acceso independiente a la memoria principal.

Hoy día muchos sistemas de memoria son más complicados, y una caché adicional, llamada **caché de nivel 2**, podría residir entre las cachés de datos e instrucciones y la memoria principal. De hecho, podría haber tres o más niveles de caché si se requieren sistemas de memoria más sofisticados. En la figura 4-37 vemos un sistema con tres niveles de caché. El chip de la CPU mismo contiene una pequeña caché de instrucciones y una pequeña caché de datos, por lo regular de 16 KB a 64 KB. Luego está la caché de nivel 2, que no está en el chip de la CPU pero podría estar incluida en su paquete, junto al chip y conectada a él con una trayectoria de alta velocidad. Esta caché suele estar unificada (es decir, contiene una mezcla

de datos e instrucciones). Un tamaño típico para la caché L2 es de 512 KB a 1 MB. La caché de tercer nivel está en la tarjeta del procesador y consiste en unos cuantos megabytes de SRAM, que es mucho más rápida que la memoria principal DRAM. Las cachés generalmente son inclusivas: todo el contenido de la caché de nivel 1 está en la caché de nivel 2, y todo el contenido de la caché de nivel 2 está en la caché de nivel 3.

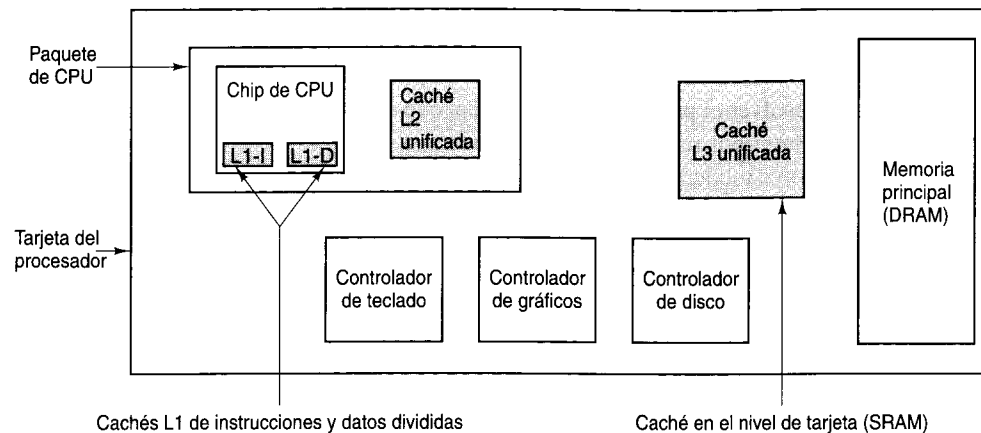


Figura 4-37. Sistema con tres niveles de caché.

Las cachés dependen de dos tipos de localidad de direcciones para lograr su objetivo. **Localidad espacial** es la observación de que es probable que en el futuro cercano se acceda a localidades de memoria con direcciones numéricamente similares a una localidad de memoria a la que se accedió recientemente. Las cachés aprovechan esta propiedad trayendo más datos de los que se solicitaron, con la expectativa de anticipar solicitudes futuras. Ocurre **localidad temporal** cuando se vuelve a acceder a localidades de memoria a las que se accedió recientemente. Esto puede ocurrir, por ejemplo, con localidades de memoria cercanas al tope de la pila, o instrucciones dentro de un ciclo. La localidad temporal se aprovecha primordialmente en los diseños de caché al tomar la decisión de qué debe desecharse cuando no se encuentra algo en ella (fallo de caché). Muchos algoritmos de reemplazo de caché aprovechan la localidad temporal desechando las entradas a las que no se ha accedido recientemente.

Todas las cachés usan el modelo que sigue. La memoria principal se divide en bloques de tamaño fijo llamados **líneas de caché**. Una línea de caché por lo regular consta de 4 a 64 bytes consecutivos. Las líneas se numeran consecutivamente a partir del 0, de modo que si el tamaño de línea es de 32 bytes la línea 0 comprende los bytes del 0 al 31, la línea 1 comprende los bytes del 32 al 63, etc. En cualquier instante dado, algunas líneas están en la caché. Cuando se hace referencia a la memoria, el circuito controlador de la caché verifica si la palabra a la que se hizo referencia está actualmente en la caché. Si es así, puede usar el valor que está ahí y evitarse un viaje a la memoria principal. Si la palabra no está ahí, se elimina una línea de la caché y la línea requerida se busca de la memoria o de una caché de nivel más

bajo para reemplazarla. Existen muchas variaciones de este esquema, pero todas se basan en la idea de mantener las líneas más usadas en la caché el mayor tiempo posible, a fin de maximizar el número de referencias a la memoria que se pueden satisfacer con la caché.

Cachés con mapeo directo

La caché más sencilla es la llamada **caché con mapeo directo**. En la figura 4-38(a) se muestra un ejemplo de caché con mapeo directo de un solo nivel. Esta caché contiene 2048 entradas. Cada entrada (renglón) de la caché puede contener exactamente una línea de caché de la memoria principal. Con un tamaño de línea de caché de 32 bytes (para este ejemplo), la caché puede contener 64 KB. Cada entrada de caché consta de tres partes:

1. El bit de **validez** indica si la entrada contiene o no datos válidos. Cuando el sistema se arranca (inicia), todas las entradas se marcan como no válidas.
2. El campo **Etiqueta (tag)** consiste en un valor único de 16 bits que identifica la línea de memoria de la cual provienen los datos.
3. El campo **Datos** contiene una copia de los datos de la memoria. Este campo contiene una línea de caché de 32 bytes.

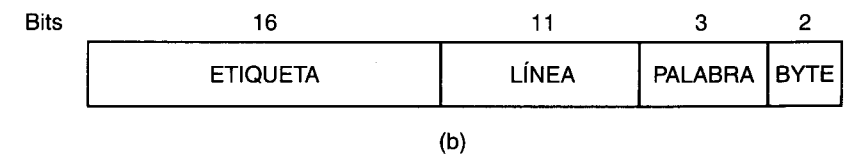
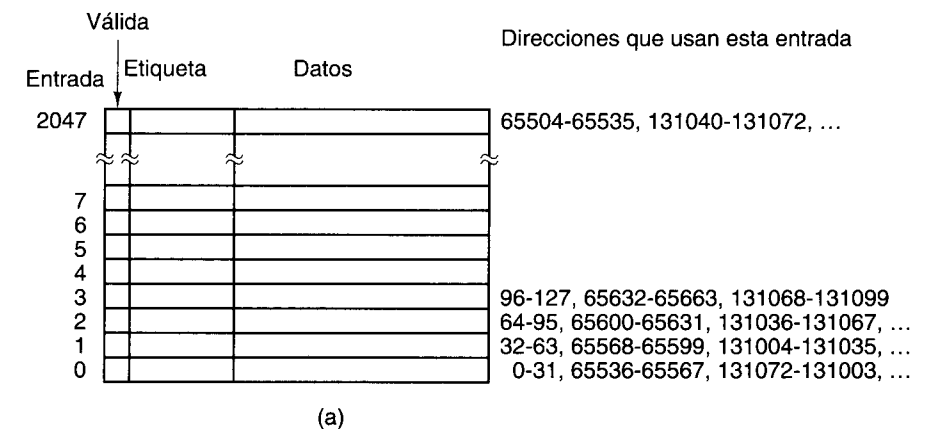


Figura 4-38. (a) Caché con mapeo directo. (b) Dirección virtual de 32 bits.

En una caché con mapeo directo, una palabra de memoria dada se puede almacenar en un y sólo un lugar dentro de la caché. Dada una dirección de memoria, sólo hay un lugar en el

que se puede buscar en la caché. Si no está ahí, entonces no está en la caché. Para almacenar y recuperar datos de la caché, la dirección se divide en cuatro componentes, como se muestra en la figura 4-38(b):

1. El campo **ETIQUETA** corresponde a los bits **Etiqueta** almacenados en una entrada de caché.
2. El campo **LÍNEA** indica cuál caché de entrada contiene los datos correspondientes, si están presentes.
3. El campo **PALABRA** indica a cuál palabra se hace referencia dentro de la línea.
4. El campo **BYTE** generalmente no se usa, pero si sólo se solicita un byte indica cuál byte dentro de la palabra se necesita. En el caso de una caché que sólo proporciona palabras de 32 bits, este campo siempre es 0.

Cuando la CPU produce una dirección de memoria, el hardware extrae los 11 bits **LÍNEA** de la dirección y los usa como índice dentro de la caché para encontrar una de las 2048 entradas. Si esa entrada es válida, se comparan el campo **ETIQUETA** de la dirección de memoria y el campo **Etiqueta** de la entrada de caché; si coinciden, la entrada de caché contiene la palabra que se solicitó, situación que se denomina **acierto de caché**. Si se acierta, la palabra que se desea leer se puede tomar de la caché, y ya no es necesario ir a la memoria. Sólo se extrae de la entrada de caché la palabra que se necesita; el resto de la entrada no se usa. Si la entrada de caché no es válida o si las etiquetas no coinciden, la entrada requerida no está presente en el caché, situación que se denomina **fallo de caché**. En este caso, la línea de caché de 32 bytes se trae de la memoria y se guarda en la entrada de caché, sustituyendo lo que estaba ahí. Sin embargo, si la entrada que estaba antes en la caché se modificó después de cargarse, deberá volver a escribirse en la memoria principal antes de desecharse.

A pesar de la complejidad de la decisión, el acceso a una palabra requerida puede ser notablemente rápido. Tan pronto como se conoce la dirección, se conoce la ubicación exacta de la palabra *si está presente en la caché*. Esto implica que es posible leer la palabra de la caché y proporcionarla al procesador al mismo tiempo que se está determinando si era la palabra correcta (comparando etiquetas). Así, el procesador realmente recibe una palabra de la caché simultáneamente, o incluso antes de saber si la palabra es la que se solicitó.

Este esquema de correspondencia coloca líneas de memoria consecutivas en entradas de caché consecutivas. De hecho, se pueden almacenar hasta 64K bytes de datos contiguos en la caché. Sin embargo, dos líneas que difieran en sus direcciones por exactamente 64K (65,536 bytes) o cualquier múltiplo entero de ese número no podrán guardarse en la caché al mismo tiempo (porque tienen el mismo valor de **LÍNEA**). Por ejemplo, si un programa accesa a datos que están en la localidad *X* y luego ejecuta una instrucción que necesita datos que están en la localidad *X + 65,536* (o cualquier otra localidad dentro de la misma línea), la segunda instrucción hará que la entrada de caché se vuelva a cargar, sobrescribiendo lo que estaba ahí. Si esto sucede con suficiente frecuencia, el funcionamiento de la caché puede ser deficiente. De hecho, el comportamiento de una caché en el peor de los casos, es peor que si no hubiera caché, ya que cada operación de memoria implica leer toda una línea de caché en lugar de una sola palabra.

Las cachés con mapeo directo son las más comunes, y son muy eficaces porque puede hacerse que colisiones como la que acabamos de describir ocurran muy raras veces, o nunca. Por ejemplo, un compilador muy inteligente puede tener en cuenta las colisiones de caché al colocar las instrucciones y los datos en la memoria. Cabe señalar que el caso específico que acabamos de describir no ocurriría en un sistema con cachés de instrucciones y de datos separados, porque las solicitudes en colisión serían atendidas por diferentes cachés. Éste es un segundo beneficio de tener dos cachés en lugar de una: más flexibilidad para manejar patrones de memoria en conflicto.

Cachés asociativas por conjuntos

Como ya mencionamos, muchas líneas de memoria distintas compiten por las mismas ranuras de caché. Si un programa que usa la caché de la figura 4-38(a) usa mucho las palabras que están en las direcciones 0 y 65,536, habrá conflictos constantes, y cada referencia podría expulsar a la otra de la caché. Una solución de este problema es permitir dos o más líneas en cada entrada de caché. Una caché con *n* posibles entradas para cada dirección se denomina **caché asociativa por conjuntos de *n* vías**. En la figura 4-39 se ilustra una caché asociativa de cuatro vías.

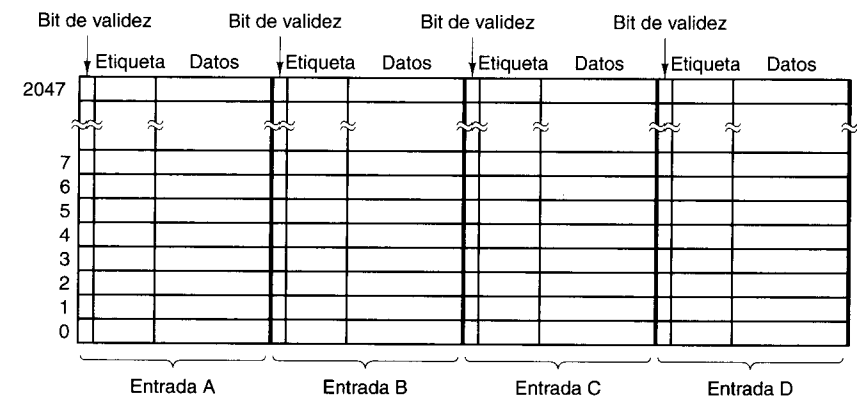


Figura 4-39. Caché asociativa de cuatro vías.

Una caché asociativa por conjuntos es inherentemente más complicada que una de mapeo directo porque, si bien puede calcularse la entrada de caché que debe examinarse a partir de la dirección de memoria a la que se hizo referencia, es preciso examinar un conjunto de *n* entradas de caché para ver si está presente la línea requerida. No obstante, la experiencia muestra que las cachés de dos y cuatro vías funcionan lo bastante bien como para que valga la pena la adición de estos circuitos.

El uso de una caché asociativa por conjuntos presenta al diseñador una opción. Cuando se debe traer una nueva línea a la caché, ¿cuál de las entradas actuales debe desecharse? Desde luego, la decisión óptima requiere conocer el futuro, pero un algoritmo excelente para casi todas las aplicaciones es el de **menos recientemente utilizada (LRU, Least Recently**

Used). Este algoritmo mantiene un ordenamiento de cada conjunto de localidades a las que podría accederse desde una localidad de memoria dada. Siempre que se accede a alguna de las líneas presentes, el algoritmo actualiza la lista, marcando esa entrada como la accesada más recientemente. Cuando se hace necesario reemplazar una entrada, se desecha la que está al final de la lista: aquella a la que se accedió menos recientemente.

Llevado al extremo, es posible tener una caché de 2048 vías que contiene un solo conjunto de 2048 entradas de líneas. Aquí todas las direcciones de memoria corresponden al mismo conjunto único, así que la búsqueda implica comparar la dirección con las 2048 etiquetas de la caché. Cabe señalar que en este caso cada entrada necesitaría lógica de comparación de etiquetas. Puesto que el campo **LÍNEA** tiene longitud 0, el campo **ETIQUETA** es toda la dirección excepto los campos **PALABRA** y **BYTE**. Además, cuando se reemplaza una línea de caché, las 2048 posiciones son posibles candidatas para el reemplazo. Mantener una lista ordenada de 2048 entradas requiere mucha contabilidad y hace impráctico el reemplazo LRU. (Recuerde que esta lista tiene que actualizarse en cada operación de memoria, no sólo cuando no hay un acierto.) Resulta sorprendente que las cachés con alta asociatividad no mejoran el desempeño mucho más que las de baja asociatividad en la mayor parte de las circunstancias, y en algunos casos tienen un desempeño peor. Por estas razones, casi nunca se usa una asociatividad mayor que de cuatro vías.

Por último, las escrituras presentan un problema especial para las cachés. Cuando un procesador escribe una palabra, y la palabra está en la caché, obviamente se debe actualizar la palabra o bien desecharse la entrada de caché. Casi todos los diseños actualizan la caché. Pero, ¿y la actualización de la copia que está en la memoria principal? Esta operación puede diferirse hasta que la línea de caché esté lista para ser reemplazada por el algoritmo LRU. Esta decisión es difícil, y ninguna de las opciones ofrece una ventaja clara. Actualizar inmediatamente la entrada en la memoria principal se conoce como **escritura a través**. Este enfoque suele ser más fácil de implementar y más confiable, puesto que la memoria siempre está al día; esto es útil, por ejemplo, si ocurre un error y se hace necesario recuperar el estado de la memoria. Lo malo es que casi siempre requiere más tráfico de escritura hacia la memoria, por lo que las implementaciones más complejas tienden a utilizar la alternativa, llamada **escritura diferida** o **reescritura**. Hay un problema relacionado que debemos resolver cuando escribimos: ¿qué sucede si hay que escribir en una localidad que no está actualmente en caché? ¿Se deberán traer los datos a la caché o simplemente escribirse en la memoria? Una vez más, ninguna de las respuestas es siempre la mejor. Casi todos los diseños que difieren las escrituras en la memoria tienden a traer los datos viejos a la caché cuando no están ya en ella, técnica llamada **asignación de escritura**. En cambio, casi ningún diseño que practique la escritura a través asignará una entrada en una operación de escritura porque tal opción complicaría un diseño por lo demás sencillo. La asignación de escritura es mejor sólo si se repiten las escrituras a la misma palabra o a palabras que están en la misma línea de caché.

4.5.2 Predicción de ramificaciones

Las computadoras modernas usan filas de procesamiento. La fila de procesamiento de la figura 4-35 tiene siete etapas; las computadoras del extremo superior a veces tienen filas de

procesamiento de diez etapas o más. Las filas de procesamiento funcionan mejor con código lineal, pues así la unidad de obtención puede leer palabras consecutivas de la memoria y enviarlas a la unidad decodificadora antes de que se necesiten.

El único problema menor con este maravilloso modelo es que no se ajusta para nada a la realidad. Los programas no son secuencias lineales de código; están llenos de instrucciones de ramificación. Considere los sencillos enunciados de la figura 4-40(a). Una variable, *i*, se compara con 0 (lo que probablemente sea la prueba más común en la práctica). Dependiendo del resultado, se asigna a otra variable, *k*, uno de dos posibles valores.

if (i == 0)	CMP i,0	; comparar i con 0
k = 1;	BNE Else	; saltar a Else si no es igual
else	Then: MOV k,1	; poner 1 en k
k = 2;	BR Next	; salto incondicional a Next
	Else: MOV k,2	; poner 2 en k
	Next:	
(a)	(b)	

Figura 4-40. (a) Fragmento de programa. (b) Su traducción a un lenguaje de ensamblador genérico.

En la figura 4-40(b) se muestra una posible traducción a lenguaje ensamblador. Estudiaremos el lenguaje ensamblador más adelante; los detalles no son importantes ahora, pero dependiendo de la máquina y el compilador es probable que se produzca código similar al de la figura. La primera instrucción compara *i* con 0. La segunda ramifica a la etiqueta *Else* (el inicio de la cláusula **else**) si *i* no es 0. La tercera instrucción asigna 1 a *k*. La cuarta instrucción ramifica al código para la siguiente instrucción. El compilador puso una etiqueta cómoda ahí, *Next*, para tener a dónde ramificar. La quinta instrucción asigna 2 a *k*.

Lo que debemos observar aquí es que dos de las cinco instrucciones son ramificaciones. Además, una de ellas, **BNE**, es una ramificación condicional (una rama que se sigue si y sólo si se cumple alguna condición, en este caso, que los dos operandos del **CMP** previo sean iguales). La secuencia de código lineal más larga aquí es de dos instrucciones. Por consiguiente, traer instrucciones a gran velocidad para alimentar los conductos no es fácil.

A primera vista, podría parecer que las ramificaciones incondicionales, como la instrucción **BR Next** de la figura 4-40(b) no presentan un problema. Después de todo, no hay ambigüedad respecto a dónde ir. ¿Por qué la unidad de obtención no puede seguir leyendo instrucciones a partir de la dirección objetivo (el lugar a donde se saltará)?

El problema radica en la naturaleza de los conductos. En la figura 4-35, por ejemplo, vemos que la decodificación de instrucciones se efectúa en la segunda etapa. Así, la unidad de obtención tiene que decidir de dónde traerá bytes a continuación antes de saber qué tipo de instrucción acaba de traer. Un ciclo después, la unidad de búsqueda podrá enterarse de que dicha instrucción fue una ramificación incondicional, pero para entonces ya empezó a buscar la instrucción que seguía a la ramificación incondicional. Es por esto que una proporción considerable de las máquinas que usan filas de procesamiento (como el UltraSPARC II) tienen la propiedad de que se ejecuta la instrucción *que sigue* a una ramificación incondicional,

aunque lógicamente no debería hacerse. La posición que sigue a una ramificación se llama **ranura de retraso**. En Pentium II [y la máquina de la figura 4-40(b)] no tienen esta propiedad, pero la complejidad interna que se requiere para superar este problema suele ser enorme. Un compilador optimante tratará de encontrar alguna instrucción útil que pueda colocar en la ranura de retraso, pero es común que no haya nada disponible, y el compilador se verá obligado a insertar una instrucción NOP ahí. Ello asegura que el programa sea correcto, pero lo hace más grande y lento.

Aunque las ramificaciones incondicionales son molestas, las condicionales son peores. No sólo tienen ranuras de retraso, sino que ahora la unidad de búsqueda no sabe de dónde tendrá que leer sino hasta mucho más adelante en la fila de procesamiento. Las primeras máquinas con fila de procesamiento simplemente **paraban** hasta que se sabía si se iba a tomar la rama o no. Parar durante tres o cuatro ciclos en cada ramificación condicional, sobre todo si el 20% de las instrucciones son ramificaciones condicionales, arruina el desempeño.

Por ello, lo que la mayor parte de las máquinas hace cuando llega a una rama condicional es predecir si se va a tomar o no. Sería bueno que pudiéramos conectar una bola de cristal a una ranura PCI libre para que nos ayudara con la predicción, pero hasta ahora este enfoque no ha sido fructífero.

Ante la falta de tal periférico, se han ideado varias formas de efectuar la predicción. Una muy sencilla es la siguiente: suponer que se tomarán todas las ramas condicionales hacia atrás y que no se tomará ninguna rama condicional hacia adelante. El razonamiento que justifica la primera parte es que es común encontrar ramas hacia atrás al final de un ciclo. Casi todos los ciclos se ejecutan varias veces, así que adivinar que se tomará una rama hacia atrás, hacia el principio del ciclo, es una buena apuesta.

La segunda parte no está tan bien fundamentada. Ocurren algunas ramas hacia adelante cuando se detectan condiciones de error en software (por ejemplo, no se puede abrir un archivo). Los errores son raros, así que casi todas las ramas asociadas a ellos no se tomarán. Desde luego, hay muchas ramas hacia adelante que no están relacionadas con el manejo de errores, así que la tasa de aciertos no es tan buena como con las ramas hacia atrás. Aunque no es una maravilla, esta regla al menos es mejor que nada.

Si se predice correctamente una rama, no hay nada especial que hacer. La ejecución simplemente continúa en la dirección objetivo. El problema surge cuando se predice erróneamente una rama. Averiguar a dónde hay que ir e ir ahí no es difícil. La parte difícil es anular instrucciones que ya se ejecutaron y no debieron haberse ejecutado.

Hay dos formas de atacar el problema. La primera es permitir que las instrucciones que se adquieren después de una ramificación condicional predicha se ejecuten en tanto no traten de modificar el estado de la máquina (por ejemplo, guardar algo en un registro). En lugar de sobrescribir el registro, el valor calculado se almacena en un registro de borrador (secreto) y sólo se copia en el registro real una vez que se sabe que la predicción fue correcta. La segunda técnica es guardar el valor de cualquier registro que esté a punto de sobrescribirse (digamos en un registro de borrador secreto) para que la máquina pueda revertirse al estado en que estaba cuando se llegó a la rama predicha erróneamente. Ambas soluciones son complejas y requieren una contabilización muy estricta para funcionar bien. Y si se llega a una segunda ramificación condicional antes de que se sepa si la primera se predijo correctamente, las cosas pueden ponerse difíciles.

Predicción dinámica de ramificaciones

Obviamente, es importantísimo que las predicciones sean correctas, ya que permite a la CPU trabajar a toda velocidad. Por ello, muchas investigaciones actuales buscan mejorar los algoritmos de predicción (por ejemplo, Driesen y Holzle, 1998; Juan *et al.*, 1998; Pan *et al.*, 1992; Sechrest *et al.*, 1996; Sprangle *et al.*, 1997; y Yeh y Patt, 1991). Una estrategia es que la CPU mantenga una tabla histórica (en hardware especial) en la que registra las ramificaciones condicionales conforme ocurren, para poder consultarlas cuando vuelvan a ocurrir. La versión más simple de este esquema se muestra en la figura 4-41(a). Aquí la tabla histórica contiene una entrada por cada instrucción de ramificación condicional. La entrada contiene la dirección de la instrucción de ramificación junto con un bit que indica si se tomó la última vez que se ejecutó. Cuando se utiliza este esquema, la predicción es simplemente que la ramificación tomará la misma trayectoria que adoptó la vez anterior. Si la predicción es errónea, el bit de la tabla histórica se cambia.

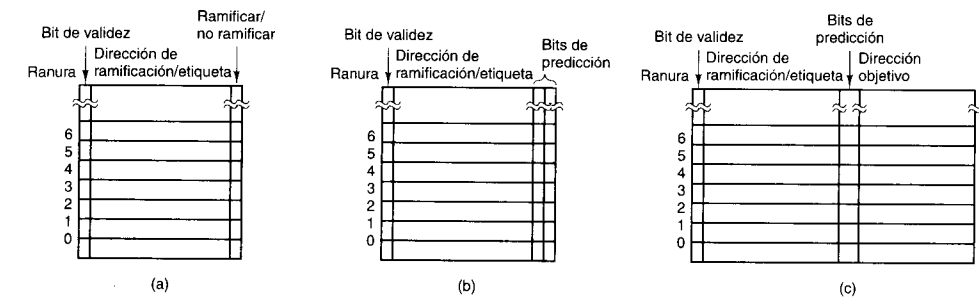


Figura 4-41. (a) Historia de ramificación de 1 bit. (b) Historia de ramificación de 2 bits. (c) Correspondencia entre la dirección de la instrucción de ramificación y la dirección objetivo.

Hay varias formas de organizar la tabla histórica; de hecho, son precisamente las técnicas que se usan para organizar una caché. Considere una máquina con instrucciones de 32 bits que están alineadas por palabras de modo que los dos bits de orden bajo de cada dirección de memoria son 00. Con una tabla histórica de mapeo directo que contiene 2^n entradas, los $n + 2$ bits de orden bajo de una instrucción de ramificación se pueden extraer y desplazar hacia la derecha dos bits. Este número de n bits se puede usar como índice de la tabla histórica, donde se verifica si la dirección guardada coincide con la dirección de la ramificación. Al igual que con una caché, no hay necesidad de guardar los $n + 2$ bits de orden bajo, así que pueden omitirse (es decir, sólo se guardan los bits de dirección superiores, la etiqueta). Si hay un acierto, se usa el bit de predicción para predecir la ramificación. Si está presente la etiqueta equivocada o la entrada no es válida, hay un fallo, igual que en una caché. En este caso puede usarse la regla de la rama hacia adelante o hacia atrás.

Si la tabla de historial de ramificación tiene, digamos, 4096 entradas, las ramificaciones en las direcciones 0, 16384, 32768, ... chocarán, en un problema análogo al que se presenta

en una caché. Se puede aplicar la misma solución: una entrada asociativa de 2, 4 o n vías. Al igual que en una caché, el caso limitante es una sola entrada asociativa de n vías, que requiere una búsqueda con plena asociatividad.

Si la tabla es lo bastante grande y tiene suficiente asociatividad, este esquema funciona bien en la mayor parte de las situaciones. Sin embargo, siempre se presenta un problema sistemático. Cuando por fin se sale de un ciclo, la predicción de la ramificación final siempre será errónea. Lo que es peor, el error en la predicción hará que se modifique el bit de la tabla histórica para indicar una predicción futura de “no tomar la rama”. La siguiente vez que se ingrese en el ciclo, la predicción de ramificación al final de la primera iteración será errónea. Si el ciclo está dentro de un ciclo exterior, o en un procedimiento que se invoca a menudo, este error podría ocurrir con mucha frecuencia.

Para eliminar esta falsa predicción, podríamos dar una segunda oportunidad a la entrada de la tabla. Con este método, la predicción sólo se cambia después de dos predicciones incorrectas consecutivas. Esto requiere dos bits de predicción en la tabla histórica, uno para lo que “se supone” que debe hacer la ramificación y otro para lo que hizo la última vez, como se muestra en la figura 4-41(b).

Una forma un poco distinta de ver este algoritmo es como una máquina de estados finitos con cuatro estados, la cual se ilustra en la figura 4-42. Después de una serie de predicciones de “no tomar la rama” acertadas, la FSM estará en el estado 00 y su predicción para la próxima vez será “no tomar la rama”. Si esa predicción es errónea, pasará al estado 01, pero su predicción para la próxima vez seguirá siendo “no tomar la rama”. Sólo si esta predicción es errónea la FSM pasará al estado 11 y comenzará a predecir “tomar la rama” todo el tiempo. Efectivamente, el bit de la izquierda del estado es la predicción, y el de la derecha es lo que se hizo la vez anterior. Aunque este diseño usa sólo 2 bits de historial, existen diseños que siguen la pista a 4 u 8 bits de historial.

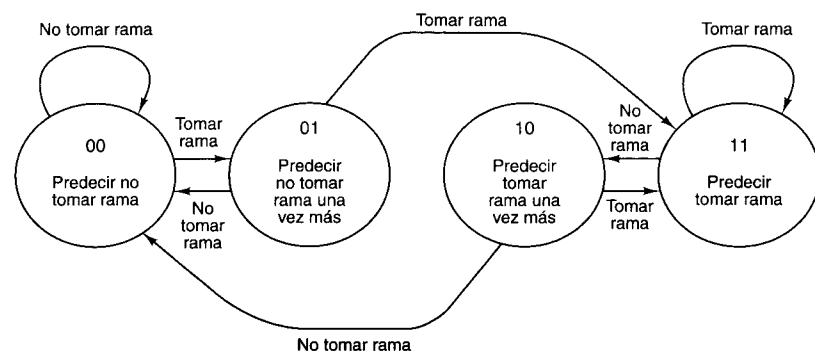


Figura 4-42. Máquina de estados finitos de dos bits para predecir ramificaciones.

Ésta no es nuestra primera FSM. La figura 4-28 también era una FSM. De hecho, todos nuestros microprogramas pueden considerarse como máquinas de estados finitos, puesto que cada línea representa un estado específico en el que puede estar la máquina, con transiciones bien definidas a un conjunto finito de estados distintos. Las FSM se usan mucho en todos los aspectos del diseño de hardware.

Hasta aquí hemos supuesto que se conoce el objetivo de cada ramificación condicional, casi siempre como una dirección explícita a la cual saltar (contenida en la instrucción misma) o como distancia relativa desde la instrucción actual (por ejemplo, un número con signo que se suma al contador de programa). Con frecuencia este supuesto es válido, pero algunas instrucciones de ramificación condicional calculan la dirección objetivo realizando operaciones aritméticas con registros, y luego saltan a la dirección calculada. Incluso si la FSM de la figura 4-42 predice con exactitud la rama que se tomará, de nada sirve tal predicción si no se conoce la dirección objetivo. Una forma de manejar esta situación es guardar en la tabla histórica la dirección a la que se saltó la última vez, como se muestra en la figura 4-41(c). De este modo, si la tabla dice que la última vez que tomamos la rama que sale de la dirección 516 nos llevó a la dirección 4000, y lo que se predice es “tomar la rama”, podremos suponer que de nuevo se saltará a 4000.

Una estrategia de predicción de ramificaciones distinta consiste en llevar un registro de si se tomaron o no las últimas k ramas condicionales que se encontraron, sin importar de cuál instrucción se trató (Pan *et al.*, 1992). Este número de k bits, que se guarda en el **registro de desplazamiento histórico de ramificación**, se compara en paralelo con todas las entradas de una tabla histórica con una clave de k bits y, si coincide, se usa la predicción que está ahí. Aunque parezca sorprendente, esta técnica funciona muy bien.

Predicción de ramificación estática

Todas las técnicas de predicción de ramificaciones que hemos visto hasta ahora son dinámicas, es decir, se ejecutan sobre la marcha durante la ejecución del programa. Además, se adaptan al comportamiento actual del programa, lo cual es bueno. La desventaja es que requieren hardware especializado y costoso y chips muy complejos.

Una estrategia diferente es hacer que el compilador nos ayude. Cuando el compilador ve un enunciado como

```
for (i = 0; i < 1000000; i++) { ... }
```

sabe muy bien que casi todas las veces se tomará la rama al final del ciclo. Si hubiera alguna forma de decírselo al hardware, se ahorraría mucho trabajo.

Aunque éste es un cambio arquitectónico (y no sólo una cuestión de implementación), algunas máquinas, como la UltraSPARC II, tienen un segundo conjunto de instrucciones de ramificación condicional además de las normales (que se necesitan por compatibilidad hacia atrás). Las instrucciones nuevas contienen un bit en el que el compilador puede especificar que cree que la rama se tomará (o no se tomará). Cuando se llega a una de estas instrucciones, la unidad de búsqueda hace lo que se le dice. Además, no hay necesidad de desperdiciar espacio valioso de la tabla histórica de ramificación para estas instrucciones, lo que reduce el número de conflictos en la tabla.

Nuestra última técnica de predicción de ramificaciones se basa en crear perfiles (Fisher y Freudenberger, 1992). Esta técnica también es estática, pero en lugar de pedir al compilador que trate de adivinar cuáles ramas se tomarán y cuáles no, el programa se ejecuta realmente (por lo regular en un simulador) y se captura el comportamiento de ramificación. Esta infor-

mación se alimenta al compilador, que entonces usa las instrucciones de ramificación condicional especiales para indicar al hardware qué debe hacer.

4.5.3 Ejecución fuera de orden y cambio de nombres de registros

Casi todas las CPU modernas usan filas de procesamiento y son superescalares, como se muestra en la figura 2-6. Lo que esto significa en general es que existe una unidad de búsqueda que saca palabras de instrucciones de la memoria antes de que se necesiten, a fin de alimentar una unidad decodificadora. Ésta emite las instrucciones decodificadas a las unidades funcionales apropiadas para su ejecución. En algunos casos, la unidad decodificadora podría desglosar instrucciones individuales en microoperaciones antes de emitir las a las unidades funcionales, dependiendo de las capacidades de estas últimas.

Es evidente que el diseño de la máquina es el más sencillo posible si todas las instrucciones se ejecutan en el orden en que se traen de la memoria (suponiendo por el momento que el algoritmo de predicción de ramificaciones nunca se equivoca). Sin embargo, la ejecución en orden no siempre da lugar a un desempeño óptimo debido a las dependencias entre instrucciones. Si una instrucción requiere un valor calculado por la instrucción anterior, la segunda instrucción no podrá iniciar su ejecución hasta que la primera haya producido el valor requerido. En esta situación (una dependencia RAW), la segunda instrucción tiene que esperar. Existen también otros tipos de dependencias, como pronto veremos.

En un intento por superar estos problemas y lograr un mejor desempeño, algunas CPU permiten saltarse por el momento las instrucciones dependientes para llegar a instrucciones futuras que no sean dependientes. Sobre decir que el algoritmo de planificación de instrucciones interno que se use debe producir el mismo efecto que se obtendría si el programa se ejecutara en el orden en que está escrito. A continuación demostraremos cómo funciona el reordenamiento de instrucciones empleando un ejemplo detallado.

Para ilustrar la naturaleza del problema, comenzaremos con una máquina que siempre emite las instrucciones en el orden del programa y también requiere que terminen su ejecución en el orden del programa. La importancia de la segunda condición se hará evidente más adelante.

Nuestra máquina de ejemplo tiene ocho registros que el programador puede ver, R0 a R7. Todas las instrucciones aritméticas usan tres registros: dos para los operandos y uno para el resultado, igual que el Mic-4. Supondremos que si una instrucción se decodifica en el ciclo *n*, su ejecución inicia en el ciclo *n* + 1. En el caso de una instrucción sencilla, como una suma o resta, la escritura del resultado en el registro de destino ocurre al final del ciclo *n* + 2. Si la instrucción es más complicada, como una multiplicación, la escritura ocurre al final del ciclo *n* + 3. Para que el ejemplo sea realista, permitiremos a la unidad decodificadora emitir hasta dos instrucciones en cada ciclo de reloj. Las CPU superescalares comerciales a menudo pueden emitir cuatro o incluso seis instrucciones por ciclo de reloj.

Nuestra secuencia de ejecución de ejemplo se muestra en la figura 4-43. Aquí la primera columna da el número del ciclo, y la segunda, el número de la instrucción. La tercera columna es una lista de las instrucciones decodificadas. La cuarta indica cuál instrucción se está emitiendo (con un máximo de dos por ciclo de reloj). La quinta indica cuál instrucción se

retiró (terminó). Recuerde que en este ejemplo estamos exigiendo tanto emisión en orden como terminación en orden, de modo que la instrucción *k* + 1 no puede emitirse hasta que se ha emitido la instrucción *k*, y la instrucción *k* + 1 no puede retirarse (después de efectuarse la escritura de resultados en el registro de destino) hasta que se ha retirado la instrucción *k*. A continuación explicaremos las otras 16 columnas.

Cic.	#	Decodif.	Emit.	Ret.	Registros leídos								Registros escritos							
					0	1	2	3	4	5	6	7	0	1	2	3	4	5	6	7
1	1	R3=R0+R1	1		1	1										1				
	2	R4=R0+R2	2		2	1	1									1	1			
2	3	R5=R0+R1	3		3	2	1									1	1	1		
	4	R6=R1+R4	–		3	2	1									1	1	1		
3					3	2	1									1	1	1		
4				1	2	1	1										1	1		
				2	1	1												1		
				3																
5	5	R7=R1+R2	4			1			1										1	
			5			2	1		1										1	1
6	6	R1=R0–R2	–			2	1		1										1	1
7				4		1	1													1
8				5																
9	7	R3=R3+R1	6		1		1						1							
			7		1	1	1	1					1		1					
10					1	1	1	1					1		1					
11				6		1		1								1				
12				7																
13	8	R1=R4+R4	8						2				1							
14									2				1							
15				8																

Figura 4-43. Funcionamiento de una CPU superescalar con emisión en orden y terminación en orden.

Después de decodificar una instrucción, la unidad decodificadora tiene que decidir si se puede emitir de inmediato o no. Para tomar esta decisión, la unidad necesita conocer la situación de todos los registros. Si, por ejemplo, la instrucción en curso necesita un registro cuyo valor todavía no se ha calculado, la instrucción no podrá emitirse y la CPU deberá parar.

Seguimos la pista al uso de registros con un dispositivo llamado **marcador**, que se usó por primera vez en la CDC 6600. El marcador tiene un pequeño contador para cada registro